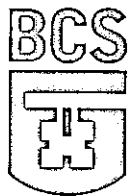


VECTOR

APL News, Views and Code ...

- | | |
|------------------------------------|----|
| • APL2002 Call for Papers | 11 |
| • First release of A for Windows | 14 |
| • McIntyre on Conjunctions | 64 |
| • Gustafsson on Gui and Databases | 78 |
| • Using the Windows Date-formatter | 97 |



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.18 No.2 October 2001

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette or via email. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word (any version).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2001–2002

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

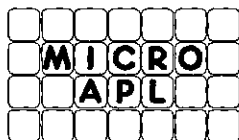
Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: apl385@compuserve.com.

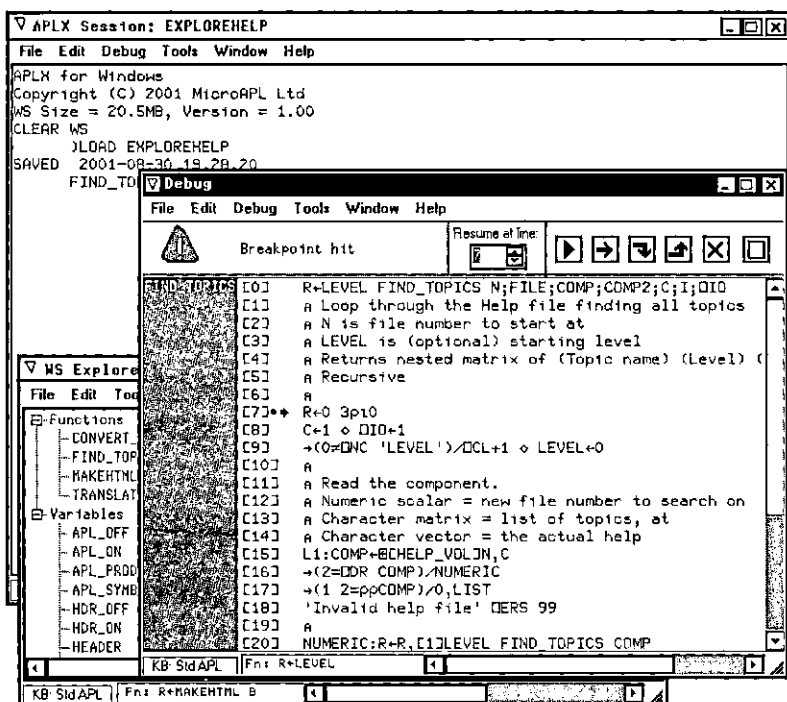
Contents

		Page
Editorial	Stefano Lanzavecchia	3
APL NEWS		
Correspondence		9
APL2002 in Madrid		
Conference Site and Call for Papers		11
British APL Association News		
An Implementation of A	Adrian Smith	14
News from Sustaining Members		18
APL Product Guide	Gill Smith	23
The Education Vector		
Zark Newsletter Extracts	edited by Jon Sandles	38
Crossword		47
J-ottings 30: Jaesthetics	Norman Thomson	50
Getting Started with A	Adrian Smith	56
An Elementary Tutorial on Conjunctions	Donald B. McIntyre	64
GENERAL ARTICLES		
Using Structures between GUI and Relational Databases	Tomas Gustafsson	78
Can Anyone Tell Me Where I'm Wrong?	Sylvia Camacho	86
TECHNICAL SECTION		
Hacker's Corner: Formatting Dates in Dyalog APL and APL+Win	Adrian Smith	97
Technical Correspondence		104
Persistence and Shadows	Stephen Taylor	110
MUCIPHER: a New Short Cipher that Escapes Cryptanalysis	Michel Dumontier	117
Function Models	Phil Chastney	132
Index to Advertisers		143

APLX



for	Mac OS 9	Windows 95/98/ME
	Mac OS X	Windows NT/2000/XP



The new, cross-platform, full-featured, user-friendly, GUI-enabled, cost-effective APL.
Check it out at <http://www.microapl.co.uk/apl>

APLX and MicroAPL are trademarks of MicroAPL Ltd. Mac is a trademark of Apple Computer Inc. Windows, Windows NT ME & XP are trademarks of Microsoft Inc. Price shown excludes VAT.

Editorial

by Stefano Lanzavecchia

"I find your lack of faith *disturbing*"

Darth Vader

Like Jedi knights, software developers have a powerful ally both in their daily chores and when they fly higher in pursuit of a space princess or of a million dollar project. Some are aware of the protection it gives them, of the numerous services provided, and this makes them respect it and seek for its help whenever needed. Some others don't understand it and simply wish it wasn't there. It wasn't long ago, even if it feels like centuries, that we were having fun but at the same time were really struggling on the noble task of colouring one single pixel on a black screen. We'd managed somehow to switch from text mode to graphic mode, the brand new graphic card we'd just bought and that was giving us hope of success. Amazing... And then there was the printer: I admit it, it was a good way to spend an afternoon, trying to understanding how to draw a picture on the painfully slow IBM 80 cps. At least it was less dangerous than to try the same thing on a mainframe's printer where a trivial mistake could send the printer into a maniac printing rampage that would consume 200 white sheets in the time required to reach the power switch.

Didn't you almost throw your shiny modem out of the window because you couldn't make it hear a word of the inspired declarations of love you thought you'd sent him? Yet, the cool serial port manipulation library you'd written after many late night hacking sessions was supposed to be flawless, never mind that the sole aid you had came from the technical documentation beautifully translated into NippInglisch from the original Japanese. Those days are over, the powerful ally is here for us: taking more and more of the burden on its shoulders, the *Operating System*. Call it uselessly bloated, mind-bogglingly complex, stupidly intrusive, if you like. I'll keep on using it and make it do all the simple and complex things that I don't have time nor I want to redo.

First of all: Operating Systems are not what they look like, but what they do. Mind you, nobody said Operating Systems have to be crude or ugly to be functional. Yet, there's a fundamental difference between a glamorously looking shell and a real Operating System. If you have programmed, not just used, both Windows 3.1 and Windows 2000 or Linux you know what I mean.

So here I am, staring at an empty APL workspace, with the desire to harness the power of my newly discovered mighty ally. "Don't!" "I will" "Don't!" "I will". Quad-NA, yes sir. The dice have been thrown: in the Microsoft world, COM components now and .NET objects in the future will provide a cleaner and less painful interface to the services exposed by the Operating System. In fact, in its richness, the .NET framework abstracts not only simple storage and graphic services, but gives support for regular expressions, inter-process communication, database access, to mention a few only. The Java Enterprise framework is already capable of all this, but it wasn't designed to be exploited from other languages, and, because of that, is less appealing to me. Still, with Sharp APL's interface to Java it may actually be possible to benefit from it even now. If any of you has experience in this field, let us know because I am really interested.

But let's return to the here and the now, to the fiend, Quad-NA, Dyalog APL's Name Association's primitive, to be precise, that is, the possibility of importing foreign functions exported from native DLLs. There's an understandable but, in my opinion, unjustified, fear of using this powerful feature. Any APL interpreter has to rely on the Operating System eventually to open a file, read data from it, close it, so it isn't intrinsically more dangerous to call straight from the Win32 API CreateFile, FileRead, CloseHandle than to use NTIE, NREAD, NUNTIE. It may be more difficult. (or is it just lack of habit?) and, in this particular case, most likely unjustified, but in no way more dangerous. I know it introduces an annoying dependence on the host Operating System and we would all love to be able to write platform-independent code, the ever-sighed "write once, run anywhere", but we also know that this is a chimera.

Many of the *Hackers' Corner* columns (like the one in this issue) contained wonderful examples of how easy it is to delegate the solution of a particular problem to an external source, more often than not, the Operating System itself. Here's another simple but interesting show-case: let's say you need to delete an APL's component file. What could be easier? Just FTIE it and the FERASE it. Let's now say that your component file is on a network drive somewhere and the network is known to be flaky, therefore the file is likely to be damaged. In other words the FTIE could very well fail, and it will surely fail at the customer's site. Still easy! NTIE it, instead, and NERASE it. Wonderful. Let's now say that the number of the files to be deleted is in the thousands and the network is overloaded (hundred of users on a 2 Mbit/s token ring). Do you really want to force the network to transmit all your open requests, cause the file server to get all excited preparing caches and various other data structures, just to delete the file immediately after? Wouldn't it be easier to simply delete the file?

Your Operating System can do it already. Just ask it to do it for you. The Win32 API has a handy kernel function called *DeleteFile* that does just that. Maybe it's something that should be supported by interpreter natively? Quad-NDELETE maybe? Maybe. And, then again, maybe not. APL interpreters have been around for a while and they have wisely tried to avoid the proliferation of Quad system functions. J is developing exactly in this direction: instead of covering more and more of the Operating System's APIs with the aid of the foreign conjunction, the implementers are de-committing more and more of the custom bang-colon (also known as !:) uses like ODBC, regular expressions, TCP sockets, and soon OpenGL as well, in favour of J's incarnation of Quad-NA coupled with supporting J scripts. The benefits are multiple: instead of opaque compiled C code, the front-end is written in plain J, there available to be customised by a developer in need; at the same time, the interpreter is slicker and the implementers can concentrate on the core language, where their time is best spent. Remember that J comes in a Windows version as well as a Linux version, so cross-platform compatibility is a serious issue, but it's possible to achieve with the right team of people and the right approach.

The Force is strong but Jedi knights need to be trained to learn to control it, or, if you prefer, powerful tools need time and affection from the user to unleash their full potential. My opinions, of course.

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

**Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.**

Please note that Vol.1 No.2 is now out of stock.

S H A R P A P L

SHARP APL 6.0 for UNIX & Linux

- Full implementation of SHARP APL
- Based on Iverson's APL Dictionary
- Rich in functionality
- Comprehensive documentation
- Linux version free for personal use

SHARP APL Java Interface

- Available for SHARP APL 6.0
- Platform independent Graphical User Interfaces
- Access from APL to Java, including Java services such as JDBC
- Access from Java to APL
- Free with personal use version of SHARP APL for Linux

For more
information contact:

Europe,
Benoit Paquin,
benoit_paquin@soliton.com

North America,
Nancy Lamb,
nancy@soliton.com

Download
SHARP APL for Linux
www.soliton.com/Linux



**SOLITON
ASSOCIATES**
Enterprise Software Solutions

Havsgaardsvej 4
DK 2900 Hellerup
Denmark
Tel: +45.2621.6099
Cell: +45.3940.1641

44 Victoria Street, Suite 2100
Toronto, Ontario,
Canada M5C 1Y2
Tel: 416.364.9355
Fax: 416.364.6159

1100 University Avenue,
Suite 111
Rochester, NY, USA 14607
Tel: 716.256.6466
Fax: 716.256.6469

Vector News and Dates for your Diary

APL 2002 – July 22 - 25, 2002, Madrid

The APL 2002 Conference will be held July 22-25, 2002 in Madrid, Spain. This is the annual international conference on APL and other array programming languages (A+, J, K, Matlab, Mathematica, Nial, S-Plus, etc.). The conference will be held in cooperation with ACM (SIGAPL) and the British APL Association.

“Clarity is the greatest virtue” – articles invited

If 80% of the life-cycle cost of most software goes on maintenance, clarity is the greatest virtue. Next issue we begin an occasional series of articles on coding practices that promote it. We’re looking for authors.

Whose coding practices do you admire? Nominate them to Vector: we’ll invite your heroes to share with our readers the practices that promote clarity and reduce maintenance and time spent in system testing.

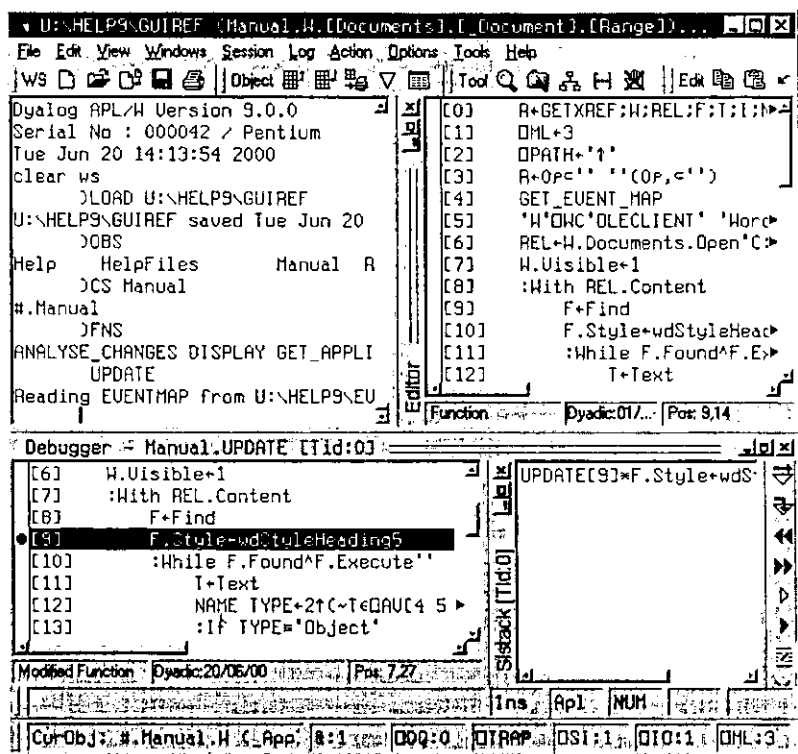
Email your nominations to the editor (stf@apl.it) and leave the rest to us.

Dates for Future Issues of VECTOR

	Vol.18 No.3	Vol.18 No.4	Vol.19 No.1
Copy date	7th December	8th March	31st May
Ad booking	14th December	15th March	7th June
Ad Copy	21st December	22nd March	14th June
Distribution	January 2002	April 2002	July 2002 (Madrid)

dyalog APL

The Definitive APL for Windows™



Version 9 - an even better IDE(A)

<http://www.dyadic.com>

Dyalog Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke,
Hants. RG24 7AL, United Kingdom.

Tel: +44 1256 811125 Fax: +44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation

CORRESPONDENCE

Vol 18.1 : "Vector is a fun game ... "

From: Ajay Askoolum

16th October 2001

From 18.1, I read that 'Vector is thriving; it has more material available than usual.' in the Minutes of the AGM and "... we are lacking a bit APL contributions.." in the Editor's reply. What am I to believe?

Vector will not last beyond 10 years if it is depleting its asset to the order of roughly £4000 per year, year on year.

Sincere congratulations to the 'Outstanding Achievement Award' winner – APL on the .Net platform keeps APL in the news and that is good news for APL. But, who were the other nominees?

Yet one more published letter on the issue of what appears in Vector! Mercifully, this time it was not about J, albeit more pages continue to carry J than APL, but about the relevance of articles. I subscribe to Vector because I want to read about APL, nothing else. If I can do that, the revised annual subscription cost is not a problem. I can read about other things elsewhere but about APL nowhere else.

Inversion of Binary Matrices

From: Mike Day

7th October 2001

Michel Dumontier writes about Inversion of Binary Matrices.

I'd just like to point out that I included a binary inversion function in my article about solving a puzzle:

Day, Mike *The MAGIC Goes Away – Opening the Boxes* Vol 10.1, 125

It wasn't particularly original; I'd adapted one of Keith Smillie's Statpack functions.

My introductory comments state:

```
R  INVERT A SQUARE BINARY MATRIX... AFTER KW SMILLIE'S STATPACK
2R NB REAL ADD/MINUS BOTH BECOME ≠
R   REAL TIMES BECOMES ×
R   REAL ÷ SHOULDN'T BE NECESSARY AS DIVISOR IS 0 OR 1
R
R  AN ALTERNATIVE METHOD IS TO TAKE
R   2 | (DETERMINANT A) × ΔA
R  BUT THAT NEEDS A DETERMINANT FUNCTION OF SIMILAR COMPLEXITY
R   TO THIS ONE
```

I recall that the late Gérard Langlet also wrote in Vector about the puzzle.



Universidad Autonoma de Madrid

APL2002 (APL-LPA 2002) in Madrid Call for Papers, Tutorials and Workshops

<http://www.apl2002.com>

Important Dates

- January 31, 2002: Deadline for paper submission (extended abstracts) and proposals.
- February 28, 2002: Notification of acceptance.
- April 30, 2002: Camera-ready Papers.
- May 31, 2002: Pre-registration.
- July 22 - 25, 2002: APL 2002 Conference in Madrid.

Introduction

The APL 2002 Conference will be held July 22-25, 2002 in Madrid, Spain. This is the annual international conference on APL and other array programming languages (A+, J, K, Matlab, Mathematica, Nial, S-Plus, etc.). The conference will be held in cooperation with ACM (SIGAPL) and the British APL Association.

Topics of Interest

The subject matter of the APL 2002 Conference is three-fold:

- **Lore:** The history of Array Programming Languages, including historical considerations on any of the following or other questions: design, implementations, applications, difficulties, standards wars, divergence, vendors, etc.
- **Problems:** Current use of Array Programming Languages in research and innovation, stressing the advantages provided by the use of the language in building a prototype or the final system. Straightforward descriptions of advanced research questions, or proposals for extensions for current or new Array Programming Languages are also encouraged.
- **Applications:** Actual use of Array Programming Languages for real-life applications, with successful case studies and comparisons with other languages. The application in question need not be at the front end of research in its field.

The APL 2002 Conference will also hold a number of Tutorials, hands-on Workshops and Panel discussions.

Submission of Proposals

The conference committee invites you to submit your proposals and intention to participate, indicating the following information:

Papers (40 minute presentations): an extended abstract with a maximum of two pages.

Short Communications (20 minute presentations): an abstract with a maximum of one page.

Tutorials (90 minute session): a one-page program of the proposed tutorial.

Workshops (two-hour hands-on session): a one-page description of the proposed workshop.

In all cases:

- Your name and affiliation.
- Your e-mail address.
- Type of proposal (paper, tutorial, workshop, panel discussion).

Submissions via e-mail:

Abstracts are welcome now and should be submitted as plain ASCII files.

Submit proposals and short communications to Xavier.Alaman@ii.uam.es

Submit proposals for tutorials and workshops to enrique@cs.york.ac.uk

Submissions via fax:

Submit all proposals to Xavier Alaman, +34 91 348 2235

Submissions via standard mail:

Submit all proposals to:

Xavier Alaman
E.T.S. Informatica
Universidad Autonoma de Madrid
Campus de Cantoblanco
28049 Madrid SPAIN

Organization

Conference Chair:

Manuel Alfonseca (Universidad Autónoma de Madrid, Spain)

Manuel.Alfonseca@ii.uam.es

Local Organizing Chair:

Pilar Rodríguez (Universidad Autónoma de Madrid, Spain)
Pilar.Rodriguez@ii.uam.es

Local Logistics:

Pablo Castells (Universidad Autónoma de Madrid, Spain)
Pablo.Castells@ii.uam.es

Francisco Saiz (Universidad Autónoma de Madrid, Spain)
Francisco.Saiz@ii.uam.es

Program Committee:

Xavier Alamán (Universidad Autónoma de Madrid) Xavier.Alaman@ii.uam.es
Robert Bernecky (bernecky@acm.org)
Robert Brown (ACM SIGAPL) bob@acm.org
James Brown (SmartArrays Corporation) jim.brown@SmartArrays.com
Dieter Lattermann (Dieter_Lattermann@compuserve.com)
Adrian Smith (British APL Association) adrian@causeway.co.uk

Tutorials/Workshop Chair:

Enrique Alfonseca (enrique@cs.york.ac.uk)

Webmaster:

Estrella Pulido (Universidad Autónoma de Madrid, Spain)
Estrella.Pulido@ii.uam.es

ACM Contact:

Robert Brown (ACM SIGAPL) bob@acm.org

Conference Site

The Conference will be held at the Technical School of Computer Science in the Universidad Autónoma of Madrid. This School is located in a new building, completed in July 1999 and providing all the opportunities for a successful conference. It has lecture theaters and a variety of seminar rooms with computers and projection equipment, plus several workshop rooms with 24 computers each. The University is located 16 km from the center of Madrid, accessible by excellent train and bus lines. Special buses will be provided for participants in the morning and the afternoon. Several hotels with a scale of prices will be offered to visitors.

British APL Association News

An Implementation of A

status report by Adrian Smith

A is best described as a reduced instruction set APL. It was designed and developed by Arthur Whitney around 10 years ago as a super-fast engine for manipulating timeseries data. It was subsequently developed as A+ by Morgan Stanley and has now been placed into the public domain under the GNU public licence. This full version includes X-window support and runs under various flavours of Unix.

The A Project at the British APL Association

[HTTP://www.vector.org.uk/aflat/](http://www.vector.org.uk/aflat/)

In spring of 2001 it was agreed to allocate funds to investigate an implementation of the core A interpreter under Windows 9x and 2000. This phase of the investigation is now complete, and the interpreter is available for download from these pages. Please feel free to try it out, but be aware that the environment is *totally primitive* and that you will have to be a little persistent to achieve more than '2+2' in console mode. You might like to start with some of the simple scripts on the *Getting Started* pages of the website, which include a translation into A of some functions from Norman Thomson's excellent *APL for the Mathematics Classroom*. Any components with Unix dependencies (such as the .sys context) are not available in this version.

The site reproduces a subset of Morgan Stanley's original A+ documentation (available in full at www.aplusdev.org), with acknowledgement of copyright where appropriate. Some sections (relevant only in a Unix environment) have been pruned. You will need the KAPL TrueType font installed to read the sample code in your browser. This font and the entire site content are available as ZIP files from the downloads page if you would like to read it offline.

Current Status of the A Project

Where it all began

The A engine was placed into the public domain late in 2000, and caused considerable interest in the APL community. In February this year, I contacted

Morgan Stanley to sound them out on attempting a Windows version of the interpreter. On Monday 12th Feb 2001 I received the following reply ...

"Morgan Stanley is very supportive of your plan to sponsor such a project. The only 'restriction' we have (as part of the GPL) is that your source code should be made available for some period. Building a wrapper around A+, and giving it away is a great idea."

The BAA committee was sounded out and approved a budget to cover the programming time to investigate the feasibility of porting the interpreter. Causeway agreed to do the work – giving us 5 days to read, understand and compile Arthur's code. This proposal was reported to the AGM in May and again raised a good deal of interest among the membership.

The painful part

Reading other people's code is never easy, and reading Arthur's C code is almost impossible. When Causeway finally made a week of clear time to attack this one, we decided to leave the core A code well alone and concentrate simply on stripping off anything that failed to compile until it produced an EXE that could run under Windows. This was easier said than done – it actually took the best part of 2 days to get it compiling under BSD Unix, then it was mostly a case of:

```
WHILE finished = false DO
  if compile_link = true
    finished = true
  else
    comment out failing code using #ifdef's
  endif
endwhile
```

The end result being a console-mode interpreter with a small supporting DLL (also freely distributable) to cover the fact that the console thinks it is running in a Unix shell!

What we have now

As far as we can tell, we have a functional A engine, including basic system facilities, for example you can run script files. Some system functions (such as `_fmt`) are missing and will be tracked down eventually. The `.sys` context is *not* included as it is strongly dependent on the Unix emulation layer, which we intend to shake off as soon as possible. A already supports 'ASCII' mode input so you can fire up the interpreter and type:

```

mode ascii
iota 12
0 1 2 3 4 5 6 7 8 9 10 11

```

What you cannot do in the console is enter (or see) APL symbols. Also you cannot use the 'up arrow' to modify or re-execute prior input lines. If you want to do anything interesting, you must write it as a script!

As part of the download set, you will find a very basic editor to allow you to write scripts with proper APL symbols, and type the APL characters using the familiar unified keyboard. Of course you can also use Notepad and write your scripts in ASCII mode if you prefer. Some simple scripts are included, for example you can make prime numbers with the sieve of Eratosthenes by running:

```
$load eratos
```

Which compares the creation of primes up to 500,000 by a recursive or old-style looping method. The script looks like this

```

A Experiments with primes to give us a rough speed check
A This is a direct rendering from 'D' as per Dyalog dynamic fns

L eratos R : {
  A Primes by the Sieve of Erastosthenes (see Scholes, Vector 13.2 p88)
  A This one uses recursion and seems very fast!
  next←1+R;
  mask←×next|R;
  if (⌵1+mask) +L,R; A Return result
  (L,next) & mask/R A Call self for the next step
}

A Now let's try the same algorithm 'old style' with a while loop

Eratos n : {
  A Primes up to n (the boring way)
  primes←2+⌵n+1;
  max←n+12;
  div←2;
  divs←div;
  while (div<max) {
    mask←×div|primes;
    primes←mask/primes;
    div←1+primes;
    divs←divs,div
  };
  (1+divs),primes
}

```

It is interesting to compare A with Dyalog APL dynamic functions – both have lexical scoping and automatic localisation. In fact the only major difference seems to be that in A you have to name your arguments! There is also a strong affinity

with the I-APL 'direct definition' approach to functions – it is remarkably easy to translate the functions from Norman Thomson's book *APL Programs for the Mathematics Classroom* into A.

Where next

Having proved that A can be made to run under Windows, the next stage is to decide what should be done with it. The initial idea was to make something which could be used in schools and colleges as an experimental environment for science and mathematics, in fact very much the same idea as the I-APL project which provided a free interpreter on the BBC Microcomputer for schools. For this to make any impact on schools in the UK, it has to support the National Curriculum and come accompanied by appropriate teaching material. Norman's book was written with I-APL in mind and he has agreed to consider a 'translation' to A which would make it an excellent starting point. However, I feel we need one more step before we can properly research this possibility.

The interpreter itself should be buried as a DLL (hence losing the last vestiges of its Unix heritage and that dreadful console) and so a suitable 'experimenting environment' will be needed to call it. In the first instance this can just be a 'session' with proper keyboard support and an APL font. Remember that this is emphatically not a Windows application development tool – what is needed is somewhere you can write, save, and run simple functions to manipulate and display numbers. Creation of suitable output files (probably in CSV format) or simply clipboard support, will be sufficient to permit transfer of results to Excel for graphing and further analysis.

Over to you

Please download the interpreter, play with some scripts and give us feedback. This is an extremely powerful APL engine, and Morgan Stanley have done the world APL community a huge service in making it publicly available. The challenge now is to make it useful in a way which does not threaten the commercial APL vendors, and which will be attractive to both hobby APLers and the educational world. To a great extent, the blossoming of APL in the 1980s was due to IBM's far-sighted approach in giving away VS APL licences to universities - with A we at last have the opportunity to do the same thing with a surprisingly modern-looking APL engine.

Sustaining Members' News

MicroAPL Limited

After a long absence from these pages, MicroAPL is back, in a big way – with a major new range of APL products for the Macintosh, Linux, and Windows!

Our long-established APL for the Apple Macintosh was the world's first APL interpreter for a modern-style windowing environment. With Apple's introduction of Mac OS X, substantial changes had to be made if we were to continue marketing this product. After a careful review of the technical and commercial options, we decided that this was the time for a major re-write. We are pleased to announce that our new APL development environment, called *APLX*, is now available – and that it runs under Windows as well as MacOS.

APLX: Cross-platform development

APLX comes in two forms, as a GUI application offering an advanced and elegant environment for developing APL, and as a server-based product for running large commercial APL applications and legacy systems. The GUI product is initially available for Windows 95, 98, ME, NT, 2000 and XP, as well as for Mac OS 9 and Mac OS X. In each case, *APLX* is a full native application, with the 'look-and-feel' appropriate to the host. For example, under Mac OS X, it implements Apple's new 'Aqua' user-interface, whereas the Windows versions follow Microsoft's style. All this is achieved using a common code base and advanced cross-platform technology, developed by MicroAPL, which will make it easy for us to continue to upgrade the product for multiple environments.

The server editions of *APLX* are available for Windows NT and 2000, for x86 Linux, and for the IBM RS/6000 under AIX.

All versions of *APLX* share the same workspace and component file layouts. This means that workspaces saved by any version can be loaded by any other, and component files can be shared across a network of different hosts. Where the two systems use different byte-ordering conventions, *APLX* transparently converts between them. Existing APL.68000 and APL Level II workspaces can also be loaded directly (*APLX* automatically converts them to the new format), and existing component files are also upwards-compatible.

For users migrating from other APL implementations, IBM-compatible APL transfer files can be read directly using the `)IN` system command, or via the File menu.

APLX: Elegant development environment

MicroAPL's long experience in developing APL windowing systems, plus our extensive exposure to 'best-of-breed' development environments for other languages, has been carefully applied to the design of *APLX* as a platform for writing APL code. The aim has been to provide very powerful development facilities, without any distracting on-screen clutter. Key features include:

- The Session window history can be as long as you like, subject to available system memory.
- Syntax colouring is available for Edit windows, with facilities for identifying and localising variables changed by a function.
- A pop-up menu is available at any time to edit an object, see its value or properties (including the syntax and first few lines of comment if it is a function/operator), or access the comprehensive on-line help system for the object or APL symbol at the cursor.
- The convenient Workspace Explorer window lets you browse quickly through the workspace contents.
- Watch windows allow you to monitor the contents of a variable or the result of any APL expression.
- The Debug window provides a user-friendly front-end for you to intervene when errors occur or a breakpoint is hit, displaying the stopped function and allowing you to browse the `⌈SI` stack. As well as using the pop-up menu to inspect variable contents, you can edit the stopped function directly in the debug window, set and clear breakpoints, and use the Step Over, Step In, Step Out and Go buttons to control execution. And if you want to resume execution from a different line, just drag the execution pointer there.

APLX: Cross-platform APL GUI applications

Under *APLX*, the APL developer has access to a rich set of user-interface objects using the `⌈WI` system function. This includes all of the standard objects such as windows, buttons, list boxes and edit fields, plus more advanced ones such as rich-edit controls, splitters, bevelled outlines, and movie players (also known as 'media' controls). In order to assist in application migration, this implementation is designed for upwards compatibility with MicroAPL's previous Macintosh APL, and is similar to the implementation in APL+Win. Enhancements include better geometry management (no need for tedious 'onResize' event handling!), and improved access to commonly-used facilities such as file-open and colour-selection dialogs.

Except for a few controls where the underlying operating system does not provide an equivalent capability, this functionality operates cross-platform. Your APL application written using *APLX for Windows* will run with little or no change under Mac OS X – and with the ‘Aqua’ look and feel.

APLX: Language features

Like its predecessors, *APLX* is a high-performance interpreter closely modelled on IBM’s APL2 language standard, combined with the tried-and-trusted APL68000 enhancements. Workspaces of up to 2GB are supported, subject to available system memory. *APLX* also implements the very powerful *ANA* system function, allowing you to call operating-system or DLL routines directly from APL, as though they were ordinary APL functions.

APLX: Development roadmap

This first release of the new *APLX* interpreter is the start of a planned series of upgrades and new features. As well as increased functionality in *AWI* and even better development environment facilities, we plan to introduce a number of language enhancements to *APLX* in future releases. We believe that high priorities include the ‘structured control’ directives and hierarchical naming, but we want to hear from you, APL developers, as to what you want.

Also on the roadmap is support for GUI programming under Linux. This will extend *APLX*’s cross-platform capability both for APL development and for applications developed in APL.

APLX: Competitive pricing

The *APLX* development environment for Windows and MacOS is competitively priced at £499 (+VAT) in the UK. Special terms are available for educational establishments and for users upgrading from previous MicroAPL products. For more information, visit our web site <http://www.microapl.co.uk/apl>.

APL 2000 Inc

APL+Win Version 4.0

APL2000 Inc. is pleased to announce the availability of APL+Win version 4.0 for Windows 95/98/NT/ME and 2000.

The most remarkable enhancement to this version of the software is the addition of the APL.Grid object to the APL+Win class library. This ActiveX control provides for the display of APL derived data in a familiar spreadsheet format. An APL.Grid workbook can contain multiple pages with the customary tabbed

interface, complete with row and column headings, fixed panes, cell decoration, and other features familiar to users of commercial spreadsheet programs.

While the grid object does not contain a calculation engine, direct transfer of array-oriented APL data into and out of the grid allows the interpreter to easily handle all computational needs. Customizable formatting for text, numbers, dates, times and currency data types is provided. Localization to native standards for dates, times, currency, and numerics is supported, as is specialized formatting based on value (*e.g.*, negative currency amounts).

Most all aspects of the grid are accessible under program control, but a standard user interface is provided as well. Cell content is easily edited by a user and verifiable by the program. Cell selection is accomplished with the usual actions, and multiple cell ranges are supported. The potential for a truly unique user interface is enhanced by several special cell types. Rather than data, a cell can contain a button, a combo box, or a check box. Tooltips can be provided to explain the content or function of specific cells, and overlay images can be set into cells to provide visual cues.

APL.Grid, as an ActiveX control in APL+Win, also benefits from those enhancements made to almost all controls in the class library. Chief among these are new properties which make the provision of truly context-sensitive help easier. Version 4.0 also enables use of the question mark button on form borders to allow for the implementation of "What's This?" help. All visible controls now have a `noredraw` property which can help control screen flicker when updating complex forms, and the new `imagelist` operation of the `Draw` method allows easy visual customization of all controls.

Other areas of the system have been enhanced in this latest release. Interaction with printers has been augmented by better connection with the standard Windows print dialog boxes. Printer capabilities such as paper bins and collation are now easily available to your functions. You can easily find or set the default printer, and can now detect and use printer resources connected to your application's station by a network. Specific controls have had their capabilities enhanced as well. The Combo box has been extended to support images and indents; the RichEdit control supports bullets, indents, and paragraph numbering; and the Selector (tab control) offers new options for tab appearance and behavior. More controls now benefit from enhanced tooltip support, and overlay images have been added to more controls' display.

Enhancements for those in special circumstances round out the list of features in this release. Large applications may benefit from a greatly increased maximum

file size for both component and native files, and a doubling of the file tie quota. Applications using ActiveX objects can exploit an expanded calling syntax which can chain method calls, name parameters explicitly, and reference defined constants. Your applications have better control of their INI files, both in accessing them and creating them, and can initialize themselves without displaying the APL+Win banner.

And last but not least, APL+Win now comes with a full complement of printed documentation. The three-volume perfect bound set contains a newly revised User Manual, Systems Functions Manual, and a Windows Reference manual. The text of the printed documentation is still distributed on the APL+Win CD, along with a comprehensive help system, practical utilities, and sample workspaces.

For the latest on this release, or to learn about our other products, please visit us at www.apl2000.com

Causeway Graphical Systems Ltd

Strange how you can stay in the same place and eventually the world comes around to you. I think the Guinness campaign slogan "All things come to he who waits" started with Badger in *Wind in the Willows* who just 'remained' while civilisations came and went around him.

Back in late 1989, I started recording rainfall data and wanted to plot it as a nice graph. The technology available was APL*PLUS/PC and my HP DeskJet 500 and GoScript (a command-line PostScript engine). The result was a set of vector-graphics primitives which eventually formed the basis of *RainPro* and *NewLeaf*. These things have rested for over 10 years until the W3C published the specification for SVG, and Adobe released PDF into the public domain. Suddenly, there are two very PostScript-like engines out there, and anything drawn with these basic primitives will render easily on either of them.

Now, *RainPro* has always been about making simple, effective charts with minimum use of redundant ink. This keeps the bandwidth down (even as images, the charts compress very well) but somehow it lacks pizzazz. With SVG you can define the most spectacular gradient fills, 3D extrusions, and even lighting. Just imagine a piechart with gradient-filled 3D sectors, lit from a spotlight positioned a few cm in front of the monitor surface, angled to bring out the shadows.

No, on second thoughts, go and have a look at the examples on www.grapl.com in the "Fun with Filters" section. Then check the file sizes. Then read the ASP script that made them. I think you will see what *RainPro* has been waiting for.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

*Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.
Please help us keep it up to date!*

All contributions and updates to the Vector Product Guide should be sent to: Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385, Email: apl385@compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Optima	IBM Compatibles	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
APL Systems IDC SL	APL2000	poa	APL Systems IDC SL are the new APL2000, Inc. distributor the United Kingdom. APL Systems will assume the sales and support role of Bloomsbury Ltd. Bloomsbury were purchased by WebGain Ltd. and will not be marketing nor supporting the products of APL2000, Inc.
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyadic Systems Ltd. products
	IBM APL products	poa	
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.

	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL.
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APs; TIME facility
	APL2 for Windows version 1.0	\$1500	Product No. 5639-d46, part number 4229558.
	APL2 Runtime environment	\$250	Part No. 39L8419 - Packager and runtime CDs. One additional runtime install \$200, 5 additional \$900, 10 \$1,500.
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
Insight Systems	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
	Cognos/APL2000 Inc	poa	Leading distributor of APL2000 products in Denmark
	Dyadic Systems Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
JSoftware Inc.	J on the Web online registration ...		
	J Professional (online reg.)	\$895	Includes manual set and one year of updates
	J Standard (online reg.)	Free	Free for download only
	Books and accessories		
	J Dictionary	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Fractals, Visualization and J	\$80	
	Concrete Math	\$40	
	Exploring Math	\$50	

Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also distributes in Switzerland and Belgium. Call for price quotes.
	APL+Unix	poa	
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APLX for Windows/MacOS	499	Cross-platform APL development environment with GUI programming facilities. Interpreter modelled on APL2. Available for Windows 95/98/ME/NT/2000/XP, Mac OS 9 and Mac OS X.
	APLX Server Edition	poa	For running large multi-user APL applications on x86 Linux, RS/6000 AIX, and Windows NT/2000.
Oasis	Dyalog APL	poa	Dyadic Systems
	APL*PLUS	poa	Manugistics
	APL68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small simple and fast" alternative to APL
Optima	Dyalog APL/W	995	Fully fledged Windows development environment.
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Soltton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and JSoftware products	poa	
	USA		
	Dyadic and JSoftware products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum	Adaytum e.Planning	poa	Adaytum e.Planning offers a Web-based solution that combines planning, forecasting, budgeting, modelling and reporting in a single, integrated application.

APL Software/Services			
	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software. Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick.holt@juno.com.
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in NewLeaf reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win.
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	3-way TANGRAM	poa	3-way TANGRAM is a DSS-OLAP product, basically a powerful and versatile handler of multi-way tables (also known as hypercubes). It entails 46 analysis modules, including computations, cube merging, sensitivity analysis, time intelligence, free format queries, HTML and LaTeX outputs. Current version is 7.0. At the time of writing, the product is available on Dyalog APL 8.3.1 for Windows 95/98/NT. Future plans include an APL+Win version and later a LINUX version.
CoSy	K.CoSy	\$30 % yr sub	K.CoSy is a general purpose computing and programming environment constructed, all in open code, in Arthur Whitney's very high level, yet structurally transparent Array Programming Language, K, and its tightly coupled GUI. K.CoSy is an extremely productive environment in one of the most powerful and fastest of APL's progeny, and therefore, likewise, of all languages. K.CoSy provides a workspace-like interactive development environment previously impossible in K. (Requires separate download of K from Kx.com.)
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/T users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
HMW	4XTRA	poa	Networked, Windows/Unix based Front End and Middle Office Foreign Exchange and Money Market Dealing System. Scalable from 1 user to 120+.
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.

	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.
	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson. Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System (AGSS)	\$250	for DOS, Product Number 5764-009
		\$500	for Workstations (OS/2, Aix, Solaris), Product Number 6764-092
		\$2500	for CMS, Product Number 5764-011
INFOSTROY	APL*PLUS/Xbase Interface (II/386 Version 2)	\$198	Complete package written in C. Comparable with the data, Index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark <i>All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!</i>
JAD Software	JAD SMS	poa	JAD SMS is a multi-user software management system for Dyalog APL™ based on shared, hierarchical databases. JAD SMS databases let you keep historical versions of apl items as well as attributes such as timestamp, user name and documentation. The software includes a graphical user interface as well as specialized functions for inclusion in applications. No charge for single-user version; \$100/user for multiple users
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on Interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and DyalogW.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and DyalogW).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	AP127/PC	poa	ODBC interface for Dyalog APL/W
	AP119/PC	poa	TCP/IP interface for Dyalog APL/W
	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	ISAP	poa	MS Windows DLL for calling Dyalog APL/W from web pages via MS Internet Information Server. Visit www.lingo.com for a demo.
	AP747	poa	ODBC interface for Dyalog APL/W
	QWIN	poa	Legacy DOS Windowing support for APL+Win

Qualedi	Qualedi	\$850-\$5,500	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +II v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.
	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Weighahead Systems	Weighahead Windows Weighing System (3WS)	poa	Recipe Weighing System for Manufacturing Industries. Pharmaceutical, Cosmetics, Foods etc. Works without keyboard or mouse. Uses Electronic Balances, Laser scanners, bar codes and label printers.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Ajay Askoolum	Consultancy	poa	APL+Win development and migration of actuarial, financial, mathematical applications.
Andrews	Consultancy	poa	APL programming and analysis, Year-2000 legacy systems, algorithms, tree-processing.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL+Win, Dyalog APL
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
AUSCAN Software	Consultancy	poa	APL software development, training
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.

Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
CoSy	Consultancy	poa	CoSy.com, Coherent Systems, provides rapid development in the K language and associated data base products, with a particular interest in quantitative (financial) modeling. A great deal can be done in a day.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	APL consultant with 20 years experience with all versions of APL. I can create your dynamic Web sites using the full power of APL working with Microsoft IIS (Internet Information Service) on Windows NT or 2000. I also undertake System design, Programming and Maintenance on all platforms, particularly MS Windows.
INFOSTROY	Consultancy	poa	Moving applications between platforms. Client/server development. Multilingual user interface.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.

Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APLW. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.
George MacLeod	Consultancy	poa	Design and programming of new APL applications. Enhancing and maintaining existing APL applications. Porting existing APL applications from one APL system to another. Supporting users of APL applications. Experienced on both mainframe, UNIX and PC APL interpreters.
Mackay Kinloch Ltd	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Object Oriented Ltd	Consultancy	poa	General APL consulting, code recycling - mainframe to PC, performance tuning.
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
Resources & Results	Consultancy	poa	Knowledge management company builds decision support, data warehouse, data mining and strategic planning systems for CFO's, CEO's, and senior management, using our Rapid Application Development (RAD) methods and tools. Extensive experience in large-scale system development and ad hoc executive support for Fortune 500 clients.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to API-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.

Weighahead Systems Consultancy		poa	Specialising in industrial systems. Links to PLCs, laser scanners, bar codes, weigh scales, label printers etc. Also programmable hand held scanners.
Stephen Wynn Consultancy		poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email 100612.1057@compuserve.com.
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL		FF350 (private) FF2800 (Company)
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join
NY SigAPL	New York, USA	Big Apple APL	Monthly meetings	\$35/\$25(ACM)
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of City SI-Info		SF60 (SI) + SF20 (SAUG)
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Stellinger Weg 19, 20255 Hamburg, Germany. Tel: +49 40 40170951 Fax: +49 40 40170952. Email: info@adapta.de
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@meibpc.org.au
Adaytum Limited	Douglas Rowley	Castlegate, Tower Hill, BRISTOL BS12 0JA. Tel: 0117 921 5555 Fax: 0117 922 7749. Email: sales@adaytum.co.uk
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205 Email: ADWilson@kencomp.net
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: apl385@compuserve.com
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@aplborealis.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Germany e.V.	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1461
APL Software/Services	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Fax: +1 (703) 528-7617; Email: dick.holt@juno.org
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cais.com
APL Systems IDC SL	Contact: Fred Honea	Alfredo Marquerie, 12 - 2 F, 28034 Madrid, Spain. Tel: +34 91 730 7008 (Office) +34 60 680 5949 (Mobile). Fax: +1 775 743 6131. Email: uksales@apl2000.net
Ajay Askoolum	Ajay Askoolum	42 Hanworth Road, Redhill, Surrey RH1 5HT Tel: 01737-771643 Email: ajay@askoolum.freeserve.co.uk
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@interlog.com
BACUS	Joseph De Kerf	Roolenberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888 Email: BeautifulSystems@golfs.net
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. email: acam@tesco.net
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
Causeway Graphical Systems Ltd	Adrian Smith	The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01653-696760 Fax: 01653-697719 Email: adrian@causeway.co.uk
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB, UK. Tel: 020 7404 5401. Compuserve: 100343,3210
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
Ian Clark	Ian Clark	1, Heifer Mill Cottages, Mosterton, Beaminster, Dorset DT8 3HG, England. Tel: +44 (0)7931 370304. Email: earthspot@aol.com
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 612 2652 Email: codework@codework.it.com
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com

CPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375.
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10038-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Ditrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL, UK. Tel: 01256-811125 Fax: 01256-811130
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email: info@DynArray.com
Evestio AB	Olle Evero	Bertellusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mailbox.swipnet.se
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-335620. Email: KenChakahwata@compuserve.com
General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 749328 Fax: 01622 749365 E-mail: martin@gsoft.freemove.co.uk
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4V 0HA, UK. Tel: 0870-1010-469; Email: HMW@4xtra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.Ltd.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0035. Email: 100612.1057@compuserve.com
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept REN/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOTROY	Alexei Miroshnikov	3 S. Tulenih Lane, St. Petersburg 191186 Russia. Tel: +7 812 312-2673 Fax: +7 812 311-2184 Email: aim@infotroy.spb.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
JAD Software	David Crossley	175 East 96th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124 Email: jadsms@usa.net
Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Buid.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022. Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KYY00361@niftyserve.or.jp
J Austria	Joachim Hoffmann	Hochstetelngasse 13/26, 8010 Graz, Austria. Tel: 0043 (0)316 91 42 51 Mobile: 0043 (0)699 1 91 42 51 2. Email: joachim.hoffmann@gmx.at
JS Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@jssoftware.com
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56, 00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: Kimmo.Kekalainen@pp.htv.fi
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.nfl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo_allegro@mail.com

Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3.tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)131 228 5235 Email: akinloch@globatnet.co.uk
George MacLeod	George MacLeod	37 Newhouse Rd, Bovington, Herts, HP3 0EJ, UK. Tel: 01442-831385 Fax: 01442-831388 Email: george.macleod@ntworld.com
Mercia Software Ltd.	Gareth Brentnall	Mercia House, Ashted Lock, Aston Science Park, Birmingham, B7 4AZ, UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Millinta Inc.	Dan Baronet	Contact Dan Baronet at danb@millinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: Ellis@mrtfarm.demon.co.uk
NY Sig	David Siegel	PO Box 2697; New York, NY 10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: info@oasis.nl or rijkse@oasis.nl
Object Oriented Ltd	Walter G. Fil	Am Grendel 2, CH-6004 Luzern, Switzerland. Tel: 41 41 418 70 70 Fax: 41 41 418 70 77 Email: info@object-oriented.com
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@mindspring.com
Optima Systems Ltd	Paul Grosvenor	1st Floor, 10 Raleigh Court, Priestley Way, Crawley, West Sussex, RH10 2PD, UK. Tel: 01293 562700 Fax: 01293 562699 Email: mailbox@optima-systems.co.uk
Qualed Inc.	Ann Wilkie (Marketing) Ed Shaw (Technical)	Merritt Crossing, 440 Wheelers Farms Road, Milford, CT 06460, USA. Tel: +1 (203) 882-9988. Fax: +1 (203) 882-9891 Email: awilkie@qualedi.com; eshaw@qualedi.com
RadSys Technologies AB	Randolph Schrab	Lovsängarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems	Ed Shaw	P.O. Box 313, Newtown, CT 06470, USA. Tel: +1 (203) 270-9729 Email: rendata@apibooks.cnhost.com or orders@apibooks.cnhost.com
Resources & Results	Frank Rhodes	2438 W. Northgate Dr., Irving, TX 75062, USA. Tel: +1 972 523 5117 Email: frank@decision-support.net
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapl@doc.aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Benoit Paquin	Soliton Denmark, Havsgårdsvej 4, 2900 Hellerup, Denmark Email: sales@soliton.com
SovAPL Russian Chapter of SIGAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askomr@obninsk.com
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Ulthelm	Novator Consulting Group AB, Svärtdvägen 11C, S-182 33 Danderyd Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CService: 100341,404

Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: si@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelalade St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org
Weighahead Systems	Philip Bulmer	Camberley House, 1 Portesbery Road, Camberley, GU15 3RB, UK. Tel +44 1276 20789 Email: sales@weighahead.com
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK. Tel: 01273-327238 Email: centre@cwcom.net
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2	ftp.software.ibm.com/ps/products/apl2
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adaptable Systems	www.assuredsystems.com.au/
Adaytum Limited	www.adaytum.com/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Journal, Germany	www.rhombos.de/rb/apljourn.htm
AUSCAN	www.interlog.com/~rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Capital PC User Group	http://cpcug.org/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyadic Systems Ltd	www.dyadic.com/
DynArray	www.dynarray.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Houben (IQL)	www.apl.olap.club.tip.nl
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/ad/apl
Infostroy	www.insight.dk/infostroy/
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.naska.co.jp/JAPLA/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
Lescasse Consulting	www.lescasse.com/

Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	www.mackaykinlochltd.co.uk/~admkl/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Oasis b.v.	www.oasis.nl/
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualifiedi.com
Renaissance Data	www.aplbooks.cnchost.com/
Resources & Results	www.decision-support.net
RE Time Tracker Oy	www.rett.fi/
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Soliton	www.soliton.com/
Snake Island Research Inc.	www.snakeisland.com
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/
Weighahead Systems	www.weighahead.com

ZARK Newsletter Extracts

introduced by Jon Sandles

Here is another Zark newsletter extract. As always, I hope you are all still enjoying reading these reprints. I certainly enjoy reading through the old issues of Zark in order to choose these extracts. Once again, can I remind you all that although these are reprints, we still expect submissions to the challenges set, as there is always a chance that new light can be shed on these old problems. We particularly encourage submissions in the more recent variants of APL, for example J and K.

Utility Corner: Polyadic Functions

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

In last issue's LIMBERING UP column, we asked you to write a function named *MORTGAGE* that returns the monthly repayment amount for any specified interest rate, loan amount, and term. For example, if the interest rate is .09, the loan amount is 90000, and the term is 30 years, the monthly repayment amount is 724.16.

To save you some time, we provided the formula:

$$V \leftarrow 1 + INT \div 12 \quad \text{a Monthly discount rate}$$

$$PMT \leftarrow PRIN \div (V - V \times 1 + 12 \times TERM) \div 1 - V$$

<i>INT</i> :	Nominal annual interest rate
<i>PRIN</i> :	Principal (amount borrowed)
<i>TERM</i> :	Number of years of load
<i>PMT</i> :	Monthly payment

J. Bruce King let us know that the formula can be algebraically simplified to:

```

IM←INT÷12
TM←TERM×12
PMT←PRIN×IM÷1-(1+IM)*-TM

```

We'll use Bruce's formula below.

At first glance, the *MORTGAGE* function seems trivial:

```

▽ PMT←MORTGAGE PARAMS;IM;TM
[1]  ♂ Return monthly repayment amount PMT
[2]  ♂ from interest PARAMS[1], principal
[3]  ♂ PARAMS[2], and term PARAMS[3]
[4]  IM←PARAMS[1]÷12
[5]  TM←PARAMS[3]×12
[6]  PMT←PARAMS[2]×IM÷1-(1+IM)*-TM
▽

```

This function assumes each input item is a scalar. The problem becomes more challenging if you allow the input items to be vectors or matrices. For example, suppose interest is 8%, 9%, 10%; principal is 90000; term is 20, 25, 30. Two questions arise:

1. How can these three arrays (.08 .09 .10 and 90000 and 20 25 30) be jammed into one or two arguments?
2. Should the result be a three-element vector or a three-by-three matrix?

It's the former if the input arrays are assumed to be scalar conformable; this is, each array has the same shape or is a singleton – the elements are in one-to-one correspondence:

Scenario 1:

Principal	=	90000	"	"
Interest	=	.08	.09	.10
Payment	=	?	?	?

It's the latter, a matrix, if each element of the input arrays is considered with every possible combination of the elements of the other input arrays:

Scenario 2:

Principal	=	90000						
Interest	=	.08 .09 .10						
		20						
		25						
Term	=	<table border="1"> <tr> <td>?</td> <td>?</td> <td>?</td> </tr> <tr> <td>?</td> <td>?</td> <td>?</td> </tr> </table>	?	?	?	?	?	?
?	?	?						
?	?	?						

30

?	?	?
---	---	---

In Scenario 1, the shape of the result is $\rho (INT+PRIN+TERM)$. In Scenario 2, the shape is $\rho (INT \circ . + PRIN \circ . + TERM)$.

Let's deal first with the question of passing three arguments to the *MORTGAGE* functions. Here are the suggestions we received:

1. `3 1 3 MORTGAGE .08 .09 .10 90000 20 25 30`

This syntax, from J Bruce King, passes all three input arrays in the right argument as a simple vector. The arrays are ravelled and catenated. The left argument is a three-element vector of the lengths of each ravelled input array. If omitted, the left argument is assumed to be `1 1 1`, permitting the syntax, `MORTGAGE .09 90000 30`.

2. `MORTGAGE (.08 .09 .10) (90000) (20 25 30)`

Fans of vector (strand) notation, Jack H Zinn and Rick Gracey fused the three input arrays into a three-item nested vector argument. The first line of the *MORTGAGE* function converts the nested array back into three simple arrays:

```

      v PMT←MORTGAGE PARAMS;INT;PRIN;TERM
[1]    (INT PRIN TERM)+PARAMS  a APL2
[1]    INT PRIN TERM+PARAMS    a APL*PLUS

```

3. `INT+.08 .09 .10 ◊ PRIN+90000 ◊ TERM+ 20 25 30 ◊ MORTGAGE`

Ignoring argument altogether, John E. Lauer suggested the expedient of passing the three input arrays as global variables. (Diamond $\circ$ is a statement separator in APL*PLUS.)

4. `MORTGAGE 'INT+.08 .09 .10 ◊ PRIN+90000 ◊ TERM+ 20 25 30'`

Not to be outdone by himself, John E. Lauer then suggested passing an executable character vector as the argument. By localizing these variables within *MORTGAGE* and by executing the character vector at the start of *MORTGAGE*, the input arrays are passed without ever existing in the global environment.

5. `MORTGAGE .08 .09 .10,90000,[1.5]20 25 30`

This syntax, from Nick Rago, permits the right argument to have any shape, as long as its last dimension is 3. In this case, the argument has shape 3 3:

```

0.08 90000 20
0.09 90000 25
0.1  90000 30

```

The shape of the result is the same as the shape of the right argument, less its last element.

Let's deal now with the second question, that of deciding whether the input arrays are to be treated in a scalar conformable fashion ($INT+PRIN+TERM$) or in an outer product fashion ($INT \circ . + PRIN \circ . + TERM$).

Some respondents wrote logic that implemented the outer product design (all possible combinations) and some wrote logic that was a hybrid of both scalar conformable and outer product designs. In the hybrid functions, the input arrays with the same shape were used in one-to-one (scalar) correspondence, and all others were applied in outer product fashion. For example, given the shape of INT is 3, the shape of $PRIN$ is 3, and the shape of $TERM$ is 4, the outer product functions return a result with shape 3 3 4, and the hybrid functions return a result with shape 3 4.

The following function is typical of those using a strictly outer product approach (from King and Zinn):

```

v PMT+MORTGAGE1 PARAMS;A;B;C;IM;PRN;TM
[1]  a Return monthly repayment amount PMT
[2]  a from interest 1>PARAMS, principal
[3]  a 2>PARAMS, and term 3>PARAMS. Result
[4]  a has shape 3, /p"PARAMS.
[5]  A+pIM*(1>PARAMS)+12 a Monthly int.
[6]  B+pPRN+2>PARAMS a Principle amount
[7]  C+pTM+(3>PARAMS)*12 a Term in months
[8]  PMT+(A*(A,B,pC)/2 1 3)&PRN*(IM*(Cp0)+1-(1+IM)*.-TM
v

```

The hybrid functions were well-written but struck us as somewhat ill-behaved. Notice the shape of the result (PMT) as the shapes of the input arrays vary.

ρINT	$\rho PRIN$	$\rho TERM$	ρPMT
1	2	3	1 2 3
2	2	3	2 3
3	2	3	3 2
4	2	3	4 2 3
5	2	3	5 2 3

(In defence of the hybrid fans, we deliberately misled them by posing problems that required both scalar conformable and outer product interpretations.)

The design of Nick Rago's function was unique, in that the nature of the right argument eliminates the scalar conformable vs. outer product question.

The shape of the argument defines the shape of the result:

```

      v PMT←MORTGAGE2 PARAMS;IM;TM
[1]  a Return monthly repayment amount PMT
[2]  a from interest (1 0 0/PARAMS),
[3]  a principle (0 1 0/PARAMS), and term
[4]  a (0 0 1/PARAMS). Result has shape
[5]  a  $^{-1+\rho}PARAMS$ 
[6]  IM←(1 0 0/PARAMS)÷12 a Monthly int.
[7]  TM←(0 0 1/PARAMS)×12 a Term in mons
[8]  PMT←( $^{-1+\rho}PARAMS$ )ρ(0 1 0/PARAMS)×IM+1-(1+IM)*-TM
      v

```

Given these two functions, here are the solutions to the problems presented in the last issue of the news:

1. Interest is 9%; principal is 90000; term is 30.

```

MORTGAGE2 .09 90000 30
MORTGAGE1 .09 90000 30

```

2. Interest is 9%; principal is 90000; term is 15, 20, 25, 30 (4-element vector result).

```

MORTGAGE1 .09 90000(15 20 25 30)
MORTGAGE2 .09,90000,[1.5]15 20 25 30

```

3. Interest is 8%, 9%, 10%; principal is 90000; term is 20, 25, 30 (3-element vector result).

```

1 1 & MORTGAGE1 (.08 .09 .10) 90000 (20 25 30)
MORTGAGE2 .08 .09 .10, 90000,[1.5] 20 25 30

```

MORTGAGE1 computes all possible combinations and then extracts those desired (on the diagonal) via 1 1 &

4. Interest is 8%, 9%, 10%; principal is 90000; term is 15, 20, 25, 30 (3 by 4 matrix result).

```

MORTGAGE1(.08 .09 .10) 90000 (15 20 25 30)
MORTGAGE2(&4 3ρ.08 .09 0.10),90000,[2.5]3 4ρ15 20 25 30

```

MORTGAGE2 uses reshape, laminate, and transpose to construct the needed three-dimensional argument.

[Ed:] In APL2 the last example above could have been written as:

```
MORTGAGE2 =>.08 .09 .10*.,90000*.,15 20 25 30
```

Limbering Up: Killing Comments

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

Ill-disciplined APL programmers rationalise their failure to include comments with statements like, "Comments make my functions too big. I hate to waste valuable workspace on useless comments."

You can argue that comments are never useless, or that comments don't consume much workspace, but your arguments are likely to fall on deaf ears.

Your better bet is to provide the lazy sceptic with a utility function that eliminates the comments from any specified user-defined function. They say, "Include plenty of comments in your functions and save them in a 'development workspace.' Use this utility function to eliminate all comments from the functions and save them in a 'production workspace.' You'll have one workspace that is big and readable and easy to maintain, and another that is small and unreadable and streamlined for running."

Your task is to design and write a function named *UNCOMMENT* that eliminates the comments and all-blank lines from a specified function. Use `⌊CR` to convert the function to its character matrix representation; manipulate the character matrix to eliminate its comments and all-blank rows; and use `⌊FX` to convert the character matrix back to a function. See the *PRIMITIVE POINTERS* section for some hints.

The function should be efficient, but extreme efficiency is not needed since the function will be used during the development process, not during production. And, by the way, your function should contain lots of comments.

Send your solutions to:

Vector Production
 Brook House
 Gilling East
 York YO62 4JJ
 UK email: apl385@compuserve.com

The notable functions and their authors' names will be printed in the next issue of *Vector*. Good luck and happy limbering.

Primitive Pointers

(The purpose of this column is to point out some facts about the primitive APL functions you may have overlooked or forgotten. If you don't believe use, try it out!)

From last issue: the first step in parsing an executable APL expression for identifier names is to ignore the text within quotes. Here's an example:

```
EXPR←'SUB+(∨/'FL'⊆NAMES)≠NAMES'
```

The identifiers used in this expression are *SUB* and *NAMES*. *FL* is not an identifier because it's within quotes. The $=\backslash$ and $\neq\backslash$ functions can be used to distinguish between text within quotes and text outside quotes. Let's try them (blanks between bit elements are omitted for clarity):

```
EXPR
SUB+(∨/'FL'⊆NAMES)≠NAMES
```

Mark the quotes:

```
EXPR='''
000000010010000000000000
```

Text within quotes including leading quote:

```
≠\EXPR='''
000000011100000000000000
```

Select all other characters:

```
(~≠\EXPR=''' )/EXPR
SUB+(∨/'⊆NAMES)≠NAMES
```

$=\backslash$ is the opposite of $\neq\backslash$:

```

      EXPR#''''
111111101101111111111111
      =\EXPR#''''
111111100011111111111111

```

Exclude both quotes, selecting just non-quoted text:

```

      (=\EXPR#'''' )^EXPR#''''
111111100001111111111111
      ((=\EXPR#'''' )^EXPR#'''' )/EXPR
SUB+(v/\_NAMES)/NAMES

```

The second step in parsing an executable APL expression is to ignore the text following lamp (⌘) symbols. An example:

```

X←'SUBv/''⌘FL''_NAMES ⌘ LOOK FOR ''⌘FL''

```

The identifiers used in this expression are *SUB* and *NAMES*, not *FL*, *LOOK*, or *FOR*. What APL expression selects from a character vector *X* just those characters that are *not* with quotes or comments?

Answer next issue. (Hint: Experiment with $\wedge\backslash$ and $v\backslash$. Consider only lamp symbols outside quotes.)

Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated, 23 Ketchbrook Lane, Ellington, CT06029, USA

[Ed: The following is a summary of relative times for 4 *COMBIN* 15 (see *Vector* 18:1) in the original 1992 article and in Dyalog APL version 8.2:]

<i>COMBIN</i> version	APL*PLUS II	Dyalog APL
1	14	23
2	10	21
3	92	166
4	17	32
5	27	56
6	5	8
7	1	1

Anne Wilson submitted the following function which unfortunately ran 242 times slower than the fastest version!

```

▽ i+n COMBIN m
[1]  A ADW 10/6/2001
[2]  A Returns n column matrix of all combinations of n from m
[3]  i+φ&(mp2)τi~1+2*m      A Generate all combinations
[4]  i+(n=+/i)/[1]i          A Keep those with n elements
[5]  i+ix(pi)pim              A Turn them into indices
[6]  i+((pi)[i],n)p(0≠,i)/,i A Remove all zeros
▽

```

Gordon Haig's function only ran 63 times slower:

```

▽ Z-R COMBIN N;LIST;VEC;UPPER;COL;COLS
[1]  LIST+(0,R)p:0
[2]  VEC+1R      A Lowest number
[3]  UPPER+(-(1N)ε1(N-R))/1N A Highest number stored for ref.
[4]  LQ1:LIST+LIST,VEC      A Current lowest number onto list
[5]  -(VEC=UPPER)/LQ2      A Finish if reached highest number
[6]  COL+R-+/(VEC=UPPER)   A Rightmost incrementable column
[7]  COLS+1(R+1-COL)      A COLS: the consecutive increments
[8]  VEC[1+COL+COLS]+VEC[COL]+COLS A Increments off set & placed
[9]  +LQ1
[10] LQ2:Z+LIST
▽

```

Finally, not a submission, but the Dyalog version 9.0 *dfns* sample workspace contains the following function which ran 3 times slower than the fastest Zark example:

```

cmat+{[ML+2      A α-combination matrix of 1w.
  +{(c<1 0p0){  A base-case: [0 w].
    -1+{        A removing base case from,
      (τ/φ{[IO,w}\1+α(1w)),w A accumulation of,
    }/w,αα      A sub- (α w-1) combinations,
    }/(10[w-α],c(αsw){α wp1w}"φ+\ap1 A with base cases [α 0].
  }

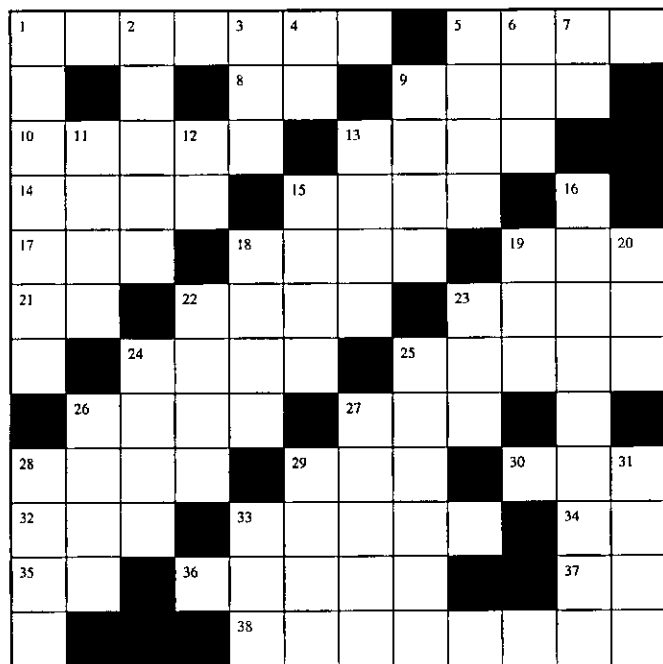
```

Solution and New Crossword

Solution to Crossword in 18.1

→	0		A	∨	D	⊃	C		A		B
L		D	+	.	≠	A			⌈	C	
	A	^	.	=	B		D	∇	N		⌈
E	⌈	.	×	B			,	C		+	/
+	.	>	B		N	?	B		<	\	B
C	+	B			ρ	B		-	/	V	
⋄	B		V	~	D		∨	\	V		?
D			ι	C		×	/	V		⋄	N
	N	φ	B		≠	\	V		ρ		/
	!	C		÷	/	V		⌈	V	÷	2
○	A		^	\	V		⊂		~		↑
D		L	/	V		~	M	∨	0	∈	V

Crossword for Vector 18.2



Across

1. For each number in B, the number of numbers in vector L larger than it
5. The values of B, capped at 3 and rounded up to the next integer
8. Same as 1 + on an empty vector
9. Cosine of (pi times R) radians
10. $\star / TAB, 25 \dots 5$ for scalar TAB
13. Remainder when the absolute value of D is divided by N
14. $+/-R$
15. Where in V are the integers from 1 to T?
17. $V - G \times \lfloor V \div G$
18. Number of possible combinations of $\star / \iota N$ objects taken 3 at a time
19. Branch to the line labelled V1
21. e to the power T
22. Shape of $A \circ \cdot \times A$
23. 5 10 15 20 in origin 1
24. 2 if P>0; 0 if P=0
25. 3 random numbers from $\iota NV4$

26. $(\sim A \in \sim B) / A$
27. Same as 27 Down
28. Is T a scalar?
29. Stick a column of 2s on the right of matrix M
30. $\sim F$ for Boolean F
32. Rank of A
33. Set K as $Z[4 + 1(\rho Z) - 4]$
34. 1 2 3 ... I
35. Increase the depth of T by 1
36. Indices of the 1 bits in B
37. Length of V
38. $10 * + / 4 \rho VE$ for VE a scalar

Down

1. Cumulative sum of the difference between T and G times itself
2. $B \times V$ where B is a Boolean vector, without $\times$
3. $\div 5 \times \div 5 \times .05$
4. First true
5. Tangent of the magnitude of T radians
6. Circumference of a circle of diameter D, rounded up to the next integer
7. $1(-R) - 1 \mid -R$
9. $N \rho \square IO, \sim \square IO$ for N a scalar
11. $((T \times T) * .5) + A$
12. $\times / R \rho e$ (the mathematical constant e)
13. Where in vector N is A factorial?
15. Number of ways the elements of vector P can be combined in sets of V elements
16. Flag each distinct value in V when it first occurs
18. Sign ($\sim 1, 0, 1$) of the first three elements of matrix B
19. Exit the function if N=0; restart it if N>0
20. $\times / 2 \quad 3 * 2 * 2 \quad 1$
22. 2 if T=0; 0 if T=1
23. Shuffle the numbers 1 5
24. 1 0 if A is nested (one deep); else 2
25. $B 0[2 + 1 3]$
26. Is A the shape of T?
27. Same as 27 Across
28. An empty vector for which $\rho \square IO > 1 +$ returns ρM
29. Set M as $0 \rho 0$
31. 2+3
33. $(+ / K) \rho 1$

J-ottings 30: Jaesthetics

by Norman Thomson

Many J users take the definition of an adverb `each` as a pseudo-primitive, and find it more comfortable to use

```
each=.&>.
```

rather than `&>.` which is 25% shorter. However, there are in my view two compelling arguments in favour of `&>.`, viz. independence from any specific natural language, and avoidance of the ambiguity which arises because `each` in some contexts is also used to mean `&>.`

I argue in this article that there are matters of taste involved in writing J, and that J programmers inevitably make choices between pieces of code which are identical as far as effects on data are concerned.

A contributor to the J forum recently made the point that for J aficionados, J phrases can become a more natural medium for expressing ideas than any natural language equivalent. Jim Lucas stated that he didn't care for expressions with lots of `@`s in them, because `@` is a fundamentally ugly symbol. I also have some reservations about `@`, but for a different reason, namely that `@` and `@:`, imply **sequence**, and pieces of code strung together with `@`s look much like the kind of line-by-line code observed in more mundane computer languages.

Two examples illustrate both of these points. The first is `=@i.@#` which gives an identity matrix of the same shape as a square matrix (see J-ottings 29 for a motivation for defining this). This phrase has a good feeling of symmetry on account of the two `@`s. It is analogous to a word such as "fever" in which consonants and vowels alternate, only now read verbs and conjunctions. As for its meaning, the `@`s say that there are three things which have to be done in sequence, first a *tally*, then an *integers* of that *tally*, and finally a *self-classify* of the result of *integers*. If I were to assign and name this verb for repeat use I would probably call it something like MIM or MidMat, standing for Matching Identity Matrix. In this case I am not sure whether MidMat or `=@i.@#` conveys more on later recall.

Now consider the somewhat similar phrase `#i.@#` :

```
(#i.@#)0 1 0 1 1
1 3 4
```

which converts a bit string into the indices of the 1s. Again the phrase has an overall shape and feel, in this case that of a word like "trot". A name for assignment purposes might be something BtoInd which has a much lower mnemonic value than MldMat, and in this case I am confident that my preference for recall purposes is #i.@# .

Another recent exchange on the forum offered two approaches to the problem of obtaining a vector of occurrence numbers. The problem is that of taking a sequence such as

```
m='mississippi'
```

and obtaining the vector 0 0 0 1 1 2 3 2 0 1 3 in which each item of *m* is replaced by its occurrence number. Chris Burke and Cliff Reiter offered two different stylistic approaches. Here is Chris's:

```
ocb=.:([ ]-{})/:@/:)i.~
```

and here is Cliff's:

```
ocr=.;@(<@i.@#/.~)/:[:/:~.i.]
```

If I were completely fluent in J, I would be able to understand and compare the two on sight without recourse to English to define intermediate verbs. Although there is a temptation to break these down into smaller verbs, I ask you, the reader, to make the experiment of pretending at this point that J is the only language you "speak", and to make sense of these expressions on that basis alone.

Looking at *ocb* first, I have to say that I am not a fan of *cap* ([:) which I usually like to replace with an equivalent @, yielding in this case

```
ocb1=..([ ]-{})(/:@/:))@ i.~
```

The case made for *cap* when it was first introduced into J was that it enabled trains of indefinite length to be constructed without parentheses. In reading J phrases I find that "breathers" in the form of parentheses and @s are positively welcome, and the *cap* argument seems a bit like the case for removing gaps between movements of symphonies on the grounds that it gets you to the end sooner.

An obvious first step in solving the occurrence number problem is to translate the vector into an "order of first appearance" integer vector, that is every 'i' in *m* maps into the index of the leftmost 'i', namely 1, and similarly for the other characters.

This is just what $i.\sim m$ does:

```

]t=.i.\m
0 1 2 2 1 2 2 1 8 8 1

```

$t\{m$ is just m , since it makes no difference whether a selecting index is that of the first or any later occurrence. I can see therefore the role of the rightmost verb in ocb , and also note that, without loss of generality, the rest of this discussion can be restricted to vectors of this sort – this is the force of the rightmost $@$ in this case, and is incidentally a general useful property of $@$. What remains is clearly a hook, whose right verb is easily recognised as the upward ranking verb $/:@/:$. A significant problem in discussing the *grade* verbs is the promiscuity of the word “rank”, which means one thing in the context of J and another in the context of arithmetic generally. I think it best to use the word “ranking” to describe the latter sense and follow J practice of counting from index origin 0, so that the score of the best golfer has upward ranking 0, that of the fifth best golfer upward ranking 4 and so on. The upward ranking of a vector is the vector of the upward ranks of its items in their original order. The left verb of the hook

$()]-() (/:@/:)$

in ocb is a fork in which the effect of *right* is to make the right argument t the left argument of *minus*, the right argument of which is $t\{gt$ where gt is the upward ranking vector of t .

Now pause awhile and consider what happens when a vector of indexes of first appearances (call this an *ifo* vector for short, and define $ifo = i.\sim$) selects *from* its own upward ranking vector. Ranking in J is a *progressive* operation in the sense that if a vector contains more than one 0, the second 0 from the left is ranked 1, the third is ranked 2 and so on. However, in selection using an *ifo* vector, duplicate items in t repeatedly select the upward ranking of first occurrence making the process *non-progressive*. This is easier to observe by tracing than to describe in words

```

t
0 1 2 2 1 2 2 1 8 8 1
/:/:t
0 1 5 6 2 7 8 3 9 10 4
t{/:/:t
0 1 5 5 1 5 5 1 9 9 1

```

NB. upward ranking of t

NB. t indexing its own ranking

Subtracting the last two vectors in the above sequence gives the occurrence number vector.

```
      ocb m
0 0 0 1 1 2 3 2 0 1 3
```

All of this can be summarised by saying occurrence number is upward ranking minus index of first appearance. I could express this in mathematical symbols, but I doubt whether this would achieve anything like the clarity of the previous sentence. It is precisely this **derived** consequence of the semantics of *grade-up* and *index* which Chris has exploited delightfully in *ocb*.

The fork in *ocb1* raises a stylistic consideration. The purpose of `]` is to achieve the argument reversal needed to obtain

```
((/:/:t) - t{(:/:t)
```

I have reservations about using `[` and `]` similar to those I expressed concerning `@`, namely that `[` and `]` are a thinly disguised way of referring directly to the data arguments, and thereby causing reversion to traditional programming language style rather than that of tacit programming.

If *up* is defined as

```
up=.:@/:
```

then the fork in *ocb1* can be written as

```
up -({up)
```

and *ocb1* can be written

```
ocb2=.(up-({up}))@ifo
```

Of course I am now taking refuge in words, or at least verb-names as pseudo-words, no doubt thereby laying bare my shortcomings in J fluency. As far as recall is concerned, while *ifo* is quite memorable (although no shorter than *i.~*), the name *up* is uselessly general. I could of course follow the practice of C++ programmers and similar infidels, and call it something like *upward_ranking* but this would be self-defeating, since `/:@/:` is much shorter and infinitely more expressive.

As a spin-off, the hook `{up` delivers the non-progressive upward ranking in which all equal values have the same ranking. For example

```

      ({up) 0 1 2 0 2 5 0
0 3 4 0 4 6 0

```

Next consider Cliff's verb:

```
ocr=.;@(<@i.@#/.~)/:/:~.i.]
```

The first thing to observe is the presence of the *key* adverb (/.~) which is what I imagine many J users, myself included, would latch on to as the germ from which to develop a solution to this problem. Again pretend that J is the only available vehicle for communication. The presence of several @s indicates that Cliff has taken an essentially sequencing approach, particularly as *cap* can, as before, be exchanged for @ to give

```
ocr1=.;@(<@i.@#/.~) /: /:@( ~. i. ])
```

This reveals that the main structure is a fork with the leftmost /: as its central prong. As far as the right prong is concerned, two things happen in sequence. First the phrase within the parentheses is *index* by *nub*, for example

```

      (~.i.])m
0 1 2 2 1 2 2 1 3 3 1

```

which I recognise as another way of describing *ifo*, and incidentally conveniently deals with my distaste for]!

Then this is graded upwards

```

      /:ifo m
0 1 4 7 10 2 3 5 6 8 9

```

to give the **positions** of the 0s, followed by the positions of the 1s, followed by the positions of the 2s and so on, in other words it is the vector of positions required to *grade-up* dyadically the sequence of all occurrence numbers ordered by item.

Key seems a natural way of obtaining this sequence. We have already seen how *i.@#* works

```

      (i.@#)m
0 1 2 3 4 5 6 7 8 9 10

```

Applying *key* using the items of *m* as keys gives

```
m(i.@#/. )m
0 0 0 0
0 1 2 3
0 1 2 3
0 1 0 0
```

which is then *boxed* followed by *raze* - a standard device to remove unwanted fill characters. Putting all of this together and including a *reflex* to keep things monadic, the left prong may be summarised as

```
ocnobykey=.;@(<@i.@#/.-)
```

and the left prong by

```
posbykey=.:@ifo
```

leading to a further redefinition of *ocr* as

```
ocr2=. ocnobykey /: posbykey
```

An objection to this use of pseudo-English is that revisiting *sortbykey* and *posbykey* again poses as great a problem in remembering exactly what was meant by these rather vague verb names as that of interpreting directly the unambiguous J strings. The only significant arguments for decomposition of long verbs into shorter verbs is the "breather" effect on the one hand and clarification of top-down structure on the other.

In summary, three separate dimensions of stylistic choice have evolved in the above discussion which have to be made in J programming. First there is the balance between English and J, (with 100% of the latter not ruled out), then secondly the different problem solving approaches of "derive a new property from old" versus "program using existing resources" (*ocb* style versus *ocr* style), and thirdly, choices between J equivalences such as *cap* versus *atop*, as well as avoidance techniques for [and]. Of course none of this takes account of the hugely important dimensions associated with implementation - often a longer piece of code will be appreciably more efficient than a more elegant alternative. At this point art and science come to a divide where things begin to get severely practical - most certainly relevant, but not the province of Jaesthetics!

Getting Started with A

notes by Adrian Smith

Installation

Just download the ZIP files from www.vector.org.uk/aflat/download, unzip into the directory of your choice, and double-click **aplus.exe** to get it running. You may like to associate **aedit.exe** with the .a extension so that you can double-click on scripts. To run aplus from the command prompt (under Windows 2000) please use the supplied batch file.

Using the Console in ASCII Mode

This has to be the first thing everyone will try, so here we go ...

```
$mode ascii
2+2
4
```

Now you will want to try out subtraction ...

```
2-2
2 -2
2 - 2
0
```

OK, what went wrong the first time? In ASCII mode you must use white space around symbols and names to be safe! A {minus} sign adjacent to a number is a [highbar] in this case. How about some real APL:

```
rev 1 2
//[parse] .rev: var?
reverse 1 2
//[parse] .reverse: var?

rot 1 2
2 1
1 rot iota 5
1 2 3 4 0
```

Well, I guessed wrong, probably so did you. The ASCII names are a bit arbitrary and follow the standard Unix convention of 'weird short codes are best' so here is a table to get you started.

APL symbols and ASCII names

APL	ASCII	APL	ASCII	APL	ASCII	APL	ASCII
$\leftarrow$	:=	$\ddot{}$	each	$\wedge$	&	$\vee$	?
$\times$	*	$\div$	%	$\otimes$	log	$\equiv$	==
$\lceil$	max	$\circ.$	fn.	$\lfloor$	min	$\in$	in
$\neq$	$\sim$	$\leq$	$<=$	$\geq$	$>=$	$\ast$	$\wedge$
ϕ	rot	ϕ	flip	ι	iota	ρ	rho
$\uparrow$	take	$\downarrow$	drop	$\uparrow$	upg	ψ	dng
$\perp$	pack	$\top$	unpack	$\subset$	bag	$\supset$	pick
$\pmb{\&}$	eval	$\pmb{\&}$	form	$\vdash$	rtack	$\neg$	where
$\circ$	pi	$\boxdiv$	mdiv	$\?$	rand	$\pmb{\&}$	beam
$\%$	ref	$\cup$	dot	$\pmb{\&}$	//	$\sim\sim$	bwnot
$\wedge\sim$	bwand	$\vee\sim$	bwor	$<\sim$	bwlt	$\leq\sim$	bwle
$=\sim$	bweq	$\geq\sim$	bwge	$>\sim$	bwgt	$\neq\sim$	bwne

We can even define simple functions ...

```
total v: {+/v}
total iota 12
66
mean v: {(total v) % #v}
# 2 3 4
3
mean iota 12.
5.5
```

You use % as {divide} here, but note that A implements {tally} with monadic # in J fashion. This makes your function generalise very nicely to higher-rank tables:

```

    mat := 2 3 rho iota 6
    mat
0 1 2
3 4 5
    mat := iota 2 3
    mat
0 1 2
3 4 5
    total mat
3 5 7
    mean mat
1.5 2.5 3.5
    # mat
2
    rho mat
2 3

```

Hopefully, that is self-explanatory. A uses the *first* dimension by default (like J) and {tally} only sees the outermost element of the shape. You might also notice that {iota} has some built-in reshape capabilities. Dyalog fans might object here ... hard to know which extension is more useful in practice.

What if it all goes wrong?

Sooner or later you will make a mistake, for example mis-typing a name. The A equivalent of)reset is \$reset (probably you guessed that) and the equivalent of a bare right-arrow is \$ on its own, so:

```

    fred
a[error] .fred: value
*
$
    2+2
4

```

... when faced with * in the prompt string, get into the habit of typing \$ at the input prompt to get rid of it.

Multi-line functions are possible – just

This reminds me of typewriter terminals and the {del} editor! Yes, you can create simple functions in the console, but really you should be writing them in a script and running it! Just to show that it can be done:

```

      fluffy n: {
*      fred := 12;
*      joe := 23;
*      fred + joe + n }

```

```

      fluffy 7

```

```

42

```

Note carefully the semi-colons which act as statement separators. Newlines are completely immaterial, as you can easily show:

```

      coo n: {
*      fred :=
*      12;
*      fred + n }

```

```

      coo
coo n: {
fred :=
12;
fred + n }
      coo 3

```

```

15

```

Notice that you display a function simply by typing its name. By implication, you cannot really have niladic functions in A – again just like J and possibly an irritant to APL old-timers? Now watch what happens if you slip up and let your semi-colon reflex run away with you:

```

      foo x: {x+2;}
      foo 3

```

Oh, oh. The result of an A function is the result of the last executable statement, which in this case is empty. This function ALWAYS returns null. Be careful! Of course it could have a side-effect:

```

      goo v: { .qq := 12; }
      goo 100
      qq
12
      hoo v: { my.qq := 25; }
      hoo 0
      my.qq
25
      qq
12
      $cx my
      qq
25

```

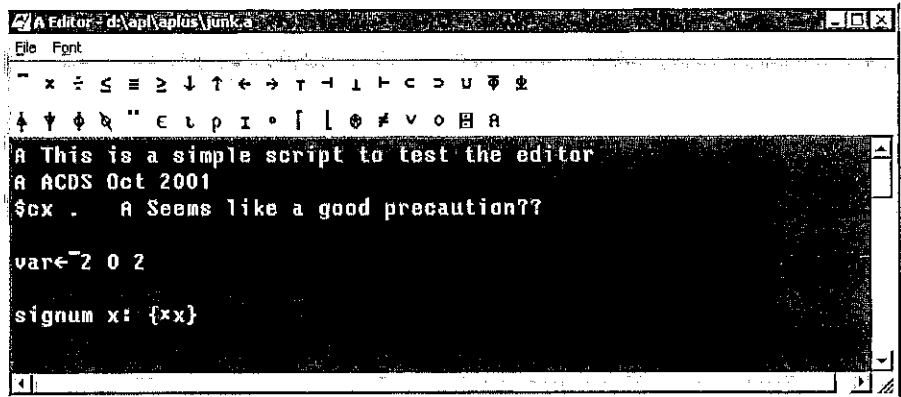
All unqualified names are strictly local, but if you provide a context for the assignment (the root context is just '.' in A) then the assignment (or reference) becomes global. Note that simply mentioning a context in an assignment is all we need to do to have it created for us. There is a slight oddity here, in that you can create contexts as deep as you like ...

```
$cx my.var
qq := 12
qq
12
$cx .
my.var.qq
//[parse] my.var: var?
```

... but the parser only copes with one level of qualification, so this is probably more of a bug than a feature, and we should assume the same capability as J has with locales.

Writing and Running Scripts

Remember that A has no concept of a saved workspace. The idea is that you write scripts (in the editor of your choice) and run them in the console. Well, it seems to work for the J community but some of us will soon be hankering after \$save, I am sure. Anyway, here is how it is supposed to work. Start the editor and type in something like:



The tool buttons all have tips if you are not familiar with the keyboard layout. Save it with the '.a' extension and switch to the A console to run:

```

$load junk
This should show ^1 0 1 if we understood correctly!
^1 0 1

```

Note that this runs in 'APL' mode even though you have set the console running in ASCII. You can set the script to ASCII mode if you prefer to see the APL {highbar} as a conventional minus sign here.

Now you just need to get into the habit of editing your script (probably called test.a or some such), selecting File,Save, clicking over into the A window and typing \$load test to run it. Remember that you *cannot use the up-arrow* to recall prior lines here, so keep script names short to save typing! If it fails, \$reset, fix it in the editor, save it and try again! Of course scripts can call other scripts, so if you want to have a 'master script' to load lots of others, that will work fine.

Some Examples from "APL for the Mathematics Classroom"

One way to get a quick feel for the compatibility of A and APL is to translate a few useful code fragments from one to the other. Norman Thomson's book is a good starting point, as he wrote this with I-APL in mind, so it is more or less in 'Direct Definition' style already. Here are a few functions from the section on *Random Numbers from Various Distributions* (pages 106-107).

Random Numbers

Compare this with Norman's originals ...

```

^ R random numbers from the uniform distribution in the range (0,1)
RND R: {(?T)÷T÷Rρ100000}

^ L random numbers from uniform distribution in the interval R[1],R[2]
L RUN R: {R[0]+(RND L)×R[1]-R[0]}

^ L random numbers from -ve exponential distribution, mean R
L RNE R: {-R÷RND L}

^ L random numbers from Normal dist, mean R[0] and SD R[1]
L RNO R: {R[0]+R[1]×((2×L RNE 1)÷0.5)×2002×RND L}

^ L random booleans where probability of drawing 1 is R
L RBO R: {R>RND L}

```

We need to switch to zero-origin indexing, and we also discover that Norman's use of `-/R[2 1]` must be rewritten in a more mundane way. The rest of it looks (and works) just as it did before:

```

load maths
RND 5
0.24219 0.10639 0.81261 0.4204 0.89144
4 RUN 10 20
13.1941 10.7872 13.5569 11.4972
20 RBO .5
0 0 1 0 0 0 1.0 1 1 1 0 1 0 0 1 0 1 0 0

```

A few more functions from the following page ...

```

a L random vars from the set (pR ...
L RSA R : {+/@1(RND L)*.≥0,~1+~R}

a Frequency distribution for a sample such as the above
L RFD R : {+/@1(1+~pR)*.= [L RSA R]

```

Note that we need to specify the axis on the sum (as A defaults to the first axis) and adjust for the index origin in the second example.

Newton-Raphson Method

This is more challenging, as it requires unpicking the 'old-style' compare-and-branch style of switching to the modern approach with control structures.

```

L NEWRAP R: { a Root-finder by NR method
T+{(R1';')~R; T1+{(1+R1';')~R; a Split expr;derivative
X~L[2]; I+0;

while (L[0]<|X-Z ~ Z~X-(~T)÷(~T1)) {
I+I+1;
if (0=L[1]|I) a See some output
+ 'AT ITERATION', (~I), ' X=', (3+L)~X;
X+Z };
'SOLUTION = ', ((3+L)~X), ' AT ITERATION', ~I
}

```

Again the index-origin change hits us - the code to split the string is a less-obvious amendment here. Note the use of the + to get A to spit out the intermediate results to the console. Also the syntax for dyadic format is slightly different which changes the way we run the example. The positive thing about this example is that the overall structure and layout will look very familiar to anyone who has written JavaScript or PERL; with a few renamed variables it should be very acceptable to the 'modern' programmer!

```

Testnr .00001 2 2 7.4
AT ITERATION 2 X= 1.5000
SOLUTION = 1.4142 AT ITERATION 3

```

A Quick Translation from the J

This might be another interesting script to read through. It implements the 'mississippi' occurrence function from Jottings, which you will find implemented in J on page 50 of this Vector.

```

A The Mississippi challenge (from Vector 18.2 - Jottings)
A The object is to take a vector and return the occurrence numbers
  s ← 'mississippi'

A The boring way with an outer product
nub v: { ((v\ v)=\pv)/v }
unq ← nub s
tbl ← unq o.= s
A Cum across and save where we had numbers
tbl ← tbl × +\@1 tbl
oc ← ~1+÷/tbl
'The answer is ',⍋oc

A Make it a function so we can time it ...
oca s : { tbl ← (nub s)o.=s ; tbl ← tbl × +\@1 tbl; {÷/tbl}-1}

A Now for the super-sneaky approach (J-forum written up in Vector)
t ← (s\unq)[unqis]
r ← ⍋t
occ ← r-r[t]
'The answer is also ',⍋occ

A Get a little nearer the J style using index ...
t ← (unqis)#s\unq
r ← ⍋t
occ ← r-t#r
'The answer is still ',⍋occ

A Finally as a function
ocb s: { u←nub s; t←(uis)#s\u; r←⍋t; r-t#r }

' oca s    A tests the boring one'
' ocb s    A tests the sneaky version'
' '

'Try some timings with various sized arrays ...'
' time qq := ocb 500000 rho s    A Arthur wrote a mean gradeup here!'

```

Over to you!

Maybe someone would like to have a go at something 'substantial' in A, for example the *VierNeun* puzzle or the 'funny cube' would be good subjects where there is already code in APL, J and K for comparison. Please send in any examples to Vector Production and we will include as many as we can on the website.

An Elementary Tutorial on Conjunctions

by Donald B. McIntyre (donald.mcintyre@ntlworld.com)
reproduced from the J Forum with permission

Introduction

J is an Executable Mathematical Notation, which explains J's conciseness, completeness, and consistency. This is the source of J's power and the reason for its appeal. J's elements are nouns, verbs, adverbs, and conjunctions. Commonly: nouns are items of data; verbs (mathematical functions) take either one or two noun arguments and have noun results; adverbs take a single (left) argument – usually a verb – and have a verb result; and conjunctions (mathematical operators) have two arguments (either nouns or verbs) and have verb results. J also supports the creation of new verbs, adverbs, and conjunctions by composition of functions, based on either conjunctions or trains of elements. The 25 possible tridents and the 12 possible bidents are listed in the J Dictionary II F. This tutorial illustrates 12 tridents and 4 bidents.

New users quickly learn how to define a verb and how to use adverbs and conjunctions, but it is less well known that users can define their own adverbs and conjunctions. Brian M. Schott (Georgia State University) recently asked me for an illustration, and this tutorial (which developed from my reply), shows how to read and write both explicit and tacit definitions. Brian cited articles posted to the c.l.a Newsgroup in October 1992, and the first of these is my starting point.

In a paper on "The Role of Composition in Computer Programming" I discussed defined adverbs and conjunctions; e.g. plotting one monadic function against another (APL95 Proceedings. See <http://homepage.ntlworld.com/donald.mcintyre> for a pdf version). "A conjunction can be used when an expression containing two verbs is generalized so that other verbs can be substituted. Defined adverbs and conjunctions are miniature computer programs with both verb and noun arguments."

This tutorial is intended to be read – and, where appropriate, executed – after reading the Dictionary II E (Parsing and Execution) and II F (Trains). The pages should be open for reference to the tables of tridents and bidents in II F.

Keep in mind that (a) execution proceeds from right to left except where modified by parentheses; (b) a conjunction's left argument is the entire verb phrase that

precedes it, and its right argument is the item immediately to its right; and (c) adverbs and conjunctions are executed before verbs.

I hope that what follows helps users to understand better 'how J works'.

Conjunctions in 1992

In October 1992, at Ken Iverson's invitation, I gave J workshops in London (UK), Toronto, Waterloo (ONT), Lawrence (KS), Claremont (CA), San Jose (CA), Dallas (TX), Phoenix (AZ), Washington (DC), and Wooster (OH). On 1992 10 21 J. Patrick Harrington (Astronomy, U. of Maryland) sent the first of a series of interesting messages on J to the comp.lang.apl Newsgroup. He referred to these messages as his "musings". His title was: "Tacit numerical integration?"

The conjunction D. was implemented in J Version 6.2, 1992 12 20; i.e. two months after my Workshop in Washington. Using D. we can show that the first derivative of the sine is the cosine:

```
sin=: 1&o. [. cos=: 2&o.
```

```
cos n -: sin D.1 "0 n=: 0.1*i. 7
1
```

Note that D. is a conjunction, which with its noun right argument (1) is an adverb, here modifying the verb sin.

```
class=: 4!0
C=: D.      NB. Conjunction   Valence 2
A=: D.1     NB. Adverb       Valence 1. C0 N1 is x C0 N1 (bident II F p.103)

class ;/'CA'
2 1
```

In his message on 21 Oct 1992 Patrick Harrington wrote:

"On the Oct 10-11 weekend, Donald McIntyre gave two fine workshops on J here in Washington, which I found both interesting and useful. As is usual with J, many new questions sprouted, Hydra-like, as the old ones were lopped off. One bit I found interesting was the use of]. and [. to construct conjunctions. Thus he showed that

```
D=: ([. (%&))@([. @([. (+&))-].)
```

is a conjunction. If we let h be a small number, say $h = .1e_8$, then the expression $(h \ D)$ is an adverb such that for a function (verb) f which applies to a noun n ,

$$f(h \ D) \ n$$

is the approximate derivative $(f(n+h) - f(n))\%h$.

e.g.,

```
f=: 1&o.          NB. f is sin(n)
] n=: 0.1*(i. 7)
0 0.1 0.2 0.3 0.4 0.5 0.6
  f (h D) n
1 0.995004 0.980067 0.955336 0.921061 0.877583 0.825336
  2 o. n
1 0.995004 0.980067 0.955336 0.921061 0.877583 0.825336
```

This makes me think that one should be able to construct a conjunction X such that $f(h \ X) \ n$ would take one step of Runge-Kutta integration, say, from x to $x+h$, where the function f describes the differential equation. ... "

Others can deal better than I can with what Patrick Harrington went on to discuss, but I concentrate on the conjunction that started his train of thought; namely:

$$D =: ([. (\%&)) @ ([. @ ([. (+&)) -].)$$

Remembering Whitehead's remark that "nothing is more incomprehensible than symbolism we do not understand", let's remove the mystery from what may appear a forbidding expression.

```
f=: 1&o.      NB. sin
'h n'=: 1e_8; 0.1 * i.8
class 'h';'n';'f';'D' NB. 2 nouns, a verb, and a conjunction
0 0 3 2
```

With a step of h , the approximate derivative of f at n is given by:

```
]z=: ((f n+h) - f n) % h      NB. Immediate execution
1 0.9950041652 0.9800665779 0.9553364877 0.9210609886 0.8775825622 0.8253356
11 0.7648421896
```

Writing this as $z=: \%h ((f n+h) - f n)$ already removes some of the mystery from D because the similarity between the two expressions is not hard to find:

$$D = : ([. (%&)) @ ([. (+&)) -] .) \\ \quad \%&h \quad ((f \quad n+h) - f \quad n)$$

The result is the quotient of two quantities. This suggests the fork trident numerator % denominator, analogous to `mean=: +/ % #` (the mean is the sum of the items divided by the number of items).

The numerator `((f n+h) - f n)` is easily recast as a fork:

$$(f@+&h - f) \quad n \quad \text{NB. } n \text{ is the tacit argument}$$

A fork, however, is a trident of three verbs (V0 V1 V2), and in our case the denominator (h) is a noun. We need, therefore, a verb that returns the constant h whatever its argument, and such a verb is created by the rank conjunction:

`h"_` (N0 C1 N2 is a verb). I have not found in the Status Reports a record of its introduction, but it was supported in J Version 7, 1993 11 14. If, indeed, it was not supported in J 5.1, 1992 7 28, this would explain why the D conjunction used in the Workshop took the form `([.(%&))@(f@+&h - f)` instead of the fork `(f@+&h - f) % h"`. It is more likely, however, that in October 1992 I had not yet learned how to use the feature.

Explicit Definition

We need a conjunction combining h and f to make a verb; we could then use this verb with argument n. The required conjunction is easily constructed using explicit definition (Dictionary p.153-156, p.106-107; Phrases p.20-24). In Ken Iverson's words:

"All that is needed to understand the explicit form is the pattern:

$$x. \quad u. \quad \text{conj} \quad v. \quad y.$$

where x. and y. are the arguments of the function produced by the conjunction, and where u. and v. are its arguments (possibly replaced by m. and n. to restrict them to nouns)."

Replace f h and n by v. m. and y. respectively. (In our case there is no x. .)

Reference to the arguments x. and y. is explicit. Note how explicit definition parallels immediate execution:

<code>z=: ((f n + h) - f n) % h</code>	NB. Immediate execution
<code>ve=: 3 : '((f n + h) - f n) % h'</code>	NB. Verb (explicit)
<code>ce=: 2 : '((v. y.+ m.) - v. y.) % m.'</code>	NB. Conjunction (explicit)

The explicit verb `ve` is included only to emphasize the parallelism between verb and conjunction. `J` does not support niladic verbs; the verb `ve` takes but ignores its argument.

```
z-: ve 3.14
z-: (h ce f) n    NB. verb      Parentheses redundant
z-: f (h ce) n    NB. adverb    Parentheses required
```

To save space, the `1` resulting from all instances of `z-:` will be omitted.

The parentheses in `(h ce f) n` are redundant. This is because the conjunction takes `f` as its right argument, thus effectively introducing a right parenthesis between `f` and `n`. Parentheses merely draw attention to a phrase which creates a verb.

Parentheses in `f (h conj) n` are required. The phrase `N0 C1` creates an adverb. Without parentheses, `n` would be the conjunction's right argument.

Explicit definitions can be written in an alternative form without quotation marks:

```
ce=: 2 : 0
  (( v. y.+ m.) - v. y.) % m.
)

v=: h ce f      NB. verb      N0 C1 V2
a=: h ce        NB. adverb    N0 C1

class 'v';'ce';'a' NB. verb conjunction adverb
3 2 1
z-: v n
z-: f a n
```

Tacit Definition

Unlike explicit definition, tacit definition makes no direct reference to the arguments (`x.` and `y.`) of the resulting function.

```
g=: f@+ - f@]    NB. verb with tacit arguments (h and n)
```

`V0 C1 V2` is a verb; e.g. `(f @ +)` and `(f @ ])` These are not forks!

`V0 V1 V2` is a verb; `g` illustrates this trident; `g` is a fork.

Remember: A trident is a fork only if all three components are verbs!

`f@+` applies `f` to the sum of its two arguments

`f@]` applies `f` to its right argument alone

```
z-: (h g n) % h      NB. (n v n) is n

z-: ((f n + h) - f n) % h      NB. Immediate execution
vt1=: (f@ (+&h) - f) % h" _    NB. Tacit verb
vt2=: (f@ +&h - f) % h" _     NB. Only required parentheses

hv=: h" _                NB. n c n A "constant" verb yielding h
class 'h';'hv';'vt1';'vt2' NB. noun and 3 verbs
0 3 3 3
```

`f@+&h` is the train `v c v c n`. Because a conjunction takes the item immediately to its right as its right argument, the train is parsed `((v c v) c n)`. This is demonstrated by a simple model:

```
+ & + & 2
(+&+)&2
```

Because both `(v c v)` and `(v c n)` are verbs, the result is `(v - f)`, which is a fork `(V0 V1 V2)`.

`h" _` is a verb `(N0 C1 N2)`, consequently `vt2` is a train of three verbs and is therefore another fork.

```
z-: vt1 n
z-: vt2 n
```

The distinction between verbs and conjunctions must not be overlooked!

```
left=: [      NB. verb
right=: ]      NB. verb

lev=: [.      NB. conjunction
dex=: ].      NB. conjunction

class 'left';'right';'lev';'dex'
3 3 2 2
```

In a tacit definition the `u.` and `v.` of an explicit definition are replaced by the conjunctions `[.` and `].`

Note the parallelism of the following:

```

z=: ((f n + h) - f n) % h      NB. Immediate execution
ce=: 2 : '((v. y.+ m.) - v. y.) % m.' NB. Conjunction (explicit y.)
vt=: ( f @ +&h - f ) % h" _    NB. Verb (tacit reference to y.)
ct=: ( ].@ +&[. - ].) % ([. " _) NB. Conjunction (tacit)

```

Although all four are based on the paradigm (numerator % denominator), only *vt* and *ct* are (V0 V1 V2) tridents; i.e. forks. In tacit forms the noun *h* in the right tine is replaced by the (n c n) verb (*h" _*), or by the (c c n) conjunction (*[. " _*). Parentheses prevent *[.* taking % as its left argument. Because *h" _* is a verb, parentheses are not needed.

The left tine of *vt* is (v c v c n v v) or (((v c v) c n) v v)

or ((v c n) v v) or (v v v) or v, which is a fork.

```

v c v c n v v
+ & + & 3 + +
((+&+)&3) + +

```

Consequently *vt* is *v % h" _* or *v v (n c n)* or *v v v*

The left tine of *ct*:

```
(([. @ +) & [. ] - ].) is the train c c v c c v c
```

A simple model shows how this is parsed:

```

(@ @ + @ @ + @)
((@ @ +) @ @) + @

```

The parenthesized display of *ct* looks like a fork, but this is misleading.

```

ct
((([. @ +) & [. ] - ].) % ([. " _)

```

The pieces are:

```

w1=: ].@+      NB. c c v or c
w2=: w1&[.     NB. c c c or c
w3=: w2 - ].   NB. The left tine c v c or c This is not a fork!
w4=: [." _     NB. c c n or c
w=: w3 % w4    NB. c v c or c
class 'ct'; 'w1'; 'w2'; 'w3'; 'w4'; 'w' NB. All are conjunctions
2 2 2 2 2 2

```

```

z-: h w f n
z-: vt n          NB. verb
z-: (h ce f) n    NB. verb. Parentheses redundant
z-: (h ct f) n    NB. verb. Parentheses redundant
z-: f (h ct) n    NB. adverb

```

(h" _ +]) is a fork because h" _ + and] are verbs, hence v v v

```

z1=: f@ (h" _ + ]) n

c1=: ].@ ([" _ + ])
c1
]. @ ([" _ + ])
c c ((c c n) v v) or c c (c v v) or c c c or c

@ @ (@ @ 3 + +)
@ @ ((@ @ 3) + +)

z1=: h c1 f n

z2=: (f@ (h" _ + ])- f) n NB. Fork whose left tine contains a fork
c2=: ].@ ([" _ + ]) - ]. NB. No forks. [" _ is a conjunction

c c (c v v) v c
@ @ (@ + +) + @
(@ @ (@ + +)) + @
(@ @ @) + @
@ + @
@

z2=: h c2 f n

c3=: c2 % ([" _ ) NB. c v c Not a fork. Parentheses required.
z-: h c3 f n

c4=: (].@ ([" _ + ]) - ].) % ([" _ )
z-: h c4 f n

(((]. @ +) & [.) - ].) % ([" _ )
Ce=: 2 : '(y.@(x." _ + ]) - y.) % x." _' NB. x." _ must be a verb for (v v v)
z-: h Ce f n

```

Invoke Gerund: Dictionary p.201

"Invoke gerund" is another way of creating a conjunction. We need to combine two functions: (a) the difference between (f (h+n) - f h), and (b) division by h.

```

x1=: % & [.
x2=: ]. @ + & [. - ].
ct2=: x1@x2          NB. C0 C1 C2
z=: h ct2 f n

```

As the Dictionary explains: "Caps make it possible to define a wider range of functions as unbroken trains." The verb [: normally caps a left branch of a fork (V0 V1 V2)

```

cap=: [:
ct3=: cap ` x1 ` x2 `:6
z=: h ct3 f n
class 'cap'; 'x1'; 'x2'      NB. v c c
3 2 2

```

x1 and x2 are conjunctions which generate verbs when associated with the arguments h and f.

A fork (V0 V1 V2) yields a verb, whereas ct2 is C0 C1 C2, and

ct3 is v c c c c c which is ((v c c) c c) or c c c which yields a conjunction.

```

ct4=: [: ` (%&[.) ` ([.@+&[. - ].) `:6 NB. Parentheses necessary
z=: (h ct4 f) n

```

Tacit Translator

```

C1=: 12 : '(y.@(x."_ + ]) - y.) % x."_'
z=: h C1 f n
1
C1
([. @ (([. " _ + ]) - ].) % ([. " _)

```

```

ct NB. Linear
((([. @ +) & [.] - ].) % ([. " _)

```

```

ct=: ([.@+&[. - ].) % ([. "_ NB. Only required parentheses

```

The classes of the elements of this train of conjunctions and verbs are:

```

(c c v c c v c) v (c c n)

```

Processing of ct is from right to left. Each conjunction 'grabs' the item to its right. Phrases in parentheses are evaluated first.

Notes

"Parsing proceeds by moving successive elements ... from the tail end of a queue ... to the top of a stack. ... The executions in the stack are confined to the first four elements only, and eligibility for execution is determined only by the class of each element ..., as prescribed in the parse table on p.101." [Dictionary II E]

"Because conjunctions have long left-scope and short right-scope, the phrase 5!:4@< (noun conjunction noun conjunction verb) parses as ((n c n) c v), which in turn resolves into (v c v), and finally to verb. ... Although execution proceeds from right to left, a sequence of verbs [and nouns] joined by conjunctions is parsed as if parenthesized from left to right. The consequences of this are easily overlooked. When in doubt, consult the boxed display and [if need be] use parentheses to control the desired order of execution." [The Role of Composition in Computer Programming, APL95 Proceedings]

c c v c c v c is therefore parsed by taking the final c c v c into the stack. The final triad c v c is not executed because it is preceded by a conjunction which grabs the leftmost c of the triad so that the initial result is c c) c v.

Proceeding in this manner from right to left, the parenthesized result is:

((c c v) c c) v c. Although the sentence is indeed parsed from right to left, the sequence of execution is modified by these implied parentheses. Thus,

(c c v) is c and (c c c) v c is C0 V1 C2 which in turn is c.

As this point is often misunderstood by a user encountering a train with conjunctions, it is necessary to understand the implications of the following statement: "Execution proceeds from right to left, except that when a right parenthesis is encountered, the segment enclosed by it and its matching left parenthesis is executed, and its result replaces the entire segment and its enclosing parentheses." (Dictionary II E item 1.)

It is also helpful to observe the parsing by using the trace conjunction 13!:16 (See Dictionary p.100, 285).

A simple test for our example is to enter and display a model of the train; e.g.

```
NB.      (c c v c c v c ) v (c c n)
          (@ @ + @ @ + @ ) + (@ @ 3)
          (((@ @ +) @ @) + @) + (@ @ 3)
```

z-: ((f n+h) - f n) % h NB. Immediate execution

If you can write a 2 : x definition of a conjunction using x. and y. without reference to u. and v., thus:

```
Ce=: 2 : '(y.@(x."_ + ]) - y.) % x.'"_
z=: h Ce f n
```

then the conjunction can be automatically translated to tacit style, thus:

```
Ct=: 12 : '(y.@(x."_ + ]) - y.) % x.'"_
Ct
([. @ ([. " _ + ])) - ].) % ([. " _)
z=: h Ct f n
```

Note that in Ce x."_ does not need parentheses. The translator recognises forks by our use of parentheses and by our making x. into a verb (i.e. x."_)

Attempting to justify those parentheses that are required is a useful exercise; e.g.

```
Ct1=: ([. @ ([. " _ + ]) - ].) % ([. " _)
z=: h Ct1 f n
```

Use these models to clarify how the parsing rules work:

```
@ @ @ @ 3 + + + @ + @ @ 3
((( @ @ @ ) @ 3) (+ + + @) @ 3
( @ @ ( @ @ 3 + + ) + @ ) + ( @ @ 3 )
(( @ @ (( @ @ 3 ) + + )) + @ ) + ( @ @ 3 )
```

Finally, when examining tree and box displays be careful to avoid interpreting every trident as a fork:

```
tree=: 5!:4@< [ . box=: 5!:2@<
tree 'Ct1'
box 'Ct1'
```

Concluding Remarks

In a brief response to Patrick Harrington on 1992 10 23 I said: "Everything of significant merit that I disclosed is due to Ken Iverson (e.g. the conjunction D) or Roger Hui. I simply put some of their gems in a context that I found satisfying."

I should state that this is still true.

The next day Roger referred to what Patrick called his 'monolog', and particularly to this comment by Patrick on tacit definition:

"I would like a conjunction X so that $(h\ X)$ is an adverb, and $f\ (h\ X)\ n$ is the integration step. And of course, in tacit form. This is supposed to be a *good thing*, although just why this is so is not yet clear to me."

Roger's response to Patrick's implied question included the following points:

"Some advantages are:

- 0) The tacit form encourages the use of components, primitive and user-defined. Use of components is one of the few effective ways of dealing with complexity.
- 1) The tacit form focuses on how components are combined, i.e. it focuses on the interface between components, where errors tend to occur.
- 2) Tacit expressions are algebraic, i.e. they are amenable to proof and other formal manipulations.
- 3) Tacit expressions are constructed using operators (adverbs and conjunctions), ideas debugged in mathematics over hundreds of years."

But it strikes me that a fourth order Runge-Kutta formula in tacit J could be a very ugly beast. And this is a concern, that on any non-trivial problem, finding the tacit forms may be too hard to be worthwhile. [JPH]

"I do not believe it is necessarily ugly, at least not if there is judicious use of well-chosen subfunctions. " [RH]

Ken Iverson gave me the following advice:

"I don't think it wise to strive too hard to write tacit definitions for conjunctions: it is not always possible, and it is almost always much more difficult to understand than the explicit form."

"During the period of design I did all I could to use only the tacit form, because this tested the possibilities and led to new forms (such as $f\ conj$ to define an adverb). Now I feel more relaxed about using the explicit form, even for verbs (although the use of the tacit form may lead to simple structuring, as in $k =: f\ @\ (g+h)$, with subsequent definition of the components)."

Roger Hui recently made the following comments to me regarding his preference for explicit or tacit definitions:

"I myself would use whatever form is clearer and more effective, and that may be tacit in some cases and explicit in others. For myself, adverbs and conjunctions are more often clearer in the explicit form."

And further:

"A rule of thumb that I find helpful: If I can not write a conjunction (adverb) without resorting to the explicit form that uses both $x.$ $y.$ and $u.$ $v.$ (and/or $m.$ $n.$), then probably the conjunction should not be written tacitly (much too complicated to easily understand)."

"If the conjunction uses only $x.$ and $y.$ (i.e. only the operator arguments and not also the derived verb arguments), that means the conjunction produces a result (the derived verb) which is tacit, and I believe (I need to prove this) there is an automatic procedure to convert such a conjunction to tacit form. (12 : x is this automatic procedure.) As well, the result of this automatic procedure is not too convoluted for my brain to grasp."

Acknowledgements

Thanks to Brian Schott for posing the question and reminding me of Patrick Harrington's "musings"; to Patrick for responding to the 1992 workshop; to Keith Smillie for suggestions and encouragement; and to Ken Iverson and Roger Hui for making all this possible, for many patient explanations, and for permission to quote their very helpful advice and observations. Remaining errors are mine, and I would appreciate receiving comments and corrections.

Points that may be of insufficient general interest to post on J Forum can be sent to me at:

donald.mcintyre@ntlworld.com

Footnote

After my tutorial on the definition of new conjunctions was posted on the J Forum, John Wilson asked a related question: "(Why) are the expressions (functions) $f@(+\&h)$ and $(f@+)\&h$ equivalent?"

Henry Rich correctly replied that "for a function f that has different behavior when applied at rank 0 and rank $_$, the expressions are not equivalent. ... If $@:$ were used in place of $@$, the expressions would be equivalent. For the actual f that was used, they are equivalent."

```
f=: <
h=: 5
f@(+&h) 1 2 3
+-----+
|6 7 8|
+-----+
(f@+)&h 1 2 3
+-----+
|6 | 7| 8|
+-----+
f@:(+&h) 1 2 3
+-----+
|6 7 8|
+-----+
(f@:+)&h 1 2 3
+-----+
|6 7 8|
+-----+
```

Using Structures between GUI and Relational Databases

by Tomas Gustafsson (*weis@sci.fi*)

"In that dialog box there, please insert an edit field for Age between Date of Birth and Id Card Number"...

The product support guy who said that did probably not realise the impact his words would cause on the program code for this almost completed application. All the GUI pieces were already in place, all the column keys in the relational database system were set, all the links and validation routines in between were functional, etc. etc. Now the coder has among others to:

- Make space for that new edit field in the page in that PropertySheet, by moving all GUI components below or to the right a number of pixels down or aside (remember consistent sizing, spacing in between, margins etc!).
- Make the code that creates the new field; give the field a name and maybe allocate a place for that name in a name collection variable.
- Modify the form population code to include this new field.
- Add code that validates this field, so that the user cannot enter an age of 200 or -5.
- Maybe add code that keeps track of the Date of Birth field's contents, to automatically calculate the content of this new field by comparing age and the current time.
- Add a new column for Age in the database table.
- Re-create that table or modify it manually if it happens to already contain nice data for current testing purposes, data that the coder does not want to get rid of yet.
- Modify the code that creates all necessary database tables upon program start, i.e. when the end user runs the system for the first time.
- In fact, do modifications to "a thousand" places which use the employee profile sheet that the new Age field is a part of.

In particular, the naming of a GUI object in combination with keeping track of its contents may cause headaches in APL (and indeed in other languages as well). APL provides a naming syntax that we know well, for this new field it could be for example (in Dyalog APL):

```
MainForm.EmployeeForm.SubForm1.EditField_Age
```

But if the application allows several instances of `EmployeeForm` to be open simultaneously, we can no longer refer to the Edit field easily; we would have to use something like

```
:With MainForm.EmployeeForm_instancenumber  
    SubForm1.EditField_Age  
:End
```

or

```
MainForm.(variable_form_reference).SubForm1.EditField_Age
```

or

```
(reference_to_the_first_three_forms).EditField_Age
```

All of above feel quite impractical to use when a GUI object is referred to numerous times. `:With` and `:End` are highly practical but only for a limited number of "flat" cases. Variable form references are very useful... but one never really knows where all the GUI objects that make a full data definition actually reside. There may be `Splitter` objects that force the use of additional `SubForms`, a property sheet holds pages that themselves are `SubForms`, a `SubForm` may contain additional `SubForms` for the purpose of grouping or fixing GUI element locations and eventually there may be `Groups` or other objects that all affect the object hierarchy. The full path to a GUI object like an Edit field is likely to change during code development and in fact my opinion is that APL code should never be based on using direct names to GUI elements.

In fact, I think we really would not like to bother ourselves with details such as "What is the name of that Edit field?" or "What caused its correct positioning?" or "When do I need to read its Text property?".

Instead, I would like to say, assuming I am to create a property sheet for editing the parameters of an "Employee" object in my application:

1. Create a property sheet with tabs Identification, Address and Duty.
2. On the first tab, put fields for Code, Name, Date of Birth, Age, Id Card Number and Comments. On the second tab, put fields for Mail Address, Street Address and Phone Number. On the third tab, put fields for Position, Supervisor, Salary.
3. Ensure that the contents of the data fields is updated and easily accessible at any time.

4. Make it easy for me to maintain a link between the database and the GUI.
5. Make it look good!

The proposed solution starts with defining a structure. Everything else, like form creation, database creation, data retrieval is based on this structure. Using the example above, our structure would look like this:

```
[Code][Name][Date of Birth][Age][Id Card Number][Comments][Mail Address][Street  
Address][Phone Number][Position][Supervisor][Salary]
```

As Dyalog APL uses namespaces, what could be more convenient than using a namespace that holds the data for an Employee object at any time? We could call the namespace `#.EMP`. At any time, no matter if we work in a property sheet or just use the data for other processes, we could have variables by keys, like

<code>#.EMP.Code</code>	(always contains the short-code of the employee)
<code>#.EMP.Name</code>	(always contains the name of the employee)
<code>#.EMP.DateOfBirth</code>	(contains the IDN value...)

That is, instead of referring to the Text property of the Edit field for "Name", we create APL code that updates the value of the `#.EMP.Name` variable whenever the user changes the corresponding field. This technique gives us a number of advantages:

1. The form creation functions can use the structure keys to automatically create appropriate data entry controls.
2. There is no need to know where the Edit field for a key actually resides, or what its name is. Callback code behind each control ensures that the control value is reflected into the namespace variables.
3. If we have a "default values" vector for a structure, we can fill the form with default values in the case of a "new employee".
4. As we have default values for each key, we can determine when a GUI control holds a non-default value.
5. The structure can be used to create the corresponding table in the database; the keys are columns and the records are rows.

Defining the structure

A sample structure definition function:

```
[0] Set_EMP;a
[1]
[2] n IDENTIFICATION
[3] a,+'Code' '' 'adVarChar'
[4] a,+'Name' '' 'adVarChar'
[5] a,+'DateOfBirth' 0 'adInteger'
[6] a,+'Age' 0 'adInteger'
[7] a,+'IdCardNumber' 0 'adInteger'
[8] a,+'Comments' '' 'adVarChar'
[9]
[10] n ADDRESS
[11] a,+'MailAddress' '' 'adVarChar'
[12] a,+'StreetAddress' '' 'adVarChar'
[13] a,+'PhoneNumber' '' 'adVarChar'
[14]
[15] n DUTY
[16] a,+'Position' '' 'adVarChar'
[17] a,+'Supervisor' '' 'adVarChar'
[18] a,+'Salary' 0 'adDouble'
[19]
[20] a+(phi3,(pa)/3)pa
[21] :With '#.EMP'[]NS''
[22] h def ado+{1}a
[23] Get+{1}w
[24] a+[]AV[4]
[25] []FX'vR+All',a,'[1] R+1'h',a,'v'
[26] Set+{a+h o 1(v a),'+w'}
[27] Set def
[28] #.S_EMP+0+,[0.1]def
[29] :End
```

The function does the following:

1. Creates a namespace called `#.EMP`.
2. Creates a vector `h` of key names.
3. Creates a vector `def` of default values.
4. Creates a vector `ado` of datatypes for the ADO interface to the database engine.
5. Creates a dynamic function `#.EMP.Get` which can be used to retrieve values if the key argument is a text vector.
6. Creates a function `#.EMP.All` which returns a vector of all values for the data structure.

7. Creates a dynamic function `#.EMP.Set` which can be used to set values of the data structure if the key argument is a text vector.

At row 27 the structure elements are set with default values for the purpose of program code being able to immediately refer to any element of the structure without failing with a VALUE ERROR.

At row 28 the code creates a global variable, a matrix with zero rows, to hold instances of the structure. There are situations, where more than one instance of the structure will be present in the workspace simultaneously and those will exist as rows in `#.S_EMP` in a controlled manner.

The contents of the three control variables:

h	def	ado
Code	[empty text]	adVarChar
Name	[empty text]	adVarChar
DateOfBirth	0	adInteger
Age	0	adInteger
IdCardNumber	0	adInteger
Comments	[empty text]	adVarChar
MailAddress	[empty text]	adVarChar
StreetAddress	[empty text]	adVarChar
PhoneNumber	[empty text]	adVarChar
Position	[empty text]	adVarChar
Supervisor	[empty text]	adVarChar
Salary	0	adDouble

Also notice that the default values basically define the APL datatypes (numeric, text, vector, matrix etc.). For example, an Edit field supports several *FieldTypes* which affect the name of the property to set and the value to pass to that property. Even though the form maintenance code explicitly sets the GUI control types, the values in *def* must match these types.

Creating the database

The applications using this technique interface to the MS Jet database engine (the MS Access format) through the Windows ADO interface.

It is fairly straightforward to create the database. Essentially, the only things we need are the column keys (*h*) and the column datatypes (*ado*). Being however outside the scope of this story, we won't discuss the ADO interface further.

Creating the GUI elements

The GUI creation is the true place for automation. I use a single function to create to propertysheet forms, the calling syntax is for example:

```
[1] txt←'Identification' 'Address' 'Duty'
[2] #._WinHL.PropForm '#._F1' 100 'EMP' '' 0 txt
```

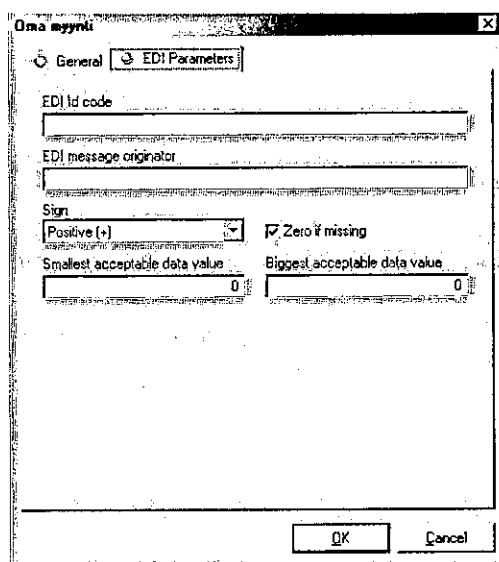
This will create a propertysheet with three tabs on it; "Identification", "Address" and "Duty". Additionally, the "OK" and "Cancel" buttons will be in place, and the form look and behaviour in general is set to one which is common throughout the application. The arguments to the function are a pointer to a namespace that contains the second part of the form creation functions plus callback handlers (if any) and form population function (if any), the name of the structure to use, and additionally some control parameters.

#.SET does the real job. Together with its callback handler *#.Hand_SET*, it does all the essentials. These sample rows (taken out of an other context) produces a tab page below:

```
[4] #.SET('^EDI Id code' '*Edit' '°EDICode')
[5] #.SET('^EDI message originator' '*Edit' '°Notifier')
[6] #.SET('^Sign' '*Combo' '°Sign' '|list_SIGN')
    ('_Zero if missing' '*Check' '°ZeroFill')
[7] #.SET('^Smallest acceptable data value' '*Edit' '°LowerLimit')
    ('^Biggest acceptable data value' '*Edit' '°UpperLimit')
```

Important features of *#.SET*:

1. Gives a common look and feel on all GUI elements, on all tab pages and throughout the application, even throughout a family of applications.
2. GUI objects are automatically placed at appropriate positions. *#.SET* maintains a "vertical pixel counter" and locates several objects on the same row if so desired.



3. Data entry controls automatically reflect the current contents of the corresponding structure upon creation (data-awareness).

4. Changes in the data entry fields automatically get reflected into the structure.

5. Trapping of non-default values. This can be seen at the tab page headers; the "lamp on" means the tab field contains a non-default value somewhere, this visible page only holds default values.

6. Supports the most common GUI controls.

7. Re-used code.

8. Uses global settings for metrics; to change the spacing between controls throughout the application, one only needs to set a single value.

#.SET uses identification characters in its arguments to determine the meaning of each single argument. For example:

- ^ The label above the GUI control
- x The field type
- o The name of the structure key
- l The name of the *Items* vector for a Combo

Each data entry control gets a callback command attached to it. That command is constructed to update the corresponding structure element. The philosophy here is essentially to avoid calling program code where possible, and instead let the GUI objects "take care of themselves" by giving them enough information in advance and, if required, attach data to them. For example, we could let the important objects respond to message number 1200 just to get them to acknowledge themselves.

Using the structure without GUI

Commonly, the coder faces a situation where he originally did all the code with the assumption that there is a GUI form available. Then someone sets a

requirement that the program shall run in "batch" or "server" mode without any visible GUI or without any user present to respond to messages etc.

Similarly, another part of the program may require the data from the definitions set in this first part. The use of structures makes it very easy to code the rest of the application. It does not matter whether the Employee form is visible or not:

- If it's the "thing" currently active, `#.EMP` contains data for it.
- If another part defines the source data for the Employee, we simply set `#.EMP` elements to contain data retrieved for example from the database accordingly.

In both cases, the programmer may use convenient references as `#.EMP.Code`, `#.EMP.Name` etc. to get the essential employee data.

If other program parts use several references to employees simultaneously, for example if the user has two instances of a salary payment editing form opened, we may fill rows of a global variable with employee structures; one row for each invoice. At any time, `#.EMP` contains data for the currently active one, where "active" is defined either by user interface states like input focus, or simply by execution stages in code.

Advantages with structures

Using a well defined structure as the core for object definition, regardless of whether it is a question of .INI-file keys and values or Employee or Invoice or similar objects, gives us a high degree of clarity, maintainability and ease of coding.

- We create the database on the basis of the structure.
- We get input form automation with common look and feel.
- At any time, we can retrieve the value of a structure element, even during code development from the APL explorer.
- We can describe practically anything using a structure, which is very easy to document and comment in code.
- We can easily "slot in" and "slot out" structures (records in a database table, rows in a two-dimensional matrix) with minimum effort.
- We can take advantage of the Dyalog APL dot-separated syntax. For example, `#.EMP.(Code Name Age)` returns a vector with three elements.

Tomas Gustafsson 23.5.2001

Can Anyone Tell Me Where I'm Wrong?

Sylvia Camacho (100612,1057@compuserve.com or acam@tesco.net)

Prologue

Stefano, our Editor, told me of one place where I was wrong, even before going to press, so I am very happy with the effect of my Title although shamed by the mistake in simple arithmetic which he spotted.

Bell's Inequality

Oldtimers will wonder why I am revisiting this topic which I wrote about way back in Vector 4.1, a text somewhat flawed because at the end an advertisement was printed over the table created by the code. However I said in that paper that I would continue puzzling and so I have, intermittently, over the last 14 years and now at least I know what I do not understand.

In 1964 John Stewart Bell described a thought experiment which, if a real experiment could be based upon it, he believed would test the correctness of Einstein's contention that orthodox Quantum Theory implies non-local action at superluminal speeds. At some time before 1985 at least two such tests were made by John Clauser and Alain Aspect and apparently supported the orthodox view against Einstein and in accordance with the interpretation attributed to Niels Bohr. I have an account of Bell's proposal in "Quantum Reality" by Nick Herbert (1985) (which is the popularising account without heavy mathematics which I reviewed in 4.1) and another in a paper "Conceptual Foundations of Quantum Mechanics" by Abner Shimony with mathematics that leaves me puzzled. I have no formal training in either physics or mathematics although my interest in both was aroused many years ago by the philosopher Karl Popper. I do know, however, that there are many Vector readers who do understand both the physics and the maths and I am hoping that some of you will be interested to show me what it is that I have misunderstood.

Einstein thought that he could demonstrate a limitation of Niels Bohr's 'Copenhagen' interpretation of quantum physics. He proposed that particles prepared in an 'entangled' state allow a simultaneous measurement of position and momentum because the position of one of the pair of entangled particles can be measured at A while the momentum of its partner is measured at B. This is the 'thought experiment' which is called EPR after Einstein, Podolsky and Rosen who devised it. This experiment being very difficult to realize, David Bohm devised an

equivalent based upon polarisation correlated photons. Photon polarisation is measured with a calcite crystal which splits the beam into up and down channels depending on whether the photon is polarised along or across the calcite's optic axis. The experiment assumes that the entangled beam of photons is completely unpolarized which means that measurement at any calcite angle results in 50% leaving by the up channel and 50% by the down channel in apparently random sequence. It is assumed that a pair of calcites can be set up such that they have the same orientation to each other in the X Y plane and that either can be rotated in this plane to establish a known angle between them. Further assumptions are that two beams of entangled photons can be generated and each made to impinge perpendicularly upon the X Y plane of one of the calcites. It is further assumed that measuring equipment can be associated with each calcite to record the channel (UP or DOWN) by which each photon leaves the calcite. A test for quantum correlation is based upon the MATCH to MISMATCH statistics of the two photon streams for different relative angles between the two calcites. According to the Copenhagen interpretation the measured correlation supports the view that there must be interaction between the twin photons from start to finish of the two measurement events, no matter how distant in space and time these are.

It is well established that if both calcites are set to the same angle the UPs and DOWNS of the two beams of photons will always match. It is also agreed that if the second calcite is set at an angle 90 degrees displaced from the first the two beams will exactly mismatch. For every UP at stream A there will be a DOWN at stream B as though the up and down channels had been reversed. It is also agreed that if the difference is set at 45 degrees there will be a mismatch in 50% of the photon pairs, some will read UU some will read DD but 50% will read either UD or DU as though each twin's path between up and down channels was equiprobable. If only these three angles, 0 45 and 90 were known we might assume a smooth transition from perfect correlation at 0 degrees to perfect anti-correlation at 90 degrees – a nice straight line. This is not what we get and the discrepancy is said to prove that there is a superluminal connection between the photons until one of them has been measured (the so-called 'collapse of the wave function'). I find it difficult to formulate any arguments to make good this claim other than those which beg the question by assuming the Quantum Theory formalism which they claim to be putting to the test.

Up to the equilibrium angle of 45 degrees the correlation lessens by $\cos^2\theta$ where θ is the angle between the calcites. At 45 degrees there is no excess of MATCH over MISMATCH. Beyond 45 degrees the anti-correlation increases by $1-\cos^2\theta$ until there is complete mismatch at 90 degrees. I note that the finding for the very similar classical experiments with light beams and polaroid filters is that the ratio

of the intensity of the light passed through two polarisers is $\cos^2\theta$ where θ is the angle between the axes of the polarizers. Now $\cos^2 30$ is 0.75 and $\cos^2 60$ is 0.25 which Herbert in his *Quantum Reality* shows as the correlations obtained in the Bell type experiments. It is agreed that the entangled beams are completely unpolarised which means they will measure 50% up and 50% down at any angle. So as measured at calcite A its sequence of UP and DOWN marks will constitute the only standard by which the match/mismatch ratio is established when compared with the results at calcite B. The percentage mismatch will be $\cos^2\theta$ whatever rearrangement of the calcites is made before the first measurement is taken or however the second calcite is subsequently rotated. The two beams are identical until they pass through the calcites. Each photon has, in Popper's terms, a set of *propensities* (or 'superposition of probability amplitudes', as the physicists call it) to interact with a calcite such that it will be deflected through the UP or DOWN channel. The same applies to its twin in the other beam. The propensities are realized as each is measured. Each outcome is contingent upon the interaction between the photon and its calcite (the 'entire experimental arrangement', as Bohr said). After a sufficiently large set of paired measurements have been made we can deduce, from the MATCH/MISMATCH statistics, the relative angle between the calcites although we cannot learn anything about the photons constituting the entangled beams. Wherein here lies the evidence for non-local action?

What, if any, correlation should we expect from purely local action? That is the assumption that there is no interaction between the photons in a pair after they have parted company. We should allow for the fact that we know there will be an orientation of the two calcites that will yield a perfect correlation and that an orientation of one of them at 90 degrees from this will yield a perfect anti-correlation. Herbert's popularising book explains that it is only the relative angle between the calcites which is significant. I take this to mean that after the initial set up either calcite may be rotated through any angle positive or negative (clockwise or anti-clockwise). His demonstration of Bell's inequality is very simple. Starting from the perfect match position rotate calcite A through an angle (say 30 degrees) and make a set of measurements. This yields a correlation of 75%:25% MATCH/MISMATCH of UP/DOWN readings. Reset to the perfect correlation orientation. Then rotate calcite B through an angle of minus 30 degrees and repeat the measurement run. This yields the same correlation of 75%:25% MATCH/MISMATCH of UP/DOWN readings. So, Herbert argues, that if an angle of 30 degrees yields a 25% MISMATCH and an angle of minus 30 degrees also yields 25% MISMATCH, an angle of twice this (i.e. 60 degrees) should yield a MISMATCH of not more than 50%. When the experiment is made, however, it will be found that setting calcite A to plus 30 degrees and calcite B to minus 30

degrees yields a MISMATCH of 75%. This difference between 50% and 75% is, he says, an example of the Bell inequality.

This leads me to consider a slight variation of Herbert's argument. Suppose we set calcite A to plus 45 degrees and calcite B to minus 45 degrees, what do we expect? The correlation MATCH/MISMATCH at 45 degrees is 50:50. Twice 50 being 100 we should have – what? Twice 45 degrees being 90 degrees we know we shall have 100% MISMATCH. Why not 100% MATCH? Because only the relative angle is significant. If we rotated both calcites to plus 45 degrees we know we should get a perfect correlation but we could equally well describe this as 'resetting our zero orientation'. To summarise: given two identical sets of photons we can pass one set through calcite environment A and the other through calcite environment B and compare the results and note that we get a correlation $\cos^2\theta$ dependent upon the relative angle θ between the environments. What is so non-local about that? The problem according to quantum orthodoxy is that a measurement 'collapses the wave function' so that the photon, until then in a state of 'objective indefiniteness', takes on a definite characteristic which closes off possibilities which until then were open to it and by extension to its twin. If by this is meant that the result of a measurement cannot be known until it is made, I have no problem. If it means that making the measurement fixes half of the conditions that will define the relative angle between the calcites, I have no problem. If the measurement of the second photon of the pair demonstrates that its response has been limited by the other's measurement it must be because there is some form of interaction despite the fact that they are moving apart faster than seems reasonable to support a signal between them. My problem is that I do not know what would be evidence for this.

I am puzzled to know why the evidence for non-locality is said to be made stronger if the distance between the calcites is made large and the two sets of measurements are demonstrably not co-incident in time (ignoring the complications of establishing co-incidence in a relative world!) It is suggested that, even while calcite A is performing its measurements, calcite B can be rotated and the voodoo communication will still work its magic to produce $\cos^2\theta$ correlation when the B stream reaches its calcite and the UPs and DOWNs are printed. So, I ask myself, "At what angle is A being measured in the absence of B or during its rotation?" The diagrams of these experiments show lines superimposed on the calcites and marked 'optic axis' but no procedure is suggested for establishing this as a preferred axis and, indeed, the purely relative character of the angle theta is several times emphasised. Nevertheless in his demonstration Herbert suggests taking initial measurements using this optic axis as the standard for establishing the correlations to be expected at plus and minus 30 degrees (this optic axis thus standing for zero degrees). We know, however

that this 'optic axis' does not have any favoured status because we are told that if calcites A and B are both set to plus 30 degrees the correlation will be a perfect match. So we know that there is no equivalence between zero and plus 30 and zero and minus 30 – if there were this would yield a perfect match instead of giving, as it does, the value for 60 degrees. The only reasonable conclusion is that the first photon stream to be measured establishes zero degrees (for the purpose of this experiment only) against which the measurements at the second calcite will be correlated and that it is as meaningless as one-handed clapping to refer to either calcite as having an angle in isolation. What is being measured is two different aspects of the photon's waveform and this does not have any preferred orientation, the photon streams being completely unpolarized. So the measurements can be made – just as Einstein said they could be. There, I have delivered myself of the ultimate blasphemy.

So now I come to the paper by Abner Shimony and his definition of Bell's Inequality. He says, "We shall now give in a few steps an outline of a simple derivation of Bell's Inequality invented by J. Clauser and M. Horne. First, it is a fact of elementary algebra that any four real numbers r', r, s', s lying between 0 and 1 satisfy the following inequalities:

$$-1 \leq r's' + r's + rs' - rs - r' - s' \leq 0$$

.....The quantum mechanical probabilities for + outcomes in an X-test on 1 and in a Y-test on 2 (i.e. of passage through polarisation analysers oriented in the X and Y directions, respectively) are easily calculated to be

$$p^1_\Psi(X) = p^2_\Psi(Y) = \frac{1}{2}$$

and the quantum mechanical probability for joint passage through the two polarisation analysers is

$$p_\Psi(X, Y) = \frac{1}{2} \cos^2(Y - X)$$

where Y and X are taken to be the angles which the optical axes of the two polarisation analysers make with a fixed axis in the plane orthogonal to the propagation direction of the photons. If the values x', x, y', y are, respectively, chosen to be $45^\circ, 0^\circ, 22.5^\circ, 67.5^\circ$, then

$$p_\Psi(x'y') + p_\Psi(x'y) + p_\Psi(xy') - p_\Psi(xy) - p^1_\Psi(x') - p^2_\Psi(y') = 0.207$$

which conflicts with the upper limit in Bell's Inequality."

Having worked out the values for the example above I get:

$$0.427 + 0.427 + 0.427 - 0.073 - 0.5 - 0.5 = 0.208$$

which demonstrates that I know how Shimony gets to this result. Any value greater than zero, of course, violates the upper limit of Bell's Inequality. This exposition implies that 0.427 is equivalent to the product $r's'$ in the 'elementary algebraic' expression although it is not a product but the squared cosine of the difference between two angles. However, if r' and s' were each a probability expressed as a number between 0 and 1, the product $r's'$ would express their joint probability and it is presumably this analogy that is intended. As quoted $r's'$ is $\frac{1}{2}\cos^2 22.5$ degrees and Shimony's choice of angles is such that $r's$ and rs' are also $\frac{1}{2}\cos^2 22.5$ degrees. The equivalent for the product rs , on the other hand, is $\frac{1}{2}\cos^2 67.5$ degrees which gives a value of 0.073. So we are asking whether the sum of the probabilities for 3 contiguous angles of 22.5 degrees is equal to the value of the probability for an angle of 67.5 degrees. This seems to be the same as asking if $3 \times \frac{1}{2} \cos^2 \theta = \frac{1}{2} \cos^2 3 \times \theta$. As we know this equation cannot be valid, the Bell inspired experiments seem like a difficult to realise and probably expensive, physical demonstration of a mathematical inequality.

I note that Shimony defines X and Y as the angles which 'the optical axes of the two polarisation analysers make with a fixed axis in the plane'. This implies that the fixed axis has a function and it is not only, as Nick Herbert insists, the relative angle that is significant. The problem that I face is that the Bell formula assumes 4 values, corresponding to 4 angles, and tests some relationship between them. It is not clear to me what relationship is being tested and therefore what bearing this might have on the question of the independence of the results of the measurements which are made at calcites A and B; which concerns only one, not four, angles. This single angle is only established when both measurements have been made and we presumably rule out any superluminal influence after the second measurement. A superluminal influence following upon the first measurement could consist of one of only two possibilities, to leave the twin photon unchanged or to increase its propensity to switch exit channel but this would have to happen before the conditions for such a switch (the chosen angle) could have been encountered. I know, of course, that the quantum orthodoxy is that the photon does not have a polarity until it is measured but I see this as a logical consequence of polarity being defined in terms of orientation and therefore meaningless outside this angular context. Either measurement will necessarily establish this context for the other set of results but the subsequent correlation is the consequence of a logical not a physical connection. The orthodox view that the photon does not have a polarity until it is measured remains true because orientation is part of the definition of polarity. It seems to me that the propensity

of a photon to exit a crystal by one of two possible paths is likely to depend upon the internal calcite angles combined with the wave characteristics of the photon and there seems no reason to suppose this relationship will be linear, for we might expect that the more different the calcite environments the more likely it is that the photon path will be switched. This indeed is what is found. Dear Reader please tell me where I am going wrong.

The very simple code below generates the values for the graph which would be obtained by generating 90 streams of 1000 entangled photon pairs and passing them through calcites opposed at angles from zero through 89 degrees. The variable called 'mark' is a table of the propensities for a given photon to switch output channel in response to a changed environment, the change being expressed as an angle and the probability of the switch being the squared cosine of the angle.

```

v r+run pop;[]io
[1]  a produce values for graph: smc 1 Sep 2001
[2]  a pop is number pairs entangled photons in
[3]  a each stream and r is 90 values showing the
[4]  a fraction of pop where the pairs mismatch
[5]  a at each setting of the calcites' angle
[6]  a between zero and 90 degrees
[7]  []io+0
[8]  cossqtheta a create global vector 'mismatch'
[9]  setmarks pop a create global boolean matrix 'mark'
[10] r+simulate pop
v

v cossqtheta;[]io;radians
[1]  a create global 'mismatch': smc 1 Sep 2001
[2]  a uses global value for radian 0.0174532925
[3]  []io+0
[4]  radians+radian*190
[5]  mismatch+1,('f6.4' []fmt(2oradians)*2),' '
v

v r+entangle n;[]io
[1]  a return random boolean vector: smc 1 Sep 2001
[2]  a emulates entangled UP/DOWN photon stream
[3]  []io+0
[4]  r+?np2
v

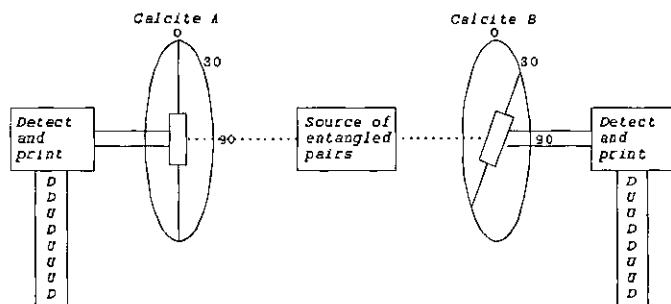
```

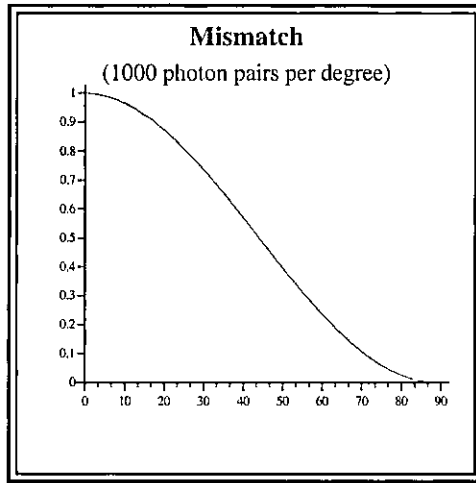
```

v setmarks pop;[]io;c;cols;flips;rl
[1]  A propensity to switch exit channel: smc 31 Aug 2001
[2]  A 'mismatch' vec is mismatch fraction at each degree interval
[3]  A pop is number of photons in this entangled stream
[4]  A global 'mark' is a propensity table for this size pop
[5]  A holding the random link constant simulates propensity as
[6]  A opposed to chance: once flipped an individual photon would
[7]  A not revert at increasing angles. The population statistics
[8]  A are not affected either way.
[9]  []io+0 o mark+(90, pop)p0 o rl+[]rl
[10] flips+=mismatch*pop A number of flips for each degree
[11] c+0 A at zero degrees is no mismatch
[12] L1:=(90=c+c+1)+end
[13] []rl+rl
[14] cols=flips[c]?pop A these photons will flip channel
[15] mark[c;cols]+1
[16] ->L1
[17] end:mark[89;]+1 A at 90 degrees is certain mismatch
v

v r=simulate pop;[]io;c;photons
[1]  A r is match/mismatch correlations: smc 31 Aug 2001
[2]  A r is vector for 90 entangled streams of pop photons
[3]  []io+0 o c+0 o r+pop
[4]  l1:=(90=c+c+1)+end
[5]  photons+entangle pop
[6]  A flip photons in the stream for degree c held in 'mark'
[7]  A then catenate total matches to r
[8]  r+r,+/photons=2\mark[c;]+photons
[9]  ->l1
[10] end:r+1,('f6.4' []fmt r+pop), ' '
v

```





References

Feynman, Richard P et al 'Lectures on Physics' Volume 1, 33-4 'Polarizers'
Addison-Wesley 1963 ISBN 0 201 02116 1 P

Herbert, Nick 'Quantum Reality' Rider & Co. 1985 ISBN 0 7126 1083 9

Popper, Karl R 'A World of Propensities' Thoemmes 1990 ISBN 1 85506 000 0

Shimony, Abner 'Conceptual foundations of quantum mechanics'
ed Davies, Paul 'The New Physics' Cambridge U.P. 1989 ISBN 0 521 30420 2

<http://www.roxanne.org/epr>

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hacker's Corner: Formatting Dates in Dyalog APL and APL+Win	Adrian Smith	97
Technical Correspondence		104
Persistence and Shadows	Stephen Taylor	110
MUCIPHER: a New Short Cipher that Escapes Cryptanalysis	Michel Dumontier	117
Function Models	Phil Chastney	132

Gene McDonnell and *At Play with J* will be back in the next issue of Vector.

**Download
from the
web for
free!**

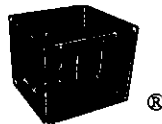
Announcing JsoftwareTM Version 4.06

- ♦ New and Improved Documentation
- ♦ New Data Types, Symbol and Unicode
- ♦ DLL Callbacks Supported
- ♦ New Language Features
- ♦ Improved Locales
- ♦ And more!

This release will make you more productive. We've listened to our users requests, and implemented many technical changes and improvements. We also added new features, for example, significant new data types.

Make sure to watch our web site for the new release. Professional Edition subscribers will receive their copies in the mail after the edition is released on the web.

Jsoftware Inc.
19235 Covington Ct.
Shorewood, MN 55331
Info@jsoftware.com
Tel (952) 470-7345
Fax (952) 470-9202



www.jsoftware.com

Hacker's Corner: Formatting Dates in Dyalog APL and APL+Win

experiments by Ray and Stefano, notes by Adrian Smith

Background

I began to think about this the first time I saw John Daintree's presentation on Dyalog.NET. He showed us how much easier it is to get at the Windows API calls, and one of his examples played some games with the date class which .NET provides. As well as letting you add and subtract dates, this has a basic 'picture formatting' capability so that you can throw it (more or less) $\square TS$ and get back 'Sunday, October 21, 2001' or any other reasonable representation. There it rested, until one of my GraPL users started kicking me for a 'date,time' capability on the X-axis. *RainPro* has always had 'date' and 'time' X-styles and it was easy enough to accept the combination, but I began to think that my habit of simply grabbing the 'short date' format from Windows would not be acceptable for much longer.

A quick dig around in Excel confirmed that you could set all sorts of formats with simple picture specifications such as 'ddd, mmmm dd, yyyy' and that (very important, this) the date would be formatted in your local language. This smelled like an API call (remember that .NET is mostly just a nice wrapper on the current API), so off to the Windows help file and hit the 'full text search' on formatting dates. Bingo ... what should we find but **GetDateFormat** (now there is a misleading name for you) which is documented as follows:

The **GetDateFormat** function formats a date as a date string for a specified locale. The function formats either a specified date or the local system date.

```
int GetDateFormat(  
    LCID Locale, // locale for which date is to be formatted  
    DWORD dwFlags, // flags specifying function options  
    CONST SYSTEMTIME *lpDate, // date to be formatted  
    LPCTSTR lpFormat, // date format string  
    LPTSTR lpDateStr, // buffer for storing formatted string  
    int cchDate // size of buffer  
);
```

There is a matching `GetTimeFormat`, which does something very similar to the time part of the date/time vector. Time to

Call in the Professionals

I knew that Ray Cannon had an extensive library of date functions, and guessed that Stef might well have done something similar, so out went a call on the internet, and sure enough, back came the code. Ray used 'conventional' Dyalog APL, but Stef had much more fun and wrote a splendid collection of 'D' to cover both calls in one go. The rest of this note explains how the calls work, and also has the equivalent implementation in APL+Win.

Dyalog covers for simple Date and Time formatting (current time)

First the simple case – format the current date with a given picture specification:

```
r←GetDateFormat;get;fmt
a Return the current system date

fmt←'dddd','' dd MMMM yyyy'
'get'⎕NA'I2 kernel32.C32|GetDateFormatA I4 I4 I4 <0T >0T I2'

r←→1/get 0 0 0 fmt 255 255
```

The function returns the length of the result and the text as a two-element vector, hence the rather interesting reduction, followed by a 'pick' (we are in `⎕ml=0` here) to get back a simple string.

The 'picture' format is quite complex, and is well explained in the Windows help:

Use the following elements to construct a format picture string. If you use spaces to separate the elements in the format string, these spaces will appear in the same location in the output string. The letters must be in uppercase or lowercase as shown in the table (for example, "MM" not "mm"). Characters in the format string that are enclosed in single quotation marks will appear in the same location and unchanged in the output string.

Picture	Meaning
d	Day of month as digits (no leading zero for single-digit days).
dd	Day of month as digits with leading zero for single-digit days.
ddd	Day of week as a three-letter abbreviation. The function uses the <code>LOCALE_SABBREVDAYNAME</code> value associated with the specified locale.

dddd	Day of week as its full name. The function uses the LOCALE_SDAYNAME value associated with the specified locale.
M	Month as digits with no leading zero for single-digit months.
MM	Month as digits with leading zero for single-digit months.
MMM	Month as a three-letter abbreviation. The function uses the LOCALE_SABBREVMONTHNAME value associated with the specified locale.
MMMM	Month as its full name. The function uses the LOCALE_SMONTHNAME value associated with the specified locale.
y	Year as last two digits, but with no leading zero for years less than 10.
yy	Year as last two digits, but with leading zero for years less than 10.
yyyy	Year represented by full four digits.
gg	Period/era string. The function uses the CAL_SERASTRING value associated with the specified locale. This element is ignored if the date to be formatted does not have an associated era or period string.

This allows you plenty of scope, but a little surprisingly, formats like 12th October are not supported here. The similar call for the formatted 'time' is:

```

r←GetTimeFormat;get;fmt;str
  ⚭Return the current system time

fmt←'HH':'mm':'ss tt'      ⚭ See Picture meaning

'get'⚭NA'I kernel32.C32\GetTimeFormatA I4 I4 I4 <0T >0T I'
r←→t/get 0 0 0 fmt 256 255

```

and the 'pictures' are rather easier to remember here:

Picture	Meaning
h	Hours with no leading zero for single-digit hours; 12-hour clock
hh	Hours with leading zero for single-digit hours; 12-hour clock
H	Hours with no leading zero for single-digit hours; 24-hour clock
HH	Hours with leading zero for single-digit hours; 24-hour clock
m	Minutes with no leading zero for single-digit minutes
mm	Minutes with leading zero for single-digit minutes
s	Seconds with no leading zero for single-digit seconds
ss	Seconds with leading zero for single-digit seconds
t	One character time marker string, such as a or p
tt	Multicharacter time marker string, such as am or pm

Just to prove they work

```

    GetDateFormat
Sunday, 21 October 2001
    GetTimeFormat
20:49:32 pm

```

That deals with the simple case – now to pass it the time of our choice, which requires a little more research as we need to adapt `ts` timestamps to fit the Windows model. Over to Stef, and a distinct change of coding style!

Dynamic functions for general-purpose date/time formatting

```

GetDateFormat+(
  n w is a ts time
  n α is a picture description of the format required
  α+'ddd','' MMM dd yy'
  l=>gdf 0 0(TS2SYSTEMTIME w)α 0 0
  2=>gdf 0 0(TS2SYSTEMTIME w)α 1 1
)

```

```

GetTimeFormat+{
  w is a ts time
  a is a picture description of the format required
  a+'hh':'mm':'ss tt'
  l←gtf 0 0(TS2SYSTEMTIME w)a 0 0
  z←gtf 0 0(TS2SYSTEMTIME w)a 1 i
}

TS2SYSTEMTIME+{
  w is ts format
  a returns a Win32 SYSTEMTIME structure
  a is the day of week: not required here
  a+0
  a[w[1 2],a,2+w]7+w
}

Init
'gdf'⌈NA'I kernel32 GetDateFormatA I U <{I2[8]} <0T >0T I'
'gtf'⌈NA'I kernel32 GetTimeFormatA I U <{I2[8]} <0T >0T I'

```

Stef has used the slightly more rigorous approach of calling the routines once, with a zero-length buffer, to get the result size, then allocating a suitable buffer at this length. To show them in action:

```

TS2SYSTEMTIME ts
2001 10 0 21 21 21 20 0
'HH:mm' GetTimeFormat ts
21:21
'hh:mm tt' GetTimeFormat ts
09:21 pm
'''Today is 'dddd, dd MMMM - yyyy' GetDateFormat ts
Today is Sunday, 21 October - 2001

```

You can actually be fairly cavalier about missing out the quotes around inclusions – as long as the characters cannot be taken for formatting cues they will just appear verbatim in the result. Putting it all together, we have:

```

FormatTS+{
  a is the picture format
  w a TS-like time specification
  a Assumes the time placeholders come all AFTER or BEFORE the dates
  a in the provided picture
  ⌈ML ⌈IO+1

  a default a is not a nice format, just a test
  a+'ddd',' MMM dd yy hh':'mm':'ss tt'
  pic←a a we'll need this name later

  gdf+{}
  gtf+{} a trick to localise the following defs
  _+'gdf'⌈NA'I kernel32 GetDateFormatA I U <{I2[8]} <0T >0T I'
  _+'gtf'⌈NA'I kernel32 GetTimeFormatA I U <{I2[8]} <0T >0T I'

```

```

TS2SYSTEMTIME+{
  a+0
  a{w[1 2],a,2+w)7+w
}

aux+{
  w+TS2SYSTEMTIME w
  1+>a a 0 0 w a 0 0 a buffer needed
  2>a a 0 0 w a 1 1 a apply
}

bq+~{wv~1φw)≠\a='''
first+{
  >(v+ bq\+aε''cbq/w)1 a let's use lexical scoping... woah!
}
format+{
  ((>a)gdf aux w)((2>a)gtf aux w)
}

ix+~1+TIMEPICTURES DATEPICTURES first''ca
</ix:εφ(ix[2]+pic)(ix[2]+pic)format w a date after time
ε(ix[1]+pic)(ix[1]+pic)format w a time after date
}

```

The variables *TIMEPICTURES* and *DATEPICTURES* simply list the allowed formats (column-1 of the tables a few pages back). Again, to test it:

```

'''It is now ''hh:mm:ssstt ''on'' ddd, MMM-dd-yy' FormatTS 0ts
It is now 10:33:57pm on Sun, Oct-21-01

```

That just about wraps it up for Dyalog – now to see what APL+Win can do.

Date and Time formatting in APL+Win

First things first – a quick trawl through *aplwadf.ini* revealed that neither of the two functions is known to *WCALL*, so a quick email to APL2000 support, and the following was swiftly provided by John Walker:

```

GetDateFormat=I(D Locale, D dwFlags, *SYSTEMTIME lpDate, *C lpFormat, >C
lpDateStr, I cchDate) ALIAS GetDateFormatA LIB Kernel32

GetTimeFormat=I(D Locale, D dwFlags, *SYSTEMTIME lpTime, *C lpFormat, >C
lpTimeStr, I cchTime) ALIAS GetTimeFormatA LIB Kernel32

```

Having patched these in (and followed the instructions in the file to recompile it), the functions were plain sailing:

```

v st←dw TS2SYSTEMTIME ts
[1] a <ts> is []ts format
[2] a Returns a Win32 SYSTEMTIME structure
[3] a <dw> is the day of week: not necessary here
[4] dw←0 o ts←7+ts o st←ts[1 2],dw,2+ts
v

[]VR'GetTimeFormat'
v fmt←pic GetTimeFormat ts;len
[1] a <ts> is a []ts time
[2] a <pic> is a picture description of the format required
[3] :if 0=[]NC 'pic'
[4]   pic←'h':'mmtt'   a 02:34pm is default
[5] :end
[6] a Checks the buffer size, then sends a buffer to hold the answer
[7] len←1+[]WCALL 'GetTimeFormat' 0 0(TS2SYSTEMTIME ts)pic 0 0
[8] fmt←2+[]WCALL 'GetTimeFormat' 0 0(TS2SYSTEMTIME ts)pic (lenp' ') len
v

[]VR'GetDateFormat'
v fmd←pic GetDateFormat ts;len
[1] a <ts> is a []ts time
[2] a <pic> is a picture description of the format required
[3] :if 0=[]NC 'pic'
[4]   pic←'ddd','' MMM dd yy'
[5] :end
[6] len←1+[]WCALL 'GetDateFormat' 0 0(TS2SYSTEMTIME ts)pic 0 0
[7] fmd←2+[]WCALL 'GetDateFormat' 0 0(TS2SYSTEMTIME ts)pic (lenp' ') len
v

```

A couple more examples, to show these in action:

```

'MMM/yy,ddd' GetDateFormat []ts   a Custom date format
October/01,Sunday
'HH:mm-ss' GetTimeFormat []ts     a Custom time
22:46-49

```

As you can see, *Vector* is already exactly a month late, and the evening is moving on. Time to slump in front of the television and watch the football!

Summary

I think these are a really useful discovery. The great benefit from my perspective is that I can offer my GraPL users 'TimePicture' and 'DatePicture' properties and any prior knowledge they have (of Excel, for example) can be immediately applied. Probably I should document these in my own help file, but all I really have to do is crib those two tables again, courtesy of Microsoft. Let's hope .NET fulfils its promise of making many more of these useful utilities easy to find and apply in our own APL and J applications.

TECHNICAL CORRESPONDENCE

Conditional Idioms

From: Phil Last (phil.last@ntlworld.com)

May 2001

Most of you will have seen something similar to:

```
[ 23 ] →DOIF INC>0  ◊  TALLY+TALLY+1
[ 24 ] →DOIF INC<0  ◊  TALLY+TALLY-1
```

where (DOIF) is something like

```
[ 0 ] R←DOIF B
[ 1 ] R←1+(0∈B)⊖1+□LC
```

Later you might have encountered

```
[ 47 ] :If INC>0  ◊  TALLY+TALLY+1
[ 48 ] :ElseIf INC<0  ◊  TALLY+TALLY-1
[ 49 ] :End
```

and if you had your wits about you, you would have laughed knowingly and absolutely KNOWN that, had you written the original code it would have said

```
TALLY+TALLY+*INC
```

Well done! here are a few more examples. I don't claim that any of these are original insights. I've had the best part of the list knocking about for years, seldom refer to it and then more often to add one than remember one I've used before.

Just thought someone else might be mildly interested.

These are not functions, they are conditional expressions that either perform an operation or not or they perform one operation or another. Wherein:

(c) is a simple logical scalar which decides the issue.

(a) & (w) are the two or (a) is the single primary input(s) to the expression.

(z) is a temporary variable.

(F) is a dyadic function.

If (c) is zero and there is no alternative (else) in the description then the first of (a) & (w) mentioned is returned unaltered other than by scalar extension.

ISO standard APL is used throughout.

- | | |
|--|--|
| 1. Multiply (a) by (w) | $a \times w \times c$ |
| 2. Reciprocate (w) | $w \times^{-1} \times c$ |
| 3. Divide (a) by (w) | $a \times w \times^{-} c$ |
| 4. Divide (a) by (w) else multiply | $a \times w \times^{-1} \times c$ |
| 5. Negate (minus) (w) | $w \times^{-1} \times c$ |
| 6. Sum of (a) and (w) else difference | $a - w \times^{-1} \times c$ |
| 7. Reverse (w) (rank 1) | $w[\psi c \times 1 \rho w]$ |
| 8. Reverse (w) (first dim) | $(\rho w) \rho \Theta (c+1, \rho w) \rho w$ |
| 9. F-Reduction (w) (first dim) | $F \neq (c+1, \rho w) \rho w$ |
| 10. F-Reduction (w) (last dim) | $F / ((\rho w), c+1) \rho w$ |
| 11. Transpose (w) | $(\psi c \times 1 \rho \rho w) \Theta w$ |
| 12. Transpose (w) by (a) (non-slicing) | $(\Delta c \times a) \Theta w$ |
| 13. Ceiling (w) else floor | $z \times \lfloor w \times z +^{-1} \times c$ |
| 14. Maximise (a) with (w) else minimise | $z \times (a \times z) \lfloor w \times z +^{-1} \times c$ |
| 15. And (a) with (w) else or | $c = (a = c) \wedge w = c$ |
| 16. Sum (+w) else last (1 + w) $\rho \Theta w$ | $c \perp w$ |
| 17. Ravel (w) | $(\times \neq (c+1, \rho \rho w) \rho \rho w) \rho w$ |

Having put the list in this form under that intro there didn't seem to be as many as I'd thought so I tried to search out a few more, added five and came to the conclusion that all but one of the new ones were just variations on a theme. I think all those that remain deserve to be there in that each either has something which is very specific that couldn't be used elsewhere or is a typical example of a more generally applicable technique.

I'd be very pleased to see if anyone else has any further examples showing any new insights or techniques.

Matrix Inversion Modulo p

From: John Sullivan (john@yddraiggoch.demon.co.uk)

October 2001

I read Michel Dumontier's articles *Matrix Inversion Modulo p* and *Inversion of Boolean Matrices* in Vol. 18 No. 1 with great interest. However, something did not seem quite right, and after a bit of thought I realized what it was. M. Dumontier is calculating matrix inverses modulo 256, yet he has resorted to multiprecision arithmetic, for which he has had to switch from APL to J. But, surely, when performing calculations modulo 256 you never need to exceed 65536 (i.e. 256^2) in any one value, for which APL should be quite sufficient.

On further investigation, I noticed that the problem is twofold:

- The inverse matrix is calculated by dividing the adjoint (transpose matrix of the cofactors) by the determinant, which is a very inefficient method.
- Numbers should be reduced modulo p at every opportunity, and not just at the end of the calculation.

The fact that we are working modulo p should not worry us. When we are working in a field, such as $\mathbb{R}$, the usual methods involve division somewhere along the way, and this division can be thought of as multiplication by the multiplicative inverse of the divisor. When working modulo p we also multiply by the multiplicative inverse, but this time modulo p . In all cases if the multiplicative inverse does not exist then the matrix will not have an inverse.

A Better Function

The best way of finding an inverse matrix is by using elementary transformations, which are:

- The interchange of two rows (not used in this algorithm)
- The multiplication of a row by a scalar
- The addition of a scalar multiple of one row to another.

The work is carried out on two matrices, one of which is the matrix to be inverted, and the other is the identity matrix. The idea is to reduce our matrix to the identity matrix by applying these elementary transformations, and to end up with the inverse matrix by applying the same transformations to the identity matrix. This works because each of these elementary transformations on the rows of the matrix is equivalent to premultiplying by a suitable matrix (which we do not need to know), giving $T_1 T_2 \dots T_n A = I$ from which we see that $T_1 T_2 \dots T_n = A^{-1}$.

We therefore have the following function:

```

      z←p matinvmod m;i;j;r;x
[1]  ⚭ Inverse of matrix m modulo p by elementary transformations
[2]  r←1+pm                      ⚭ Rank
[3]  z←(pm)p(r+1)+1              ⚭ Start with identity matrix
[4]  ⚭ Get the matrix into upper triangular form
[5]  :For j :In 1r
[6]    i←j
[7]  ⚭ Find the multiplicative inverse of the jth diagonal element
[8]  ⚭ If the inverse is 0, add another row to this one and try again
[9]    :While 0=x+m[j;j]Inv_mod p
[10]      [SIGNAL(r=i+1)/11      ⚭ There isn't one so no inverse mod p
[11]      x←(m[j;j]+m[i;j])Inv_mod p
[12]      :If 0=x
[13]        m[j;]+m[j;]+m[i;] ⚭ z[j;]+z[j;]+z[i;]
[14]      :EndIf
[15]    :EndWhile
[16]  ⚭ Scalar multiply the jth row of both matrices by x
[17]    m[j;]+p|m[j;]*x ⚭ z[j;]+p|z[j;]*x
[18]  ⚭ Position [j;j] should now be 1. Subtract row j from the other rows
[19]    :For i :In (j+1)+1r
[21]      m[i;]+p|m[i;]-m[j;]*x-m[i;j] ⚭ z[i;]+p|z[i;]-z[j;]*x
[22]    :EndFor
[23]  :EndFor
[24]  ⚭ Now work backwards to clear the upper triangle
[25]  :For j :In ϕ1+1r
[26]    :For i :In 1j
[27]      z[i;]+p|z[i;]-z[j;]*m[i;j]
[28]  ⚭ m[i;]+m[i;]-m[j;]*m[i;j] ⚭ Uncomment for testing
[29]    :EndFor
[30]  :EndFor

```

To calculate the multiplicative inverse of a number modulo p I have used function *Inv_mod*. This function originally appeared in *Vector* Vol. 12 No. 4, p.75, in one of a series of articles I wrote on multiprecision arithmetic in APL. Since we are not dealing with multiprecision here, I have removed the multiprecision path from the program, and I have also rewritten it to use control structures, rather than branches and labels. Note that this function returns the impossible value 0 if the inverse does not exist, and this value can be caught and acted upon in the calling function.

```

      z←x Inv_mod n;a;b;N;rn;r0;r1
[1]  ⚭ Calculate the multiplicative inverse of x mod n
[2]  ⚭ If the inverse does not exist, return 0
[3]  a←0 ⚭ N+n
[4]  z←0 ⚭ x+N|x
[5]  :If x=1
[6]    z←1
[7]  :Else
[8]    :While 0=x

```

```

[9]      a+a,>b*(0,x)rn
[10]     n+x o x+1>b
[11]     :EndWhile
[12] * At this point n holds the GCD of the original n and x.
[13] * If this is not 1 then there is no mult. inverse.
[14]     :If 1≠n
[15]       r0+1 0 o r1+0 1
[16] * Recurrence relation is r(n)=r(n-2)-q.r(n-1)
[17] * i.e.                rn ← r0 - (÷a).r1
[18]       :While 1<pa
[19]         rn+r0-r1>a
[20]         r0+r1 o r1+rn o a+1+a
[21]       :EndWhile
[22]       z+N|1>rn
[23]     :EndIf
[24]   :EndIf

```

The function *matinvmod* will work very efficiently on the rank-10 matrix shown on p.140 of Vol. 18 No. 1 without the necessity for multiprecision arithmetic. I also applied it to matrix *a* on p.139, and I'm afraid to say that I discovered that the matrix *b* shown there is not the inverse of *a*. (I think this *b* must have slipped in by accident, because the other calculations imply that *b* should be the inverse of *a* as shown below.)

a

33	192	116	135	55	11
173	173	238	97	132	211
8	13	135	171	1	97
17	106	175	150	237	215
134	23	166	106	178	232
194	66	12	187	83	161

256 *matinvmod a*

79	194	6	146	174	129
30	190	240	162	13	150
246	128	199	74	9	153
73	90	137	61	87	211
11	179	96	240	171	86
186	153	237	33	96	228

b

175	50	54	130	174	65
46	166	248	202	13	54
198	72	47	82	9	185
121	146	33	53	87	179
123	139	24	136	171	182
186	153	237	33	96	228

256|a+.×256 *matinvmod a*

1	0	0	0	0	0
0	1	0	0	0	0
0	0	1	0	0	0
0	0	0	1	0	0
0	0	0	0	1	0
0	0	0	0	0	1

256|a+.×*b*

1	0	0	0	0	0
0	1	0	0	0	0
0	0	1	0	0	0
0	0	0	1	0	0
80	136	40	200	1	32
0	0	0	0	0	1

The Boolean Case

This function will also work just as well on boolean matrices, by setting p to 2. However, in the boolean case there is a fair bit of baggage that can be removed, as follows:

```

      z=boolematinv m;i;j;r;x
[1]  a Inverse of boolean matrix m by elementary transformations
[2]  r+1+pm a Rank
[3]  z+(pm)p(r+1)+1 a Start with identity matrix
[4]  a Get the matrix into upper triangular form
[5]  :For j :In r
[6]    1+j
[7]  a Find the multiplicative inverse of the jth diagonal element
[8]  a This will be 1 if the element is 1, make it 0 otherwise
[9]  a If the inverse is 0, add another row to this one and try again
[10]  :While 0=x+m[j;j]
[11]    [SIGNAL(r=i+1)/1] a There isn't one
[12]    x+m[j;j]*m[i;j]
[13]    :If 0=x
[14]      m[j;]+m[j;]*m[i;] o z[j;]+z[j;]*z[i;]
[15]    :EndIf
[16]  :EndWhile
[17] a Scalar multiply the jth row of both matrices by x
[18]  m[j;]+m[j;]^x o z[j;]+z[j;]^x
[19] a Position [j;j] should now be 1. Subtract row j from the other rows
[20]  :For i :In (j+1)+r
[21]    m[i;]+m[i;]*m[j;]^x+m[i;j] o z[i;]+z[i;]*z[j;]^x
[22]  :EndFor
[23] :EndFor
[24] a Now work backwards to clear the upper triangle
[25] :For j :In 1+r
[26]   :For i :In j
[27]     z[i;]+z[i;]*z[j;]^m[i;j]
[28]   :EndFor
[29] :EndFor

```

Persistence and Shadows

by Stephen Taylor (email: sjt@lambenttechnology.com)

Abstract

Two changes are proposed to the definition of functions: that local objects might retain their values when a function leaves the execution stack, and that names might be shadowed by default rather than by declaration. These changes would replicate and extend the facilities offered in Dyalog APL by namespaces, context switching and `⌈PATH`, and so make redundant the extra scoping rules introduced by them. Close similarities are identified between workspaces and functions so defined.

Introduction

This paper is about workspace logistics: how one organises communications between the different parts of a software system. I use the term *logistics* because I want to draw a military analogy. Military commanders try to keep their lines of communication short. So do good programmers.

Avoiding global variables is an example of this. Reading and setting global variables is a long line of communication when many functions are suspended on the execution stack. Measuring distance in stack levels corresponds to our intuition that global variables are 'far away' from the evaluated code line. When communication paths are long it is harder for the human reader to see what is passing through the global variables. In contrast information passed between functions as arguments and results is 'close to' the evaluation. Short lines of communication help soldiers and programmers understand and control what is happening.

Logistics suggests a related issue: *mobility*. Software is mobile when it is usable in many environments. Mobile software is easy to isolate and test.

The most mobile functions refer to nothing but their arguments and results. This tempts us to write large and monolithic functions, contrary to our good practice of dividing code into short, independently testable functions. The desire to preserve the mobility of sets of related functions has produced over the years both code management systems and extensions to the language, such as packages (SHARP APL) and namespaces (Dyalog APL).

Public and Private Objects

Objects in a workspace¹ can be divided into two distinct classes according to how confidently one can predict the contents of the execution stack when the object's name is resolved.

With some objects one has no idea what the execution stack will look like when reference is made to the object. Utility functions belong to this class. That is the nature of utility functions: one cannot say in advance which other functions will find them useful.

With other objects one can predict very well what the execution stack will look like when reference is made. For example, one function performs part of the work of another, and is called only when the other is already on the stack. We call such a function a *subfunction*, and regard it informally as owned by its calling function. Most commonly, no other object will ever refer to it.

Let us distinguish the first class of objects as *public*, and the second as *private*. We are distinguishing them by the sets of other objects that refer to them.

The issue of mobility occurs differently for public and private objects. Public objects must always be around. They form the environment of the software system. Private objects are subject to management. They can be divided into *modules* individuated by how much of the execution stack they share when called.

The focus of this paper is the mobility of modules. We are thus particularly concerned with private objects.

Persistence

I want to distinguish two properties of local objects that have so far always been treated as the same property.

The names of local objects are localised by a function on the stack; the objects thus become local *to* that function. When the function leaves the stack, the name-value pairings for these names disappear; the objects can be referred to only when the function is on the execution stack; this is what we mean by their property of *locality*.

¹ I am using *object* here indifferently to denote variables, defined functions and operators; not to refer specifically to Dyalog APL namespaces.

Now I want to distinguish from locality the property of *persistence*, which can have two values: *persistent* or *ephemeral*. A local object is *ephemeral* if, when its function is placed on the stack, the local object is undefined. This is how local objects always have behaved. Local objects have always been *ephemeral*.

A *persistent* local object would differ in this respect. When its localising function is placed on the stack, the local object is already defined. The name-value pairing would be the same as when the localising function last left the stack.

What would this buy us?

Predefined constants, default results

Much function code consists of lines that define variables as constants, look-up tables and so on for use by later lines. We can now save these variables inside the functions that use them. No function lines need to be executed to produce them. Functions become clearer to read and faster to execute.

What would this look like? Suppose that we used in the function header colons instead of semicolons to declare persistent local variables. A function that begins:

```
[0]  left FUNCTION right;a;b;c:LookUpTable
[1]  a ← LookUpTable[right]
[2]  ...
```

would suspend in [1] on a value error at the first reference to *LookUpTable*. *LookUpTable* can then be defined and execution resumed. *LookUpTable* will retain its value on subsequent calls of *FUNCTION*.

Many functions examine their arguments for certain conditions in which they will return a default result, otherwise unrelated to the arguments. This default result can now be stored inside the function rather than evaluated each time.

Organising subfunctions

Persistent local objects allow us to bind private functions to the functions that call them. They then appear only when their parent is on the stack and disappear afterwards. No code-management, path setting or references to namespaces is required. Consider a function *CONTROLLER* whose work is divided between three subfunctions. Define it as follows:

```
[0]  CONTROLLER:SUBFN1:SUBFN2:SUBFN3
[1]  SUBFN1
[2]  SUBFN2
[3]  SUBFN3
```

Call it: it suspends on [1] as *SUBFN1* is undefined. Define *SUBFN1*, *SUBFN2* and *SUBFN3*. As soon as *CONTROLLER* leaves the stack, the three subfunctions disappear, to reappear whenever *CONTROLLER* is next executed – exactly and only when they are wanted.

When a function is assigned or otherwise copied, all its private objects come with it by default. No explicit management is required.

Functions can emulate objects, with their own memories

We considered above using persistent local variables as constants. But nothing precludes assignment during execution. For example:

```
[0]  HOWMANYTIMES:Counter
[1]  'You have now done this', (▼Counter+Counter+1), ' times'
```

Consider a subsystem that prepares a document for printing. Many parameters control the result: e.g. page size, margins, footer and header text. These, and objects like the text to be printed, can be conveniently managed in a collection of variables, or file components, ready for spooling or printing. But this is not an arrangement that lends itself easily to replication. In contrast, imagine the process managed by a function with persistent local objects.

```
[0]  DOCUMENT instruction:PageSize:Margins:Header:Footer:Text
[1]  :Select 1>instruction
[2]  :Case 'PAGESIZE'
[3]    PageSize+2>instruction
[4]  :Case 'MARGINS'
[5]    Margins+2>instruction
[6]  :Case 'HEADER'
[7]    Header+2>instruction
[8]  :Case 'FOOTER'
[9]    Footer+2>instruction
[10] :Case 'TEXT'
[11]   Text+2>instruction
[12] :Case 'APPEND'
[13]   Text,+2>instruction
[14] :Case 'INSERT'
[15]   Text+(2>instruction),Text
[16] :Case 'PRINT'
[17]   (PageSize Margins)SPool Text Header Footer
[18] :Case 'EMAIL'
[19]   (2>instruction)EMAIL Text Header Footer PageSize
[20] :EndSelect
```

Now the current state of the document is completely represented by the mobile function *DOCUMENT*. We can trivially make copies at any time. When we do so, everything required comes with the function. Template functions, specialised versions of *DOCUMENT*, can have appropriate initial values for their local variables.²

```
Letter ← A4Letterhead ⍝ take a copy of the template
Letter 'ADDRESS' Correspondent[1 2 3]
Letter 'TEXT' standardLetterText
Envelope ← C4ENVELOPE ⍝ take a blank envelope
Envelope 'ADDRESS' Correspondent[1 2 3]
Letter 'PRINT'
Envelope 'PRINT'
(Email ← Letter) 'PAGESIZE' 'US Letter'
Email 'EMAIL' 'mailbox@lambenttechnology.com'
```

These functions behave like objects in languages like Smalltalk-80: they contain both data and the methods needed to manipulate them. Notice how casually multiple documents can be handled and consider how less conveniently the same objects would be represented using files or reserved globals.

Shadowing by Default

Had APL been designed by a programmer rather than a mathematician, the convention for declaring local names might have been the reverse of what it is.

Conventionally, one declares in a function's header all the names which are to be local; that is, *shadowed*. Reversing the convention, one would declare the names which are to be external, that is, *not shadowed*. (In terms of the length of communication paths, one would declare which names refer to distant objects.) The function header would thus show all the communication paths in and out of the function: through the arguments and through the named external objects.³

Either persistent objects or default shadowing could be implemented without the other. Combining them, however, produces surprising results.

² This fails to take account of the use of shallow reference in Dyalog APL.

³ This applies equally to functions and variables. A function's header would list all the external objects referred to by it or by its subfunctions.

Namespaces and functions

Namespaces in Dyalog APL already exhibit some of the properties discussed. The objects in a namespace are both *local* to it and *persistent*. They can be referred to only when a function local to the namespace has been placed on the stack. The namespace does not have its own entry on the stack, but once implicitly there (by virtue of one of its functions being executed) it shadows all names.

This makes a namespace a good owner for private functions which do not refer to external objects, and for monolithic public functions (for example, utility functions that refer only to their arguments) ; but a poor owner for public functions that do make external references.

Compare a namespace *NS* containing a function *SUB*, with a function *FN* with the same subfunction defined as a persistent local object.

```
[0]  FN expression:SUB
[1]  ⍠expression
```

Then

```
#.NS.SUB 100
```

is equivalent to

```
FN 'SUB 100'
```

Let the function *FN* and the namespace *NS* shadow all external names by default. *FN* could claim two advantages over *NS*:

- The shadow cast by *NS* is impenetrable: *NS.SUB* can refer only to objects local to *NS*. In contrast, *FN.SUB* can be given selective access to objects outside *FN*; declarations in the headers of *FN* and *SUB* would suffice.
- The selective access above could be made general by use of a wildcard for declaring external references. Thus, in the reversed convention, if *EXT* in a function header declares *EXT* to refer to an external object, then *;* * would declare *all undeclared names* to refer to external objects. Correspondingly, *;* * would declare all references to be to persistent local objects.

Scoping rules

The scoping rules used by *FN* are more flexible than those that cover *NS*; they permit more control. Arguably, they are also a narrower extension of classic APL scoping rules.

In Dyalog APL there are three sets of scoping rules, which together determine how names are resolved:

- classic APL scoping rules;
- rules for namespaces of functions called directly from their namespaces;
- rules to cover the effects of $\square PATH$ and $\square CS$, at least one of which (the use of $\dagger$ in $\square PATH$) exists to pierce the shadows cast by namespaces.

By executing FN and its private function on the stack instead of executing a function in its namespace, we reduce the sets of scoping rules from three to two.

By putting FN on the stack instead of setting $\square PATH$, we collapse the scoping rules to just those for functions, plus the variations proposed here. Instead of listing namespaces in $\square PATH$, one would invoke corresponding functions like FN . The execution stack would grow a trifle deeper, but the maintenance programmer would more easily trace references to objects distant from the evaluation, and use only one set of rules in doing so.

Functions and workspaces

From here it is easy to see strong similarities between a workspace and a suspended function.

- A workspace is equivalent to a suspended niladic function with names shadowed by default and all local objects persistent.
- The lines of the function correspond to $\square LX$.
- Where arrangements exist to start a workspace and evaluate an expression in it, this corresponds to invoking a monadic version of the function and evaluating its right argument, as the function FN above.
- Exiting the function is equivalent to $)CONTINUE$.

Conclusion

Two small changes in function definition – allowing local objects to persist, and names to be shadowed by default – combine to replace and extend the facilities offered by namespaces, $\square PATH$ and context switching. Workspaces are seen to be special cases of suspended functions so defined.

MUCIPHER: a New Short Cipher that Escapes Cryptanalysis

by Michel Dumontier (michel.dumontier@free.fr)

What is an idea about which one does not think at all?
Alain (philosopher).

Preamble

We have an answer to Alain's proposal: the mind of an APL or J user is full of ideas which others cannot think: they do not have the "tools of thought", they are neither "symbol-minded", nor "APL-minded", nor "J -minded".

This fact disturbs some people, and to keep their place in the sun they try to eliminate us, APL- or J-minded. Worse still, they despise our precious languages, because it is well known that one despises the things that one does not understand. However, we hear them proclaim with annoying regularity that APL is dead and yet this lie does not suffocate them. I wouldn't be surprised if some of my readers had the same experience...

Why Mucipher? Modulo, Undecipherable, cipher. And it rhymes with Lucifer...

Summary

In this paper I will introduce a ciphering algorithm based on the classes of congruence modulo p , applying the theory described in [1]: some of the possibilities will be presented here but there are several more that I won't describe.

Indeed, the user (as well as the author) can freely adapt the theories because the ideas generated by the experience with J, and in general the array-oriented mindset, possibly following a long experience with APL, are many and varied.

Reminder of Vocabulary

The message (vector of characters M) called *clear* is transformed into a vector of characters C called *cipher* by the sender using a ciphering or coding algorithm. The receiver then takes the cipher C and with a *decoding* or *deciphering* algorithm (the opposite of the *encipherment*), reconstitutes (legitimately) the message M .

An attacker can intercept the cipher C and attempt to reconstitute M , without having all the information on the ciphering techniques used. I'll refer to this act (a true vandalism!) as *hacking*.

Reminder of properties of the inversion modulo p described in [1]

One can represent a file or a message as a string of bytes. A byte can take the values 0 to 255. It can be helpful to remember that the indices in $\square AV$ will vary from 1 to 256, if the index origin is 1.

Let's take matrices whose elements vary between 0 and 255. We have seen that it is useless to go beyond 255 because a matrix A is equivalent to a matrix $256 | A$ in our case.

Let's choose to work in modulo 256 to cover the entire domain.

To encourage the readers to carry on, I will give some idea about how difficult is already the cryptanalysis of such a system, but I warn them to be aware that this is still nothing compared to the obstacles I am going to add later.

The number of 10×10 matrices, built of values from 0 to 255, is 256^{100} , that is: 6668014432879854274079851790721257797144758322315908160396257811764037237817632071521432200871554290742929910593433240445888801654119365080363356052330830046095157579514014558463078285911814024728965016135886601981690748037476461291163877376

This number's order of magnitude is 10^{240} , to be compared with the total number of atoms in the universe, estimated to be around 10^{75} ... it makes you wonder!

But the real strength of the idea goes beyond the impossibility of trying all the combinations. A hacker still wouldn't know that we are working in modulo 256. In fact, if one wanted to send a message, he could restrict himself to a reduced set of alphabet + numbers + 10 symbols at least, and choose p in the range [47, 256].

Cryptanalysis is not limited to the brute force attack, of trying all the matrices: a cleverer approach is based on the analysis of the cipher. In general, the knowledge of the language in which the original message was written, permits one to use the frequencies of the doublets, of the triplets, and perhaps also of the quadruplets, but no more than that ... a simple workaround at this kind of analysis would be to cipher in parcels of 10 bytes, which means that a hacker would need to study frequencies of $256^{10} = 1.20893e24$ combinations of 10-uplets. Want more? What about a matrix of order 20!

The example I will give below should be enough to convince you of the power of this approach. And if it doesn't, nothing will.

Anyway, after this small digression, let us return to our subject.

Here is the coding function for a message taken from the complete ASCII alphabet. The same function serves to *decode*, that is, to recover the original message from the cipher, since one passes as argument the transformation matrix.

```

code=: 3 : 0
:
n=:0{$x.
t=:0$0
while. 0 < $y. do.
t=:t,x. +/ . * n{.y.
y.=.n}.y.
continue.
end.
r=:p|t
)

```

Here's an example:

v

Voici la fonction de codage d'un message pris dans l'alphabet ASCII complet. C'est la même fonction qui sert à décoder c'est à dire retrouver le message original à partir du chiffré puisqu'on lui passe en argument la matrice de transformation.

```

    $v
243

```

The text has 243 characters; it's convenient to pad it, to make it 250 (multiple of 10) characters long.

Let us take p=256.

Here is a matrix a and its inverse b modulo 256.

a

```

85 176 36 74 170 217 80 36 153 1
234 97 131 87 67 74 212 31 161 12
116 96 39 192 169 2 202 143 73 105
81 131 183 214 59 118 95 12 18 51
144 212 107 214 74 46 91 238 150 94
118 160 240 103 12 113 151 190 220 33
26 169 212 148 167 250 4 234 49 246
133 71 183 150 213 212 168 186 163 89
136 169 82 221 112 234 70 242 29 106
213 245 84 134 136 208 237 169 62 240

```

b

```

154 77 9 49 133 74 15 190 99 136
197 59 146 115 39 103 1 237 244 245
174 156 179 186 113 54 196 109 116 83
183 37 27 242 37 15 175 55 191 202
13 141 107 204 191 217 238 201 0 222
155 88 86 133 133 0 29 80 150 226
42 138 171 52 48 36 84 223 110 211
177 76 50 162 226 13 126 144 217 251
80 78 191 15 124 132 102 48 250 17
56 149 22 91 250 158 160 214 103 245

```

```

a=:x: a          NB. Convert to extended integers
det a
10825048232774544296531
b=:a dio p
b=:x: b          NB. the same for b
det b
1780901000902498614235

```

Let us calculate w :

```

+w=:a code a. i. 250{.v
4 221 197 249 209 173 112 149 86 60 201 98 109 252 181 232
255 108 80 171 232 241 117 87 22 57 166 202 43 196 144 54
48 44 193 100 112 242 14 103 149 47 105 66 146 184 91 2
199 87 41 249 123 6 211 234 218 36 53 171 239 14 192 174
64 92 184 56 166 145 185 4...

```

What we will transmit is not the vector of numbers w because that would be too cumbersome, but rather the vector of corresponding ASCII characters.

$,u=:w\{a.$

```
_YÁuÑ~p.V<ÉbmüuèÿlP«èñuW•9|Ê+Ä□60,Ádpòg•/iB'.[_ÇW)ù(_ÓêÚ$5«iÀ@@\.8|'
'1Á@__±OÀ,'-ÿ6qJ<X_¼„™Ä/££pQ6ä™^"Ü5_ç]8W/é•O□ÿ•%zMÀ{xù".?.,|t^iÝ□?
•s¹=YúÜNÀ( Z_qÔDÝ¬'ä^•X†5È.ýWöwxŠ"•_EY¿Öb¬|`Ær¿xÔª. •4π">_ÉÓPýx •¥_.I
p~@k@-oiaÉ Ç"___Q-øaÄfŠF¿²$Xá_ì-S.N+"Ê<~_ù
```

Let's decode u , or, more correctly, w , which is equal to $a. i. u$

```
+x=:b code a. i. u
86 111 105 99 105 32 108 97 32 102 111 110 99 116 105
111 110 32 100 101 32 99 111 100 97 103 101 32 100 39
117 110 32 109 101 115 115 97 103 101 32 112 114 105
115 32 100 97 110 115 32 108 39 97 108 112 104 97 98
101 116 32 65 83 67 73 73 32 99 111 109 11...
```

and read it:

$x\{a.$

Voici la fonction de codage d'un message pris dans l'alphabet ASCII complet. C'est la même fonction qui sert à décoder c'est à dire retrouver le message original à partir du chiffré puisqu'on lui passe en argument la matrice de transformation.

We have of course recovered the text and found the 7 extra spaces added as padding to make the text length a multiple of 10, in J simply 250{.v

Knowing the matrix a , the function `dio` presented in [1] allows us to calculate b according to p . There exists an infinity of pairs a, b that work fine but we will not show here how to calculate matrices a that generate a matrix b that works... If you are interested (and I hope you are a rich businessman), please send an e-mail to the author.

Implementing other transformations

This paper could have stopped here. The cipher algorithm just introduced is equivalent to a long XOR (XOR on the message with a key as long as the message) and the coding, as well as the decoding, is instantaneous.

But I will not stop there. Because this cipher is so quick, we can do 2 or 3 further transformations, similarly reversible, the decoding consisting of executing the transformations in reverse order.

I will only give a single example to save other equally if not more interesting transformations from public exposure, therefore increasing their effectiveness.

Those who wish to know other transformations *will* have to address the author who can provide a great many others. With APL and J one is never short of ideas. Warning: the usage of the cryptography is regulated. In the same way that you have the right to own a kitchen knife, but not the right to use it for unlawful purposes, you can publish revolutionary properties in mathematics, risking everything for fame, but remember you are being watched by big brother, in case you intend to use your discovery in forbidden ways.

In [2] I showed that there were 16 ways to calculate a remainder, all giving different results. Now is the time to show how this can be put to good use. Another revolution: Euclid had only thought of one way of doing this, and that idea lasted for centuries. [2] has passed unnoticed; will people take note of it now, after reading the example given here?

In fact I am not going to give a supplementary transformation yet, but only to substitute the APL (or J, it is the same) remainder primitive by one of the 16 possible remainders described in [2]. The 16 remainders are numbered 0 to 15, that implemented as primitive in APL or J corresponding to number 10.

Let us apply the same transformation on our sample text, this time with a different remainder: I will show that we obtain a different cipher but one that can be decoded by using the new definition of remainder. This is one of the ways I mentioned before to fool the hacker. But since it is given here you will probably want to look for others (or contact yours truly).

While this fact may not surprise everybody, it is still interesting to notice that the matrix inverse (modulo p) is the same whatever kind of modulo one takes. Yet the 16 remainders all give different results...

To keep the paper short, I will give examples in APL and using only matrices of order 4.

```
+A+4 4p~255+16?510
~222 128 71 ~138
~1 ~128 ~89 ~95
~133 229 131 134
~37 ~82 ~65 57
```

Calculating the determinant:

```
DET A
-62052270
```

Calculating the inverse modulo $p=257$ (note: we can take $p=257$ here, because we are not going to work on indices of characters):

```
+B+A DIO P
4 140 148 107
18 140 108 14
18 99 20 130
40 58 148 53
```

I will use the coding function described above called *CODE* (based on APL's modulo primitive), and another function called *CODES* to distinguish it, which uses another modulo taken randomly from one of the 16 possible. Please notice that in [2] I hadn't given a vectorised version of the 16 functions *RESIDUE0* to *RESIDUE15* but left it up to the reader to develop the extension, but since I need them now, I am going to use the vectorised version of *RESIDUEN*, N being passed as a parameter.

```
▽ Z←I MODV V;J;⊂IO;N;C;D
[1] ⊂IO←0 ∘ N←▽I[1] ∘ C←'Z+Z','N',' RESIDUE',(▽I[0]),' '
[2] D←ρV ∘ J←0 ∘ Z←10
[3] BO:⊂C,▽V[J] ∘ →(D>J+J+1)/BO
▽
```

In the coding function, I use the function *RESIDUE1* (taking the absolute value of the result). For those who do not have the paper and want to try the examples, here is the listing of the function *RESIDUE1*.

```
▽ Z←A RESIDUE1 B;⊂IO;TYPE;SGN;T
[1] ⊂IO←0 ∘ TYPE←0 15 ∘ SGN←0 2 1
[2] Z←A|B
[3] →((A=0)∨B=0)/0
[4] T←TYPE[SGN[1+×A]∨SGN[1+×B]]
[5] Z←⊂'A RESIDUE',(▽T),' B'
▽
```

This calls *RESIDUE0*:

```
▽ Z←A RESIDUE0 B
[1] A ALWAYS NEGATIVE
[2] Z←B MODULON A
▽
```

which itself calls *MODULON*:

```

      ▽ Z←A MODULON B;Q;C
[1]  A THE RESIDUE IS ALWAYS NEGATIVE
[2]  Z←B|A
[3]  →(Z<0)/0
[4]  Q←(A-Z)÷B
[5]  Q←Q×B
[6]  Z←A-B×Q
[7]  →((|B)×|Z)/0
[8]  Z←0
      ▽

```

Here is the function *a*:

```

      ▽ Z←A CODES V;□IO;T ;N
[1]  □IO←0 ◊ T←10 ◊ N←1+pA
[2]  BO:T+T,A+.*N+V ◊ V←N+V ◊→(0<pV)/BO
[3]  Z←|(1,P)MODV T
      ▽

```

Here is the test:

```

      +V←127256
9 190 237 222 120 211 121 223 40 156 142 165
      B CODE A CODE V
9 190 237 222 120 211 121 223 40 156 142 165
      B CODES A CODES V
9 190 237 222 120 211 121 223 40 156 142 165

```

We recover the vector *V* on decryption.

But as we said, the cipher is not the same, therefore what is transmitted down the line (what the attacker tries to decipher) is different, and the poor, mystified hacker might as well go and take a running jump!

Here is the proof:

```

      B CODE V
142 211 144 140 86 88 232 248 19 114 105 60
      B CODES V
115 46 113 117 171 169 25 9 238 143 152 197

```

Bravo to those who have followed so far!

I will reward these brave and smart ones with yet an extra diversion. Some will have noticed that we use the 256 ASCII characters, some of which are not strictly

Since the dyadic iota in APL gives only the first occurrence, searching for the index of any character of AL in AL will always give an index in the range [1, 122].

An example at random:

```
AL ⍋ 'é'
61
```

This is where the function *IOTA* comes in handy to calculate the indices of all occurrences.

```
⍋ Z+X IOTA Y
[1] Z+(X=Y)/⍋X
⍋
```

Let us look for the 2 places where our character occurs in the table:

```
AL IOTA 'é'
61 176
```

You have guessed the trick by now: I am going to use a modulo of 244 to give me numbers in the range [0,243], as if I was working with an alphabet of 244 characters.

For each character of the message M to be encrypted we found the corresponding index: its position in the AV; now, we are going to pick its index randomly in one of the 2 repetitions in AL, so that the vector V representing the indices of the characters of M will consist of numbers in the range [0,243].

As usual we will calculate the transformed vector W also composed of numbers in the range [0,243], and for the cipher character vector C to be transmitted we will take from the indices W in the table AV on which we have carried out a complete permutation.

This permutation of AV is called AP (from the French *alphabet permuté*).

```
,AP+AV[244?244]
Qà+0án+ÊÊ*→i+xœBg2ó+qê["â&vTεW{wK0ib|ù.¿fφu+|γ`ιħI<CĀñH~Jō|ŪιU
f|A4u;|ô+kλ-θL8Y|ō+ÑÇT9>⊞PF=α|Ç]=S1~θ::|ϕtϕ_G⊞i⊞(JM7~Vv⊞+Dc:a-n1
đ1⊞"iâΔ×#εZ+r⊞Ā|~R<Ê⊞é|?ôψ\+û⊞f o+⊞Z+êÊτu≤l oΓP-£/5ΔúĀ!6Ñø~m#|I=@
X#ūāwy'° ⊞$-δE%~eŪ~+Êsâō⊞→"⊞v+3n[~,+ÑεΔ×ō)øjΔ[β;ο&L+ρ)
```

A possible pair of matrices A, B working in modulo 244 is as follows:

A

33	192	116	135
55	11	173	173
238	97	132	211
8	13	135	171

B

176	15	128	243
27	21	181	185
44	174	193	109
179	23	216	211

DET A

302408861

Listing of the test function:

```

v TEST X; IO
[1]  IO←0 ∘ P+ΔΔP+244
[2]  'CALCULATION OF INDICES OF V'
[3]  N←4×[(ρX)÷4 ∘ X+N+X ∘ I←0 ∘ V←10
[4]  BO:V←V,(AL IOTA X[I])[?2]
[5]  →(N>I+I+1)/BO ∘ V
[6]  'CALCULATION OF TRANSFORMATION OF INDICES'
[7]  W←A CODES V ∘ W
[8]  'CALCULATION OF THE CIPHER TO SEND'
[9]  C←AP[W] ∘ C
[10] 'DECODING OF C'
[11] '1) RECONSTITUTION OF W'
[12] □←W←AP1C
[13] '2) RECONSTITUTION OF V'
[14] □←V←B CODES W
[15] '3) RECONSTITUTION OF CLEAR'
[16] AL[V]
v

```

The message to send, code and decipher is:

M

Maintenant, vous n'avez plus qu'à aller vous faire cuire un œuf!

Here is the test:

TEST M

CALCULATION OF INDICES OF V

```
85 157 11 13 192 195 13 33 13 192 17 18 54 110 229 0 167 13 96 157 54 195 78
167 243 53 77 0 167 123 77 96 65 167 157 164 53 195 165 167 213 110 229 0 1
67 162 33 184 116 195 167 31 77 184 116 34 167 77 13 18 4 229 162 27
```

CALCULATION OF TRANSFORMATION OF INDICES

```
132 182 118 159 37 77 154 243 29 146 56 79 66 123 174 163 166 95 58 168 189
80 48 158 201 59 61 148 221 37 131 22 103 55 111 42 134 212 26 133 167 162 1
52 111 109 48 180 167 79 165 44 167 205 243 144 162 167 221 219 150 75 188 1
8 28
```

CALCULATION OF THE CIPHER TO SEND

```
4*+[]80)[]|4-5Ez#00X0i+0|i|n|â|tHJφ#0&*=é\J&iN|u+le)<êln+V000-γ
```

DECODING OF C

1) RECONSTITUTION OF W

```
132 182 118 159 37 77 154 243 29 146 56 79 66 123 174 163 166 95 58 168 189
80 48 158 201 59 61 148 221 37 131 22 103 55 111 42 134 212 26 133 167 162 1
52 111 109 48 180 167 79 165 44 167 205 243 144 162 167 221 219 150 75 188 1
8 28
```

2) RECONSTITUTION OF V

```
85 157 11 13 192 195 13 33 13 192 17 18 54 110 229 0 167 13 96 157 54 195 78
167 243 53 77 0 167 123 77 96 65 167 157 164 53 195 165 167 213 110 229 0 1
67 162 33 184 116 195 167 31 77 184 116 34 167 77 13 18 4 229 162 27
```

3) RECONSTITUTION OF CLEAR

Maintenant, vous n'avez plus qu'à aller vous faire cuire un œuf !

Let us note in passing that the indices in V are indeed in the range [0,243] and also that we have been careful to write 'œuf' correctly, which forced us to include the symbol corresponding to the APL $\diamond$ for the 'œ'.

Reminder of other properties (for the cryptanalysts).

We reproduce the TEST function as far as the ciphering. In order to see the cipher produced, we name it TESTC :

V TESTC X;[]IO

```
[1] []IO←0 ⋄ P+ΔΔP+244
[2] A'CALCULATION OF INDICES OF V'
[3] N←4×[(ρX)÷4 ⋄ X+N+X ⋄ I←0 ⋄ V←10
[4] BO:V←V,(AL IOTA X[I])[?2]
[5] →(N>I+I+1)/BO ⋄ A V
[6] A'CALCULATION OF THE TRANSFORMATION OF INDICES'
[7] W←A CODES V ⋄ A W
[8] 'CALCULATION OF THE CIPHER TO SEND'
[9] C←AP[W] ⋄ C
```

V

Property 1: The same message sent consecutively has nothing like the same cipher (caramba!).

If you experimented at home and did not obtain the same cipher as in the examples, please stop scratching your head: now, you (must) have understood!

Proof:

```

TESTC M
CALCULATION OF THE CIPHER TO SEND
7Z<eà}-Ai|CÀ=è\Juñ+T/wjÈ^ΔIoA*qavñi[èmoniυō=8zuOΔ| |□3ûx~f 4~\a
C

```

```

TESTC M
CALCULATION OF THE CIPHER TO SEND
υUCΔoZ÷<~"voif□?/τm□*†Á□XtYq&fâAm\ô/It;1-z~Eâυ8zu□-5□γ×Uqei-!a_>$
□

```

Property 2: If one shifts the message, even by one place, all is changed:

Proof: (to have the same cipher, it is enough to reset the random seed)

```

□RL+11111
TESTC M
CALCULATION OF THE CIPHER TO SEND
këvΔâ}-A-c£EG+εiβY^/wjÈ5omûc+Ëçqou3;1-z□?/τ□ûóR|u+l_εwEaf~\~>$
□

```

```

□RL+11111
TESTC M
CALCULATION OF THE CIPHER TO SEND
këvΔâ}-A-c£EG+εiβY^/wjÈ5omûc+Ëçqou3;1-z□?/τ□ûóR|u+l_εwEaf~\~>$
□

```

OK? Let's go:

```

□RL+11111
TESTC 1φM
CALCULATION OF THE CIPHER TO SEND
vW\â~dñ"ñn"}â[Δ|~"7\w+W†"□û"+→2uθ□@@ceô3âAÖN;=GéD+pZφυ>|τÇ\cÄËÖl

```

Property 3: Even if the random seed is reset, it's enough to change a single character in an n-uplet for everything to change! (n being the order of the coding and decoding matrices.)

Here is a test:

```

□RL+123456789
TESTC M
CALCULATION OF THE CIPHER TO SEND
-1, > * Ñ' o □ | ð c + Ñ ð ð - c / w j Ë ú ð ð A - ð K ð ð / I t | U 7 + = è \ J 8 ð u □ Ñ Ë l 7 * U q € â □ α o _ + $ 8
U + ( 4 * 1 6 ) + 2 1 6 p 4
M 1 + M o M 1 [ U ] + ' X '
□RL+123456789
TESTC M1
CALCULATION OF THE CIPHER TO SEND
NKÖ4Ç"Y+uøM18o7||ß+|Ñ.6|≤-7 * p ß Ñ + # F Æ H c h " " L 7 ã m □ w u s Y | Ñ H G 1 / C u ð i e p v f -

```

It is because at any position in the cipher there is effectively a random character, that we said this algorithm is equivalent to a XOR with a key as long as the message itself.

Any of you with a light in their eyes? If some find this brilliant, I hope that they will have understood!

Do you believe that:

1. a non-APLer would have been able to imagine and write that?
2. the non-APLers have understood? I have my doubts: we will experiment.

And now, go to your keyboards! And have fun.

Some grumpy spirits (or mad!) will still claim that this is decipherable.

OK, here is a text enciphered by this method: the person who can decipher it will have the right to see the next obstacle!

```

hDb Šð..°¿Æ~ßUh+ã ð1ZÇÿFy²+ Ü·Ôlù*çù†"ß8~9ð ð"
~|°p ¼)rVm[m3 ¹±1( 3CÜ!|)7qN°Fw^ fðTn»JÛÁÖÆ ã\SÇE

```

There are a lot of "gaps" in the cipher above, so let us give it in integer values to distinguish them (96 values):

88	219	214	172	66	181	208	197	39	119	93	200
25	29	0	88	58	6	114	181	75	30	124	111
37	111	98	201	144	138	189	155	184	176	3	131
176	52	71	29	125	13	28	4	203	172	159	162
97	1	10	119	141	5	61	109	195	211	213	205
213	113	217	22	158	75	198	218	113	178	16	153
7	163	73	157	9	37	171	9	12	173	203	174
44	224	166	225	169	185	200	121	40	32	102	124

But I don't think it is necessary to raise additional barriers... I remind my readers that the previous cipher is equivalent to producing a random character set... Let us just give the name of a single supplementary tool that one can use, tweaked or not: the exponential modulo.

Note: to make the vectors v and w more 'random' or more deceptive, one could take a vector $AV \leftarrow (1024 \rho AV) [1024 ? 1024]$, which implies that each character of $\square AV$ is found there 4 times (in this case $p=1024$), and then do the same operation on AL . Then one would take $V \leftarrow V, (AL \text{ IOTA } X[I]) [?4]$. Then do the same thing to AV . One could also, when coding, rotate AV one place each time: $AV \leftarrow 1 \phi AV$, and when decoding do the opposite: $AV \leftarrow -1 \phi AV$.

There are even worse (for the hacker) ways to mangle the ciphers, ways that come from the APL or J culture, but I shall keep them undisclosed for precaution.

Conclusion

When one has APL and J tools (and knows how to use them...) it is useless to go mad with the prime numbers of the RSA. First of all, nothing proves that nobody has been able to factorise big numbers. Moreover, it is even possible to break the RSA without factorisation. And the same applies to other fashionable encryption schemes...

References

- [1] Michel J. Dumontier. Inversion matricielle modulo p , Systèmes linéaires modulo p . *Les Nouvelles d'APL* N° 35 pp. 45-54. Translated as Matrix Inversion modulo p , linear Systems modulo p , *Vector* 18.1. p.134
- [2] Michel J. Dumontier. Residue and quotient revisited, *APL-CAM Journal*, Vol 13, No 2. Copyright 1991: Michel J. Dumontier
- [2bis] Michel J. Dumontier. Résidu et quotient revisités. *Les Nouvelles d'APL* N° 35 pp. 33-44

Function Models

by Phil Chastney (*philip_chastney@yahoo.com*)

1. Introduction

We have defined a simple descriptive formalism, and used it to define the data domains available in APL. Because functions can be characterised by the data domains of their inputs and outputs, we started to examine ⁶, the functions domain, and we shall continue with that examination in a little more detail.

The expression defining the domains of a function's inputs and outputs is known as the **function mapping**, or **model**. The terms are used interchangeably.

Wherever the term "function" is used without qualification, this may be taken to mean a "partial function". A function is a mapping from a set of input values to a set of output values. A **total function** is defined for all elements of the input set; a **partial function** need not necessarily be defined for all elements of the input set. Thus, the *reciprocal* function, whose mapping is $\mathbb{R} \leftarrow \mathbb{R}$, is not defined for zero, and is therefore a partial function, while the *signum* function, whose mapping is $\mathbb{Z} \leftarrow \mathbb{R}$, is a total function. We shall have little need to identify which functions are total functions.

Unless we wish to add the rule explicitly, there is nothing which automatically restricts an object to being a function or a variable.

There is nothing restricting the arguments or results of a function to data items. A function can take a function definition as an argument, in which case it is often known as a **functional** or an **operator**. Examples are the *reduction* operator in APL, and the differential operator in calculus.

There is nothing requiring the arguments or result(s) of a function to be exclusively functions or data – as witness the use of slash to denote compression or reduction, according to context.

There is nothing restricting a function to a single mapping. The complete mapping for a particular function may consist of the domain sum of other, simpler, mappings.

2. Dyadic Functions

The function models so far considered have all been monadic.

A dyadic function could be defined with a mapping over the domain product of its left and right arguments, but there are difficulties. We could, for instance, define a function *ADD* as having the mapping $R \leftarrow R \otimes R$, which is entirely correct, so far as it goes, but suggests that in practice the function is applied to a 2-element numeric right argument. It would be better if we could also indicate the syntax of the call in some way.

This desire becomes a necessity with mixed functions such as *reshape*, whose mapping (ignoring character right arguments) would be something like

$$(R * N * N) \leftarrow (N * N) \otimes R * N * N$$

which looks OK, but might lead us to interpret $(2 \rho 3 \ 4)$ as defining a 2-by-3 matrix filled with 4s. (Why? Because the mapping requires an integer vector for the new shape, and can accept any simple array (including scalars) as the element list. This must mean that the shape is 2 3, and the element list is the scalar 4.)

It is common practice to construct a logical calculus with 1-place functions only. This can then be extended to n-place functions via a mechanism popularised by Curry, but actually due to Schönfinkel. The process is best illustrated using conventional notation:

the application of a 3-place function f , to arguments a , b and c , conventionally written $f(a,b,c)$, can be conceived to be the following process:

- i) construct a 1-place function f' which can be applied to a to yield another 1-place function f''
- ii) apply f'' to b to yield another 1-place function f'''
- iii) apply f''' to c , to yield the final value.

As an example of that ready wit for which logicians are everywhere famous, the f', f'', f''' , etc, are known as **Curried functions**.

Assuming conformance, the APL function call *ADD(THIS;THAT)* could be deemed to represent the following sequence:

$$\begin{aligned} \text{ADD(THIS;THAT)} &\equiv (\text{ADD1 THIS}) \text{ THAT} \\ &= \text{ADD2 THAT} \\ &= \text{result} \end{aligned}$$

We would expect the result to be an element of $\mathbb{R}$, so that the function ADD2 has the mapping $\mathbb{R} \leftarrow \mathbb{R}$; this function is the result of applying ADD1 to THIS, so that the function ADD1 has the mapping $(\mathbb{R} \leftarrow \mathbb{R}) \leftarrow \mathbb{R}$. Since the expression ADD(THIS; THAT) is merely a syntactically convenient way of describing the above process, we may say that the mapping for ADD itself is also $(\mathbb{R} \leftarrow \mathbb{R}) \leftarrow \mathbb{R}$. This, or something similar, is the notation used in most books on semantics. Or the manuals for functional programming languages, where this sort of stuff is commonplace.

Unfortunately, it gives us no indication of the order of evaluation. The sequence

$$\begin{aligned} \text{ADD}(\text{THIS}; \text{THAT}) &= (\text{ADD1 } \text{THAT}) \text{ THIS} \\ &= \text{ADD2 } \text{THIS} \\ &= \text{result} \end{aligned}$$

also gives a final mapping of $(\mathbb{R} \leftarrow \mathbb{R}) \leftarrow \mathbb{R}$.

To revert to infix notation, observe how the Curried function approach reflects the process in the APL interpreter. In an expression such as

THIS × THAT,

which may be seen as shorthand for

(TIMES_LEFT THIS) THAT,

the left argument is examined, and

if LA null, (TIMES_LEFT THIS) $\in (\mathbb{Z} \leftarrow \mathbb{R})$

if LA non-null, (TIMES_LEFT THIS) $\in (\mathbb{R} \leftarrow \mathbb{R})$

and the resultant function applied to the right argument THAT. This means that the mapping for TIMES_LEFT is

if LA null, TIMES_LEFT $\in (\mathbb{Z} \leftarrow \mathbb{R}) \leftarrow \mathbb{R}$

if LA non-null, TIMES_LEFT $\in (\mathbb{R} \leftarrow \mathbb{R}) \leftarrow \mathbb{R}$

which may be summarised as

TIMES $\in ((\mathbb{Z} \leftarrow \mathbb{R}) \leftarrow \mathbb{R}) \oplus (\mathbb{R} \leftarrow \mathbb{R}) \leftarrow \mathbb{R}$.

Problem solved. We shall adopt the convention that, in an expression like $(A \leftarrow B) \leftarrow C$, we have the domain of the result (A), the domain of the *right* argument (B) and the domain of the *left* argument (C). Or, to put it another way,

$A \leftarrow C \text{ } f \text{ } B \text{ iff } f \in (A \leftarrow B) \leftarrow C$.

This convention captures the fact that (i) the "control" argument is conventionally on the left, and (ii) while evaluating from right to left, the interpreter will first check out any left argument, before deciding what to do with the right.

We would also observe that any mapping of the form $\langle D \leftarrow \lambda \rangle$ is identically equal to D alone, so that the expression could be simplified to

$$\text{TIMES} \in \langle \mathbb{Z} \leftarrow \mathbb{R} \rangle \oplus \langle \mathbb{R} \leftarrow \mathbb{R} \rangle \leftarrow \mathbb{R}$$

except that we lose any indication of whether the monadic definition takes its argument on the left or the right.

With arithmetic functions, however, there is no obvious "control" variable and most of us would feel probably more comfortable with $\text{ADD}(\text{THIS}; \text{THAT})$ than $\langle \text{ADD1 THIS} \rangle \text{ THAT}$, even given that the expressions are equivalent.

We can avoid such difficulties if we explicitly identify the domain of the left argument and the domain of the right argument separately. For this, we need a **sequence** or a **list of domains**.

In keeping with our policy of using distinguishing syntax for the domain definitions, we enclose a domain list with angle brackets. The list elements are domain expressions, separated by semicolons, e.g:

$$\langle A * N; B \oplus C \rangle .$$

The significant difference between a list of domains and their cross-product lies in the way they are affected by null entries. If we denote the cardinality of domain A by $n(A)$, then (in conventional notation) the cardinality of a cross-product is

$$n(A \otimes B) = n(A) \times n(B)$$

which evaluates to zero, if either domain is empty. The domain list,

$$\langle \lambda; \mathbb{R} * \mathbb{N} * \mathbb{N} \rangle$$

on the other hand, still has two separately identifiable elements, albeit one of them is null. If we adopt the convention that all function models require a two-element domain list, then two non-null elements indicates a dyadic function, while monadic functions would be defined with one element null.

Another possible refinement is to include the names (α and w) of the left and right arguments in the domain list. We can now avoid all ambiguity, by defining the mapping of the *reshape* function on numeric arrays thus:

$$(\mathbb{R} * \mathbb{N} * \mathbb{N}) \leftarrow \langle \alpha \in \mathbb{N} * \mathbb{N}; w \in \mathbb{R} * \mathbb{N} * \mathbb{N} \rangle$$

which we can improve by adding in the definition for character arrays, and allowing for scalar left arguments, to give:

$$\begin{aligned} \langle \mathbb{H}^* \mathbb{N}^* \mathbb{N} \rangle &\leftarrow \langle \alpha \in \mathbb{N} \oplus \mathbb{N}^* \mathbb{N}; \omega \in \mathbb{H}^* \mathbb{N}^* \mathbb{N} \rangle \oplus \\ \langle \mathbb{R}^* \mathbb{N}^* \mathbb{N} \rangle &\leftarrow \langle \alpha \in \mathbb{N} \oplus \mathbb{N}^* \mathbb{N}; \omega \in \mathbb{R}^* \mathbb{N}^* \mathbb{N} \rangle. \end{aligned}$$

An example: *addition* is a scalar dyadic function, and therefore has the mapping $\mathbb{R} \leftarrow \langle \mathbb{R}; \mathbb{R} \rangle$, while the *derived reducing function* has the mapping $\mathbb{R} \leftarrow \mathbb{R}^* \mathbb{N}$, so that the *reduction operator* itself has the mapping

$$\langle \mathbb{R} \leftarrow \mathbb{R}^* \mathbb{N} \rangle \leftarrow \langle \alpha \in \mathbb{R} \leftarrow \langle \mathbb{R}; \mathbb{R} \rangle \rangle.$$

This is quite a neat definition. If we dispense with α and ω , it becomes

$$\langle \mathbb{R} \leftarrow \mathbb{R}^* \mathbb{N} \rangle \leftarrow \langle \mathbb{R} \leftarrow \langle \mathbb{R}; \mathbb{R} \rangle; \rangle.$$

Using the $\langle A \leftarrow B \rangle \leftarrow C$ convention defined above, the definition becomes

$$\langle \langle \mathbb{R} \leftarrow \mathbb{R}^* \mathbb{N} \rangle \leftarrow \rangle \leftarrow \langle \mathbb{R} \leftarrow \mathbb{R} \rangle \leftarrow \mathbb{R}.$$

which may be less perspicuous.

That, at least, is the story if the interpreter permits the expression $+/$ to appear without a right argument. Many interpreters insist that the derived function be applied immediately, in which case the slash looks more like an infix function, with the mapping (for the reduction operation) of

$$\mathbb{R} \leftarrow \langle \mathbb{R} \leftarrow \langle \mathbb{R}; \mathbb{R} \rangle; \mathbb{R}^* \mathbb{N} \rangle$$

and another (for the replication operation) of

$$\langle \mathbb{R}^* \mathbb{N} \rangle \leftarrow \langle \mathbb{N}^* \mathbb{N}; \mathbb{R}^* \mathbb{N} \rangle$$

giving a combined definition of

$$\langle \langle \mathbb{R}^* \mathbb{N} \rangle \leftarrow \langle \mathbb{N}^* \mathbb{N}; \mathbb{R}^* \mathbb{N} \rangle \rangle \oplus \mathbb{R} \leftarrow \langle \mathbb{R} \leftarrow \langle \mathbb{R}; \mathbb{R} \rangle; \mathbb{R}^* \mathbb{N} \rangle,$$

unless, of course, you prefer

$$\langle \langle \mathbb{R}^* \mathbb{N} \rangle \leftarrow \mathbb{R}^* \mathbb{N} \rangle \leftarrow \mathbb{N}^* \mathbb{N} \rangle \oplus \langle \mathbb{R} \leftarrow \mathbb{R}^* \mathbb{N} \rangle \leftarrow \mathbb{R} \leftarrow \langle \mathbb{R}; \mathbb{R} \rangle.$$

3. Storage and Assignment

Even with the foregoing, we still cannot describe the assignment function adequately. For this we need two more primitive domains.

The stages in interpreting a statement such as

NAME $\leftarrow$ EXPRESSION

are roughly as follows:

- evaluate EXPRESSION, and leave the result somewhere in memory;
- find (or make) an entry in the symbol table for NAME;
- (re)set the pointer associated therewith to point to the result of EXPRESSION.

These operations involve two new domains, whose elements are (i) names, and (ii) something approximating to memory locations.

The following quotation is taken from §2.1 of "The Varieties of Programming Language":

"The domain ... *Id*, is the set of *names* or *identifiers* in the programming language. [It] is an elementary domain, and it is also the only domain we shall encounter whose members are elements of the text of the program, and are therefore parts of the programming language itself, and not objects manipulated by it. *Id* is generally defined by the syntax rules of the programming language. It is a very simple domain whose only basic property is the relation of equality – it is always possible to say if two names are the same or not, but names in general have little internal structure.

The only remarkable thing about *Id* is the number of different words which have been used to describe its members. The fact that they have been called "names", "identifiers" and "variables" in different languages would not matter so much if these same words had not been also used with quite different meanings."

That which Strachey refers to as "*Id*" will be denoted herein by the double-struck character $\mathbb{I}$.

Strachey then goes on to formalise the idea of "storage" – splendid pioneering work, but flawed.

"The problem with inconsistent formalizations (like Strachey's potentially was) is that they provide no basis for reliable reasoning", (Gordon, 1979). Point taken: Gordon himself (p.15) defines **state** (for a tiny language) as follows:

State = Memory $\times$ Input $\times$ Output
 Memory = $\mathbb{I} \rightarrow \{ \text{Value} + \{ \text{unbounded} \} \}$
 Input = Value*
 Output = Value*
 Value = Num + Bool

We recognise the domain Value as our (implementation dependent) universal domain $\mathcal{V}$, and since the domain mapping function includes, by definition, all partial functions, we do not need the {unbounded} element Gordon introduces, so we can rewrite the above as follows:

$$\begin{aligned}\mathcal{V} &= \text{Memory} \times \text{Input} \times \text{Output} \\ &= (\mathcal{V} \leftarrow \mathcal{V}) \otimes (\mathcal{V} * \mathbb{N}) \otimes (\mathcal{V} * \mathbb{N}).\end{aligned}$$

Gordon's formulation allows for just two channels: Input and Output; Unix defines three by default: *stdin*, *stdout* and *stderr*; a realtime rate feed may have half a dozen. We would allow arbitrarily many:

$$\text{CHANNELS} = (\mathcal{V} * \mathbb{N}) * \mathbb{N}.$$

Precisely what is required to describe the machine's state at any point in the execution of a program varies from author to author. A useful definition would be for **state** to include

- all **environment** information ($\square \text{IO}$, $\square \text{CT}$, ... , state of all channels, etc);
- the current **symbol table** (consisting of pairs of <name, value pointer>, with the most recent name entry being the one deemed to be currently in scope);
- the **memory**, containing the current value (if any) of all named variables;
- the current **stack** (only one stack is required in theory).

This would give us a definition of

$$\mathcal{S} = \text{Environment} \times \text{Symbol_Table} \times \text{Memory} \times \text{Stack} \times \text{Channels}.$$

Not that it matters a lot. At this stage, we shall content ourselves with identifying changes to a state whose precise nature we at present leave unspecified.

Each change in state will be an element of $\mathcal{S} \otimes \mathcal{S}$, the set of all pairs of "before" and "after" states. The domain of all state changing functions is, of course, $\mathcal{S} \leftarrow \mathcal{S}$, which, being the set of partial functions from $\mathcal{S}$ to $\mathcal{S}$, admits the existence of states we cannot change out of.

If a function FOO has side-effects, i.e, FOO is a function whose execution produces a change of state, then we say

$$\text{FOO} \in (\mathcal{S} \leftarrow \mathcal{S}).$$

Right parenthesis is just such a function, since its action is to place a marker on the stack. Semicolon is another. Left parenthesis is yet another: it delivers the

topmost item or items from the stack, down to the most recent right marker, and finally removes the marker itself. Left parenthesis therefore has the mapping

$$(\mathfrak{s}_9 \leftarrow \mathfrak{s}_9) \otimes (\mathfrak{r}_9 * \mathbb{N}) .$$

Assignment is another such a function, and (for the time being) its mapping is

$$(\mathfrak{s}_9 \leftarrow \mathfrak{s}_9) \leftarrow (\mathfrak{r}_9 ; \mathfrak{s}_9) .$$

4. Headers

For primitive functions, the obvious repository for all this is the APL standard⁴. For user-defined functions, the obvious repository is the function header.

Currently, the function header provides a syntactic template and a list of locals. Our suggestion is that the localisation of names be performed separately, and that the present syntactic template is replaced with a more informative semantic specification.

In many cases, there will be no arguments and no result: in such cases a default header of $\mathfrak{s}_9 \leftarrow \mathfrak{s}_9$ can be assumed. This tells us that the purpose of the function is to produce a change of state (an externally visible change of state, that is), by reading input, producing output, or assigning a value to an external variable.

Where a header is provided, it will be the function mapping. From this, the interpreter can determine the function's valence, including any ambivalence, any restrictions on the type or rank of arguments, whether the block has any side-effects, whether the function is actually a user-defined operator, etc.

There will no longer be any need to name the result or the arguments.

With an explicit header, side-effects will be permitted only if they have been included in the header. If side-effects have not been implicitly or explicitly allowed for, then any attempt to assign a new value to an external variable, for instance, will be reported as a DOMAIN ERROR.

The polyglot reader will already have noticed that specifying the domain(s) of the parameter(s) and the domain of any result is very close to the idea of pre- and post-conditions of abstract data types.

⁴) except that a rigorous prosecution of the need for consistency may suggest changes which make consensus difficult to obtain

By way of introduction to the subject, the following quote from Rachel Harrison may be useful:

In Guttag (1977) the term *abstract data type* is used to refer to "a class of objects defined by a representation-independent specification".

- A data type (or type) is a means by which we can classify objects which have identical properties. A data type can be either intrinsic (built-in) or user-defined.

When using data abstraction, we are more concerned with *what* we can do with the data types than in *how* it is implemented. This leads to the definition shown below:

- An abstract data type (or ADT) consists of a data type together with a set of operations, called *access functions*, which define how the data type may be manipulated.

These two definitions imply the use of a strongly typed language.

Well, maybe, but not necessarily . . .

5. Parameters

As already noted, any left argument is denoted by α , any right argument is denoted by w .

If these two names are excluded from the domain $\mathcal{A}$, they cannot appear on the left-hand side of an assignment. Thus the slightly dodgy habit of updating actual parameters is prevented, and all functions without side effects can be rerun from the top — a useful thing to have when debugging.

When the function is applied, the actual arguments are evaluated, and, where applicable, type and rank checked against that specified by the mapping. In the event of a mismatch, the function will report a DOMAIN ERROR without proceeding further. This is known as **strong typing**.

"Strong typing", said the sage, "is for weak minds". What this mechanism provides, however, is *optional* strong typing. The parameter checking can always be rendered ineffective by specifying a header like:

$$(\lambda \oplus^{\mathbf{a}_2}) \leftarrow (\lambda \oplus^{\mathbf{a}_2}; \mathbf{a}_2)$$

which may or may not return a result, may or may not take a left argument, but certainly takes a right argument, which may be of any arbitrary type, shape, size, colour or creed.

6. Results

The actual domain of the result will also be checked for type and rank against that specified in the function mapping. In the event of a mismatch, the function will report a DOMAIN ERROR.

This, of course, begs the question of how we identify the result of a function. There has not, heretofore, been any reserved name for the result of a function, and *w*, the most poetically apposite name, has already been appropriated.

We could make some rule about using the value of the last expression executed before exiting, or maybe the last uncommitted value before exit. C coders use

```
return expression
```

which identifies the value to be returned, and then exits the function. This is bad design. The two actions should be separated, thus:

```
result expression  
return
```

which would permit post-processing (restoring the environment, for instance) after the result has been obtained.

We do not need the **return** function, since APL is already adequately equipped in that respect, but the addition of a **result** function would be really useful.

Each use of the result function would trigger a check that the value of *expression* fell within the domain prescribed in the function header. In the event that it did not, the interpreter would report a DOMAIN ERROR within the function.

As a particular case of the above, any attempt to use a result statement with a non-null argument, when the function header specified a null result, would also trigger DOMAIN ERROR within the function.

Exiting a function without having declared a result, when one was expected, would trigger a RESULT ERROR outside the function.

7. Caveats

The complete model for a function, which may consist of more than one simple mapping, should enable us to identify unambiguously which mapping to apply, knowing only the type and rank of the arguments. This may not be easy.

Conformance requirements are not addressed. They would require a rather more elaborate apparatus than that being developed here, which would hide the essential elements of this discussion.

"Shy" results too could be accommodated with more syntax.

If the function header is redesigned as suggested, we need to address the question of how local variables are identified. We shall tackle this in due course.

Those deliberations will also generate a requirement for user-defined named domains, and that will in turn suggest other uses for named domains.

8. References

- 8.1 H B Curry, R Feys, *Combinatory Logic, Vol 1*, North Holland, Amsterdam [1968]

This is the standard text, which appears to have been a major influence on the design of J, although the device now known as "Currying" first appeared in —

M Schönfinkel, *Über die Bausteine der mathematischen Logik*
(in) *Mathematische Annalen*, 92 [1924]

- 8.2 Christopher Strachey, *The Varieties of Programming Language*
(in) *Proc. International Computing Symposium*,
Cini Foundation, Venice [1972]

It is highly likely that this is the paper which has been reprinted in —
Peter O'Hearn, Robert D Tennent, *Algol-like Languages, Vol 1*,
Springer-Verlag [1996]

- 8.3 Michael J C Gordon, *The Denotational Description of Programming Languages*
Springer-Verlag, New York [1979]

- 8.4 Rachel Harrison, *Abstract Data Types in standard ML*
Wiley, Chichester [1993]

Harrison herself references —

J V Guttag, *Abstract Data Types and the development of data structures*
CACM 20 [1977]

Index to Advertisers

Dyadic Systems Ltd	8
MicroAPL	2
Soliton	6
Strand Software	96
Vector Back Numbers	5

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2001/2002 Committee

Chairman	Adrian Smith 01439-788385 apl385@compuserve.com	Brook House Gilling East YORK YO62 4JJ
Secretary	Anthony Camacho 0117-973-0036 acam@tesco.net	11 Auburn Road Redland BRISTOL BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road Waterbeach Cambridge CB5 9NJ
Journal Editor	Stefano Lanzavecchia stf@apl.it	c/o APL Italiana Corso Vercelli 54 20145 - Milano Italy
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School Swansea University Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 ray_cannon@compuserve.com	21 Woodbridge Road Blackwater, Camberley Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@csi.com	138 Burton Stone Lane YORK YO30 6DF
Education	Vacant post	
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road Waterbeach Cambridge CB5 9NJ

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Anthony & Sylvia Camacho (0117-973-0036), Ray Cannon (01252-874697), Stephen Taylor (077-1340-0852) Bob Hockstra (01483-771028), Jon Sandles (01904-612882), Marc Griffiths	

Typeset by APL-385 with MS Word for Windows

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Maltings, Castlegate,
MALTON, North Yorks YO17 7DP
Tel: 01653-696760
Fax: 01653-697719
Email: sales@causeway.co.uk
Web: www.causeway.co.uk

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01438-514500

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants. RG24 0AT
Tel: 01256-811125
Fax: 01256-811180
Email: sales@dyadic.com
Web: www.dyadic.com

Dutch APL Association
Postbus 1841
3480 BH Nieuwegein
Netherlands
Tel: +31 347 342 337

Insight Systems ApS
Nordre Strandvej 119C
DK-3450 Hellebæk
Denmark
Tel: +45 70 26 18 26
Fax: +45 70 26 18 25
Email: info@insight.dk
Web: www.insight.dk

HMW Computing
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0JA
Tel: 0270-1010-469
Email: HMW@4xtra.com

MicroAPL Ltd
Pine Roller Mill, Mill Lane
Uckfield
E Sussex TN22 5AA
Tel: 01825-768050
Fax: 01825-749472
Email: MicroAPL@microapl.demon.co.uk
Web: www.microapl.co.uk/apl

Soliton Associates Ltd
Havsgårdsvej 4
2900 Hellerup
Denmark
Tel: +45 2624 6099
Email: sales@soliton.com

APL2000
Rapid Application Development
6610 Rockledge Drive, Suite 502
Bethesda MD 20817, USA
Email: sales@apl2000.com
Web: www.apl2000.com